Linux & Server Certification Prep Guide

Red Hat RHCSA · Linux Foundation LFCS · AWS SysOps Administrator · Cisco NetAcad Linux

Prepared for Rizwan Khalid · September 2026

Study plans, hands-on labs, original practice questions and readiness checklists. Prepared with AI assistance (Claude). Every exam and badge assessment is taken by you, under each provider's rules. Exam facts were checked against official pages in September 2026; confirm prices and dates before booking.

Contents

Overview: your path through the four credentials 4

Recommended order . . . 4

How to use each chapter . . . 4

Ground rules that protect your certificates . . . 4

Chapter 1: Red Hat RHCSA (EX200) 5

At a glance . . . 5

Objectives map . . . 5

Study plan . . . 6

Lab setup . . . 6

Hands-on tasks . . . 8

Practice questions . . . 21

Exam-day tips . . . 23

Readiness checklist . . . 24

Sources . . . 24

Chapter 2: Linux Foundation LFCS 26

At a glance . . . 26

Objectives map . . . 26

Study plan . . . 27

Lab setup . . . 28

Hands-on tasks . . . 29

Practice questions . . . 48

Exam-day tips . . . 50

Readiness checklist . . . 51

Sources . . . 52

Chapter 3: AWS Certified SysOps Administrator - Associate 54

At a glance . . . 54

Objectives map . . . 55

Study plan . . . 55

Lab setup . . . 56

Hands-on tasks . . . 58

Practice questions . . . 76

Exam-day tips . . . 81

Readiness checklist . . . 81

Sources . . . 82

Chapter 4: Cisco NetAcad Linux Unhatched & Linux Essentials 84

At a glance . . . 84

Objectives map . . . 85

Study plan . . . 85

Lab setup . . . 86

Hands-on tasks . . . 86

Practice questions . . . 93

Exam-day tips . . . 95

Readiness checklist . . . 96

Sources . . . 96

Overview: your path through the four credentials

You already run production Linux: Node.js services under PM2, Nginx reverse proxies, Docker, cron jobs and SSH on live servers. This guide turns that experience into credentials employers recognize. It is ordered so each step builds on the last and gives you a visible result early.

You take every exam and assessment yourself. This guide gives you the plan, the labs and the practice. The badges only mean something because you earned them.

Recommended order

Step	Credential	Why this order	Rough time
1	Cisco NetAcad Linux Unhatched / Essentials	Free quick win; badge on Credly within weeks	2-3 weeks
2	Linux Foundation LFCS	Ubuntu-based, closest to your servers	6-8 weeks
3	Red Hat RHCSA	The most respected; adds the RHEL skills	6-8 weeks
4	AWS SysOps Administrator - Associate	Puts your Linux ops on AWS	6-8 weeks

LFCS and RHCSA overlap by roughly two thirds. Doing them back to back means the second one mostly covers the distro differences: apt vs dnf, netplan vs nmcli, ufw vs firewalld, AppArmor vs SELinux.

How to use each chapter

1. Read **At a glance** and **Objectives map**. The "Your current strength" column shows where your job already covers you (Strong) and where to spend your time (Refresh, New).
2. Build the lab from **Lab setup** once, then work through the **Hands-on tasks** without looking at the solutions. Check yourself with the **Verify** commands.
3. Do the **Practice questions** in one sitting, timed. Anything you miss goes back into the week's plan.
4. Book the exam only when every box in the **Readiness checklist** is ticked.

Ground rules that protect your certificates

- Use only official objectives, official docs and original practice material. Never use brain-dump sites: they break every provider's terms and can get a certification revoked.
- Performance-based exams (LFCS, RHCSA) test what you can actually do on a live system, so the labs matter more than the reading.
- Check prices, versions and retake rules on the official pages before booking. Each chapter lists its sources and when they were checked.
- Add each badge to LinkedIn (Licenses & certifications) and your GitHub profile README once it's issued, using the verification link Credly provides.

Chapter 1: Red Hat RHCSA (EX200)

This guide prepares you; you sit the exam and every assessment yourself.

As of September 2026.

Version alert: this chapter follows RHEL 9 conventions, as requested. But the official EX200 page now says the exam is **based on RHEL 10**. Its current objective list adds Flatpak and systemd timers and no longer lists containers. Whether a RHEL 9 version of EX200 can still be booked is **unverified**. Check the version on the booking screen before you pay. Almost every command below works the same on RHEL 9 and RHEL 10. Tasks that apply to only one version are labelled.

At a glance

Fact	Detail
Credential	Red Hat Certified System Administrator (RHCSA)
Exam code	EX200
Current exam version	Based on RHEL 10 (official page, Sept 2026)
RHEL 9 version still bookable	Unverified - confirm at scheduling
Format	100% hands-on tasks on live systems; no multiple choice
Duration	3 hours (unverified - third-party sources)
Price	About US$500 list; varies by region (unverified)
Passing score	210 of 300 (unverified - community-reported)
Delivery	Remote exam (from home) or Red Hat testing station
Remote kit	Gov photo ID, bootable Live USB, external webcam, mic
Allowed resources	Docs on the exam system only (man, /usr/share/doc)
Not allowed	Internet, notes, books, own documentation
Results	By email within 5 business days; per-area breakdown
Retake	One free retake per paid first attempt
Validity	Current for 3 years from the date earned
Prerequisite	None required; RH124 + RH134 or RH199 suggested

Why it fits you: you already run Linux in production, so the RHCSA builds on skills you have instead of starting from zero. The main work is learning the RHEL-specific tools you rarely touch on Ubuntu (SELinux, LVM, firewalld, nmcli, autofs, tuned). It is also the prerequisite for RHCE (Ansible), which fits naturally with the deployment automation you already do.

Objectives map

The areas below follow the official EX200 objective list, reworded. The last row applies only if you get the RHEL 9 version of the exam.

Objective area	What you must be able to do	Your strength
Essential tools	Shell, redirection, grep/regex, ssh, su	Strong
Essential tools (files)	tar/gzip/bzip2, links, ugo/rwx, man/info	Strong
Manage software	Configure dnf repos; install/remove RPMs	Refresh
Manage software (Flatpak)	Add Flatpak remotes; install/remove apps	New
Simple shell scripts	if/test/[ ], for loops, args, command output	Strong
Operate running systems	Boot, reboot, targets, interrupt boot	New
Operate running systems	Kill/renice processes, tuned profiles	Refresh

Objective area	What you must be able to do	Your strength
Operate running systems	journalctl, persistent journal, scp/sftp	Refresh
Configure local storage	GPT partitions, PV/VG/LV, swap, UUID/label	New
Create file systems	vfat/ext4/xfs, NFS, autofs, grow LVs	New
File system permissions	Diagnose and fix permission problems	Refresh
Deploy and maintain	at, cron, systemd timers, services	Refresh
Deploy and maintain	Default target, chrony, dnf, GRUB changes	New
Basic networking	IPv4/IPv6 with nmcli, hostname, resolution	Refresh
Basic networking	Restrict access with firewalld	New
Users and groups	Users, groups, password aging, sudo	Refresh
Security	umask, SSH keys, firewalld	Refresh
Security (SELinux)	Modes, contexts, port labels, booleans	New
Containers (RHEL 9 only)	podman images, rootless, systemd, storage	Refresh

Study plan

Ten weeks at 5 to 8 hours a week. Each week, do the tasks for that area in the lab twice: once while reading, and once from memory a few days later.

Week	Focus	Tasks	Resources
1	Build lab, snapshots, local repo	Lab setup	Rocky/Alma docs, Hyper-V docs
2	Essential tools, docs, scripts	1-6, 10-11	man pages, RHEL 9 docs (free)
3	Software, Flatpak, users, sudo	7-9, 36-39	RHEL 9 "Managing software" docs
4	Boot targets, root reset, processes	12-18	RHEL 9 "Configuring basic sysadmin"
5	Partitions, LVM, swap	19-25	RHEL 9 "Managing storage devices"
6	File systems, NFS, autofs	26-30	RHEL 9 "Managing file systems"
7	Scheduling, chrony, GRUB, networking	31-35, 40-42	RHEL 9 networking docs
8	SELinux, firewalld, SSH, umask	43-48	RHEL 9 "Using SELinux" docs
9	Containers (if RHEL 9 exam) + weak areas	49-50	RHEL 9 "Building containers" docs
10	Two timed 3-hour mock runs + review	All	Your own task list; practice Qs

Resources:

- Free: the RHEL 9 product documentation at https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9 (use the RHEL 10 docs if you book the RHEL 10 exam).
- Free: the man pages and `/usr/share/doc` on your lab VM. These are the only resources you get in the exam, so practise using them.
- Free: Rocky Linux or AlmaLinux 9 ISOs (compatible rebuilds of RHEL, no account needed).
- Official (paid): RH124 and RH134, or the RH199 rapid-track course. You don't need these, given your background.
- Optional: the Red Hat Developer subscription gives a free RHEL licence, but it needs an account. That is your choice to make; the Rocky/Alma lab works without one.

Lab setup

You're on Windows 11 Pro, so Hyper-V is free and built in. WSL2 is not suitable: it has no GRUB, no boot targets and no real block devices, so you couldn't practise the root reset, LVM or fstab. Build two VMs:

- **server** (192.168.50.1): serves a dnf repo, an NFS export and NTP. You don't need to be careful with this one.
- **client** (192.168.50.10): the "exam box". Has two extra blank disks. Take a snapshot when it's clean and roll back to it often.

Step 1: enable Hyper-V (in an admin PowerShell, then reboot).

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
```

Step 2: download the Rocky Linux 9 DVD ISO (https://rockylinux.org/download) or AlmaLinux 9 (https://almalinux.org/get-almalinux/) to `C:\ISO\`.

Step 3: create a private lab switch and both VMs (admin PowerShell).

```
New-VMSwitch -Name LabNet -SwitchType Internal
$iso = "C:\ISO\Rocky-9-latest-x86_64-dvd.iso"
foreach ($n in "server","client") {
  New-VM -Name $n -MemoryStartupBytes 2GB -Generation 2 `
    -NewVHDPath "C:\VMs\$n.vhdx" -NewVHDSizeBytes 20GB -SwitchName "Default Switch"
  Set-VMProcessor -VMName $n -Count 2
  Set-VMFirmware -VMName $n -SecureBootTemplate MicrosoftUEFICertificateAuthority
  Add-VMNetworkAdapter -VMName $n -SwitchName LabNet
  Add-VMDvdDrive -VMName $n -Path $iso
  Set-VMFirmware -VMName $n -FirstBootDevice (Get-VMDvdDrive -VMName $n)
}
New-VHD -Path C:\VMs\client-d1.vhdx -SizeBytes 5GB -Dynamic
New-VHD -Path C:\VMs\client-d2.vhdx -SizeBytes 5GB -Dynamic
Add-VMHardDiskDrive -VMName client -Path C:\VMs\client-d1.vhdx
Add-VMHardDiskDrive -VMName client -Path C:\VMs\client-d2.vhdx
Start-VM server, client
```

Step 4: install both VMs as "Server with GUI" or "Minimal Install", and set a root password. After installing, remove the DVD from the boot order (keep the ISO attached to **server** for the repo), then take a snapshot:

```
Checkpoint-VM -Name client -SnapshotName clean
# roll back later with:
Restore-VMSnapshot -VMName client -Name clean -Confirm:$false
```

Step 5: on **server**, give the LabNet NIC a static address and publish the DVD as a repo over HTTP. Run `nmcli dev` to find the name of the second NIC (for example `eth1`).

```
nmcli con add con-name labnet ifname eth1 type ethernet \
  ipv4.method manual ipv4.addresses 192.168.50.1/24
nmcli con up labnet
mkdir -p /var/www/html/repo
echo '/dev/sr0 /var/www/html/repo iso9660 ro,nofail 0 0' >> /etc/fstab
systemctl daemon-reload && mount -a
dnf install -y httpd nfs-utils chrony
systemctl enable --now httpd
firewall-cmd --permanent --add-service={http,nfs,rpc-bind,mountd,ntp}
firewall-cmd --reload
```

Step 6: on **server**, create NFS exports for the NFS and autofs tasks.

```
mkdir -p /export/share /rhome/{alice,bob}
echo "hello from nfs" > /export/share/readme.txt
useradd -u 3001 alice; useradd -u 3002 bob
chown alice: /rhome/alice; chown bob: /rhome/bob
cat > /etc/exports <<'EOF'
/export/share 192.168.50.0/24(rw,sync)
/rhome        192.168.50.0/24(rw,sync)
EOF
systemctl enable --now nfs-server
exportfs -rv
```

Step 7: on **server**, turn it into an NTP source for the lab.

```
echo 'allow 192.168.50.0/24' >> /etc/chrony.conf
echo 'local stratum 8' >> /etc/chrony.conf
systemctl restart chronyd
```

Step 8: on **client**, disable the installer's DVD repos by adding `enabled=0` to the Rocky repo files in `/etc/yum.repos.d/`. Then configure the lab repos yourself (Task 7). That matches the exam, where you're usually

given repo URLs.

Disk names: Hyper-V shows extra disks as `/dev/sdb` and `/dev/sdc`. The exam usually uses `/dev/vdb` (KVM). The tasks below use `/dev/vdb` and `/dev/vdc`, so substitute your own names (check with `lsblk`).

Hands-on tasks

Do these tasks as root on **client** unless a task says otherwise. The exam requires that every change **survives a reboot**, so reboot and re-verify after each block of tasks.

Task 1: Find files by owner and copy them

Task: Find all regular files owned by user `alice` anywhere on the system and copy them, keeping their directory structure and attributes, into `/root/alice-files`.

Solution:

```
useradd alice 2>/dev/null; touch /tmp/a1; chown alice /tmp/a1
mkdir -p /root/alice-files
find / -user alice -type f -not -path '/proc/*' \
  -exec cp -a --parents {} /root/alice-files/ \; 2>/dev/null
```

Verify:

```
find /root/alice-files -type f
ls -l /root/alice-files/tmp/a1
```

Task 2: grep with regular expressions and redirection

Task: Write every line of `/etc/passwd` that belongs to a user with a 4-digit UID to `/root/uid4.txt`. Put every non-comment, non-blank line of `/etc/ssh/sshd_config` into `/root/sshd-active.txt`.

Solution:

```
grep -E '^[^:]+:[^:]*:[0-9]{4}:' /etc/passwd > /root/uid4.txt
grep -Ev '^[[:space:]]*(#|$)' /etc/ssh/sshd_config > /root/sshd-active.txt
```

Verify:

```
cat /root/uid4.txt
grep -c '^#' /root/sshd-active.txt   # expect 0
```

Task 3: Create compressed archives

Task: Create a gzip archive of `/etc` at `/root/etc.tar.gz` and a bzip2 archive at `/root/etc.tar.bz2`. Extract `etc/hosts` from the bzip2 archive into `/tmp/restore`.

Solution:

```
tar -czf /root/etc.tar.gz /etc
tar -cjf /root/etc.tar.bz2 /etc
mkdir -p /tmp/restore
tar -xjf /root/etc.tar.bz2 -C /tmp/restore etc/hosts
```

Verify:

```
file /root/etc.tar.gz /root/etc.tar.bz2
tar -tjf /root/etc.tar.bz2 | head
diff /etc/hosts /tmp/restore/etc/hosts && echo same
```

Task 4: Hard and soft links

Task: Create `/data/report.txt`, a hard link to it at `/data/report.hard`, and a symbolic link to it at `/root/report.link`.

Solution:

```
mkdir -p /data && echo "q3 numbers" > /data/report.txt
ln /data/report.txt /data/report.hard
ln -s /data/report.txt /root/report.link
```

Verify:

```
ls -li /data/report.txt /data/report.hard   # same inode, link count 2
```

```
ls -l /root/report.link                    # -> /data/report.txt
cat /root/report.link
```

Task 5: Standard permissions and ownership

Task: Create `/data/secret.txt`, owned by `alice` and group `wheel`. The owner can read and write it, the group can read it, and nobody else has any access.

Solution:

```
touch /data/secret.txt
chown alice:wheel /data/secret.txt
chmod 640 /data/secret.txt      # or: chmod u=rw,g=r,o= /data/secret.txt
```

Verify:

```
stat -c '%A %U %G %n' /data/secret.txt   # -rw-r----- alice wheel
```

Task 6: Find answers in the system documentation

Task: Without using the internet, find the man page that documents the `/etc/fstab` format, the config file example for `autofs`, and every man page related to `semanage`.

Solution:

```
man 5 fstab
ls /usr/share/doc/ | grep -i autofs; man 5 auto.master
man -k semanage          # if empty, rebuild the index: mandb
```

Verify:

```
whatis fstab
man -k semanage | head
```

Task 7: Configure dnf repositories

Task: Configure the system to use the repositories at `http://192.168.50.1/repo/BaseOS` and `http://192.168.50.1/repo/AppStream`, with GPG checking disabled.

Solution:

```
cat > /etc/yum.repos.d/lab.repo <<'EOF'
[lab-baseos]
name=Lab BaseOS
baseurl=http://192.168.50.1/repo/BaseOS
enabled=1
gpgcheck=0

[lab-appstream]
name=Lab AppStream
baseurl=http://192.168.50.1/repo/AppStream
enabled=1
gpgcheck=0
EOF
dnf clean all
```

Alternative: run `dnf config-manager --add-repo http://192.168.50.1/repo/BaseOS` (from dnf-plugins-core), then add `gpgcheck=0` to the file it generates.

Verify:

```
dnf repolist
dnf list available | head
```

Task 8: Install, query and remove packages

Task: Install the package that provides the `semanage` command and the `autofs` package. Remove `nano` if it is installed. Find out which package owns `/etc/chrony.conf`.

Solution:

```
dnf provides '*/semanage'          # policycoreutils-python-utils
dnf install -y policycoreutils-python-utils autofs
dnf remove -y nano
rpm -qf /etc/chrony.conf
```

Verify:

```
rpm -q policycoreutils-python-utils autofs
rpm -q nano        # "package nano is not installed"
dnf history | head -5
```

Task 9: Configure a Flatpak remote and install an app (RHEL 10 objective)

Task: Add the Flathub remote system-wide, install `org.gnome.Calculator` from it, then remove it. In the exam you'll be given the remote URL to use instead.

Solution:

```
dnf install -y flatpak
flatpak remote-add --if-not-exists flathub \
  https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install -y flathub org.gnome.Calculator
flatpak uninstall -y org.gnome.Calculator
```

Verify:

```
flatpak remotes --show-details
flatpak list --app     # empty after the uninstall
```

Task 10: Script with arguments, conditions and a loop

Task: Write `/usr/local/bin/mkusers`, which takes one argument: a file with one username per line. For each name it creates the user if the user doesn't exist, and otherwise prints `exists: NAME`. If the argument is missing or the file can't be read, it prints a usage message to stderr and exits with status 1.

Solution:

```
cat > /usr/local/bin/mkusers <<'EOF'
#!/bin/bash
if [ $# -ne 1 ] || [ ! -r "$1" ]; then
  echo "Usage: $0 USERFILE" >&2
  exit 1
fi
for u in $(cat "$1"); do
  if id "$u" &>/dev/null; then
    echo "exists: $u"
  else
    useradd "$u" && echo "created: $u"
  fi
done
EOF
chmod 755 /usr/local/bin/mkusers
```

Verify:

```
printf 'tom\njerry\nroot\n' > /tmp/names
mkusers /tmp/names          # created tom, created jerry, exists: root
mkusers; echo "rc=$?"       # usage, rc=1
```

Task 11: Script that processes command output

Task: Write `/usr/local/bin/bigfiles`, which lists every file larger than the size given as its first argument (for example `10M`) under `/var`, one per line, in the format `SIZE PATH`. Save its output for `5M` to `/root/big.txt`.

Solution:

```
cat > /usr/local/bin/bigfiles <<'EOF'
#!/bin/bash
limit=${1:-10M}
find /var -type f -size +"$limit" 2>/dev/null | while read -r f; do
  size=$(du -h "$f" | cut -f1)
  echo "$size $f"
done
EOF
chmod 755 /usr/local/bin/bigfiles
bigfiles 5M > /root/big.txt
```

Verify:

```
cat /root/big.txt
```

```
bash -n /usr/local/bin/bigfiles && echo "syntax ok"
```

Task 12: Set and switch boot targets

Task: Make the system boot into the text-mode multi-user target by default. Switch to it now without rebooting.

Solution:

```
systemctl get-default
systemctl set-default multi-user.target
systemctl isolate multi-user.target
```

Verify:

```
systemctl get-default                # multi-user.target
ls -l /etc/systemd/system/default.target
systemctl is-active graphical.target  # inactive
```

Task 13: Reset a forgotten root password (interrupt the boot)

Task: You don't know the root password. Reset it to `Lab-Root-2026` using the console, and keep SELinux working.

Solution:

```
1. Reboot. At the GRUB menu press an arrow key to stop the countdown.
   (If the menu is hidden, hold Shift or press Esc during boot.)
2. Highlight the default kernel entry and press e.
3. Go to the end of the line starting with "linux" and append:  rd.break
4. Press Ctrl+x to boot. You land in an emergency shell (switch_root:/#).
```

```
mount -o remount,rw /sysroot
chroot /sysroot
passwd root                 # type Lab-Root-2026 twice
touch /.autorelabel         # fixes the SELinux label on /etc/shadow
exit                        # leave the chroot
exit                        # continue booting; relabel then reboot happen
```

Verify:

```
# After the automatic relabel and reboot, log in as root with the new password.
ls -Z /etc/shadow          # system_u:object_r:shadow_t:s0
getenforce                 # Enforcing
```

Task 14: Boot once into rescue or emergency target

Task: For one boot only, start the system in `rescue.target` from GRUB. Then return to normal operation.

Solution:

```
At GRUB press e, append to the "linux" line:  systemd.unit=rescue.target
Press Ctrl+x, then enter the root password when asked.
(Use systemd.unit=emergency.target for an even more minimal shell,
 where / is mounted read-only.)
```

```
systemctl default          # from the rescue shell, continue to default target
```

Verify:

```
systemctl get-default      # unchanged: the edit was for one boot only
cat /proc/cmdline          # no systemd.unit= after a normal reboot
```

Task 15: Identify and stop processes

Task: Start `sleep 3000` as user `alice` in the background. Find the processes using the most CPU. Stop alice's sleep process gracefully, and force-kill it only if it ignores that.

Solution:

```
su - alice -c 'nohup sleep 3000 >/dev/null 2>&1 &'
ps -eo pid,user,%cpu,%mem,cmd --sort=-%cpu | head
pgrep -u alice -a sleep
pkill -TERM -u alice sleep
# only if still running:
pkill -KILL -u alice sleep
```

Verify:

```
pgrep -u alice sleep || echo "no sleep processes for alice"
```

Task 16: Adjust process scheduling priority

Task: Start `sha256sum /dev/zero` with a nice value of 10. Then change the running process to nice value 15.

Solution:

```
nice -n 10 sha256sum /dev/zero &
pid=$(pgrep -f 'sha256sum /dev/zero')
renice -n 15 -p "$pid"
```

Verify:

```
ps -o pid,ni,cmd -p "$pid"      # NI column shows 15
kill "$pid"
```

Task 17: Select a tuning profile

Task: Make sure `tuned` is running and enabled, and set the system to the profile it recommends.

Solution:

```
dnf install -y tuned
systemctl enable --now tuned
tuned-adm recommend             # e.g. virtual-guest
tuned-adm profile "$(tuned-adm recommend)"
```

Verify:

```
tuned-adm active
tuned-adm verify
systemctl is-enabled tuned
```

Task 18: Persistent journal and log searching

Task: Keep the systemd journal across reboots. Then show all messages of priority `err` or higher from the current boot, and the `sshd` messages from the last hour.

Solution:

```
mkdir -p /etc/systemd/journald.conf.d
cat > /etc/systemd/journald.conf.d/persist.conf <<'EOF'
[Journal]
Storage=persistent
EOF
systemctl restart systemd-journald
journalctl -b -p err
journalctl -u sshd --since "1 hour ago"
```

Verify:

```
ls -d /var/log/journal/*     # machine-id directory exists
# after a reboot:
journalctl --list-boots      # more than one boot listed
```

Task 19: Transfer files securely

Task: Copy `/etc/hosts` from the client to `/tmp/` on the server, pull `/export/share/readme.txt` back with sftp, and mirror `/etc/yum.repos.d` to the server with rsync over SSH.

Solution:

```
scp /etc/hosts root@192.168.50.1:/tmp/client-hosts
sftp root@192.168.50.1 <<'EOF'
get /export/share/readme.txt /root/readme.txt
EOF
dnf install -y rsync
rsync -av -e ssh /etc/yum.repos.d/ root@192.168.50.1:/tmp/repos/
```

Verify:

```
ssh root@192.168.50.1 'ls -l /tmp/client-hosts /tmp/repos'
cat /root/readme.txt
```

Task 20: Create GPT partitions

Task: On `/dev/vdb`, create a GPT label and three partitions: 1 GiB for XFS data, 512 MiB for swap, and the rest for LVM.

Solution:

```
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart data xfs 1MiB 1025MiB
parted -s /dev/vdb mkpart swap1 linux-swap 1025MiB 1537MiB
parted -s /dev/vdb mkpart lvm1 1537MiB 100%
parted -s /dev/vdb set 3 lvm on
udevadm settle
```

Verify:

```
parted -s /dev/vdb print
lsblk /dev/vdb
```

Task 21: Physical volumes, a volume group and a logical volume

Task: Create volume group `vgdata` on `/dev/vdb3` with a 16 MiB extent size. Create logical volume `lvdata` of exactly 20 extents, formatted as ext4 and mounted persistently at `/mnt/lvdata` by UUID.

Solution:

```
pvcreate /dev/vdb3
vgcreate -s 16M vgdata /dev/vdb3
lvcreate -l 20 -n lvdata vgdata          # 20 x 16 MiB = 320 MiB
mkfs.ext4 /dev/vgdata/lvdata
mkdir -p /mnt/lvdata
echo "UUID=$(blkid -s UUID -o value /dev/vgdata/lvdata) /mnt/lvdata ext4 defaults 0 0" \
  >> /etc/fstab
systemctl daemon-reload
mount -a
```

Verify:

```
vgdisplay vgdata | grep 'PE Size'     # 16.00 MiB
lvs -o lv_name,seg_size_pe vgdata     # 20
findmnt --verify
df -h /mnt/lvdata
```

Task 22: Mount a file system by label

Task: Format `/dev/vdb1` as XFS with the label `DATA` and mount it persistently at `/data1` using the label.

Solution:

```
mkfs.xfs -f -L DATA /dev/vdb1
mkdir -p /data1
echo 'LABEL=DATA /data1 xfs defaults 0 0' >> /etc/fstab
systemctl daemon-reload
mount -a
```

Verify:

```
blkid /dev/vdb1          # LABEL="DATA" TYPE="xfs"
findmnt /data1
```

Task 23: Add a swap partition without disturbing existing swap

Task: Turn `/dev/vdb2` into swap, activate it, and make it persistent. Leave the existing swap in place.

Solution:

```
mkswap /dev/vdb2
echo "UUID=$(blkid -s UUID -o value /dev/vdb2) none swap defaults 0 0" >> /etc/fstab
systemctl daemon-reload
swapon -a
```

Verify:

```
swapon --show            # original swap plus /dev/vdb2
free -h
```

Task 24: Add swap on a logical volume

Task: Add a 256 MiB logical volume `lvswap` in `vgdata` and use it as extra persistent swap.

Solution:

```
lvcreate -L 256M -n lvswap vgdata
mkswap /dev/vgdata/lvswap
echo '/dev/vgdata/lvswap none swap defaults 0 0' >> /etc/fstab
swapon -a
```

Verify:

```
swapon --show | grep dm-
lvs vgdata
```

Task 25: Remove a logical volume cleanly

Task: Create a temporary LV `lvtemp` (100 MiB, XFS, mounted at `/mnt/temp` in fstab), then delete it completely without leaving a broken fstab entry.

Solution:

```
lvcreate -L 100M -n lvtemp vgdata && mkfs.xfs /dev/vgdata/lvtemp
mkdir -p /mnt/temp
echo '/dev/vgdata/lvtemp /mnt/temp xfs defaults 0 0' >> /etc/fstab
systemctl daemon-reload && mount -a
# now remove it:
umount /mnt/temp
sed -i '\#/dev/vgdata/lvtemp#d' /etc/fstab
systemctl daemon-reload
lvremove -y /dev/vgdata/lvtemp
```

Verify:

```
lvs vgdata                # no lvtemp
grep lvtemp /etc/fstab || echo "fstab clean"
findmnt --verify
```

Task 26: Create and mount a VFAT file system

Task: Create a 200 MiB LV `lvvfat` in `vgdata`, format it as VFAT, and mount it persistently at `/mnt/vfat` so that user `alice` owns the files in it.

Solution:

```
dnf install -y dosfstools
lvcreate -L 200M -n lvvfat vgdata
mkfs.vfat /dev/vgdata/lvvfat
mkdir -p /mnt/vfat
echo "/dev/vgdata/lvvfat /mnt/vfat vfat uid=$(id -u alice),gid=$(id -g alice) 0 0" \
  >> /etc/fstab
systemctl daemon-reload && mount -a
```

Verify:

```
findmnt /mnt/vfat           # FSTYPE vfat
su - alice -c 'touch /mnt/vfat/ok && ls -l /mnt/vfat'
```

Task 27: Extend a logical volume and its file system

Task: Grow `lvdata` to 500 MiB, together with its ext4 file system, while it stays mounted. If the VG is short of space, add `/dev/vdc` to it first.

Solution:

```
vgs vgdata                          # check VFree
pvcreate /dev/vdc && vgextend vgdata /dev/vdc     # only if needed
lvextend -r -L 500M /dev/vgdata/lvdata
# -r resizes the file system too (resize2fs for ext4, xfs_growfs for xfs)
```

Verify:

```
lvs /dev/vgdata/lvdata
df -h /mnt/lvdata                   # about 480M usable
```

Task 28: Mount an NFS share persistently

Task: Mount `192.168.50.1:/export/share` at `/mnt/nfs` persistently.

Solution:

```
dnf install -y nfs-utils
showmount -e 192.168.50.1
mkdir -p /mnt/nfs
echo '192.168.50.1:/export/share /mnt/nfs nfs defaults,_netdev 0 0' >> /etc/fstab
systemctl daemon-reload
mount -a
```

Verify:

```
findmnt -t nfs,nfs4
cat /mnt/nfs/readme.txt
```

Task 29: Automount network home directories with autofs

Task: When a user logs in, `/rhome/USERNAME` must be automounted read-write from `192.168.50.1:/rhome/USERNAME`. Users `alice` (UID 3001) and `bob` (UID 3002) must exist locally with those home directories.

Solution:

```
dnf install -y autofs nfs-utils
useradd -u 3001 -M -d /rhome/alice alice 2>/dev/null \
  || usermod -u 3001 -d /rhome/alice alice
useradd -u 3002 -M -d /rhome/bob bob
echo '/rhome /etc/auto.rhome' > /etc/auto.master.d/rhome.autofs
echo '* -rw,sync 192.168.50.1:/rhome/&' > /etc/auto.rhome
systemctl enable --now autofs
```

If `alice` was created earlier with another UID, `usermod -u` changes it. Also fix the ownership of any files alice owns (see Task 1) with `find / -user OLDUID -exec chown alice {} +`.

Verify:

```
ls /rhome                    # empty until accessed
su - bob -c 'pwd; touch ~/x; ls -l ~'
findmnt /rhome/bob           # nfs4 mount appears
```

Task 30: Fix a permission problem on a shared directory

Task: Members of group `devs` (`tom`, `jerry`) must be able to create and edit files in `/srv/devs`. New files must belong to group `devs` automatically. Users outside the group get no access. Also give `alice` read-only access to the directory using an ACL.

Solution:

```
groupadd devs
usermod -aG devs tom; usermod -aG devs jerry
mkdir -p /srv/devs
chgrp devs /srv/devs
chmod 2770 /srv/devs                        # setgid + rwx for owner/group
setfacl -m u:alice:rx /srv/devs
setfacl -d -m u:alice:rx /srv/devs          # default ACL for new files
```

Verify:

```
ls -ld /srv/devs                            # drwxrws---+ root devs
su - tom -c 'touch /srv/devs/t1'; ls -l /srv/devs/t1   # group devs
su - alice -c 'ls /srv/devs; touch /srv/devs/no'      # ls works, touch denied
getfacl /srv/devs
```

Task 31: Schedule a recurring job with cron

Task: As user `tom`, run `logger "tom-cron"` at 14:23 every day. As root, run `/usr/bin/df -h > /root/df.log` every 15 minutes on weekdays.

Solution:

```
crontab -u tom -e
# add:  23 14 * * * /usr/bin/logger "tom-cron"
crontab -e
# add:  */15 * * * 1-5 /usr/bin/df -h > /root/df.log
```

Verify:

```
crontab -l -u tom
crontab -l
systemctl is-active crond
```

Task 32: Schedule a one-time job with at

Task: In five minutes, write the output of `date` to `/root/at.txt`, once.

Solution:

```
dnf install -y at
systemctl enable --now atd
echo 'date > /root/at.txt' | at now + 5 minutes
```

Verify:

```
atq                     # one job queued
at -c "$(atq | awk 'NR==1{print $1}')" | tail -3
# after 5 minutes:
cat /root/at.txt
```

Task 33: Schedule a job with a systemd timer (RHEL 10 objective)

Task: Every day at 02:00, archive `/etc` to `/root/etc-DATE.tgz` using a systemd service and timer called `etcbackup`. If the machine was off at 02:00, run the job at the next boot.

Solution:

```
cat > /etc/systemd/system/etcbackup.service <<'EOF'
[Unit]
Description=Archive /etc

[Service]
Type=oneshot
ExecStart=/bin/bash -c 'tar -czf /root/etc-$(date +%%F).tgz /etc'
EOF
cat > /etc/systemd/system/etcbackup.timer <<'EOF'
[Unit]
Description=Daily /etc archive

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
EOF
systemctl daemon-reload
systemctl enable --now etcbackup.timer
```

Verify:

```
systemctl list-timers etcbackup.timer
systemctl start etcbackup.service && ls -l /root/etc-*.tgz
systemd-analyze calendar '*-*-* 02:00:00'
```

Task 34: Manage services and start them at boot

Task: Install `httpd`, make it start at boot, and start it now. Stop and disable `cups` if it is present. Make sure nothing else can start `cups`.

Solution:

```
dnf install -y httpd
systemctl enable --now httpd
systemctl disable --now cups 2>/dev/null
systemctl mask cups 2>/dev/null
```

Verify:

```
systemctl is-enabled httpd; systemctl is-active httpd
systemctl is-enabled cups     # masked (or not found)
```

Task 35: Configure a time service client

Task: Sync time from `192.168.50.1` using chrony, and remove the default pool servers.

Solution:

```
dnf install -y chrony
sed -i 's/^pool /#pool /' /etc/chrony.conf
echo 'server 192.168.50.1 iburst' >> /etc/chrony.conf
systemctl enable chronyd
systemctl restart chronyd
timedatectl set-ntp true
```

Verify:

```
chronyc sources -v        # ^* 192.168.50.1 once synced
timedatectl               # System clock synchronized: yes
```

Task 36: Create users and groups

Task: Create group `sysadmins` with GID 4000. Create user `harry` with UID 2001 as a supplementary member of `sysadmins`, and user `sarah`, who has no interactive shell and is not in `sysadmins`. Set both passwords to `Lab-Pass-2026`.

Solution:

```
groupadd -g 4000 sysadmins
useradd -u 2001 -G sysadmins harry
useradd -s /sbin/nologin sarah
echo 'Lab-Pass-2026' | passwd --stdin harry
echo 'Lab-Pass-2026' | passwd --stdin sarah
```

Verify:

```
id harry                        # uid=2001 ... groups=...,4000(sysadmins)
getent passwd sarah             # ends in /sbin/nologin
getent group sysadmins
```

Task 37: Password aging

Task: Make `harry`'s password expire every 90 days with a 7-day warning, and force him to change it at his next login. Every user created from now on must get a maximum password age of 60 days.

Solution:

```
chage -M 90 -W 7 harry
chage -d 0 harry
sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS\t60/' /etc/login.defs
```

Verify:

```
chage -l harry
grep ^PASS_MAX_DAYS /etc/login.defs
useradd testage && chage -l testage | grep Maximum    # 60
```

Task 38: Modify and lock accounts

Task: Add `sarah` to group `devs` without removing her from any other group. Set her account to expire on 2026-12-31. Lock `tom`'s password.

Solution:

```
usermod -aG devs sarah
usermod -e 2026-12-31 sarah         # or: chage -E 2026-12-31 sarah
usermod -L tom                      # or: passwd -l tom
```

Verify:

```
id sarah
chage -l sarah | grep 'Account expires'
passwd -S tom                       # shows the password as locked
```

Task 39: Configure privileged access with sudo

Task: Members of `sysadmins` can run any command as root through sudo. User `sarah` can run only `/usr/bin/systemctl restart httpd`, without a password.

Solution:

```
echo '%sysadmins ALL=(ALL) ALL' > /etc/sudoers.d/sysadmins
echo 'sarah ALL=(root) NOPASSWD: /usr/bin/systemctl restart httpd' \
  > /etc/sudoers.d/sarah
chmod 440 /etc/sudoers.d/sysadmins /etc/sudoers.d/sarah
visudo -c
```

Verify:

```
sudo -l -U harry
sudo -l -U sarah
```

Task 40: Configure IPv4 and IPv6 with nmcli

Task: On the LabNet interface (`eth1`), set IPv4 `192.168.50.10/24` with gateway and DNS `192.168.50.1`, and IPv6 `fd00:50::10/64`. The connection must come up automatically at boot.

Solution:

```
nmcli dev status
nmcli con add con-name lab-static ifname eth1 type ethernet \
  ipv4.method manual ipv4.addresses 192.168.50.10/24 \
  ipv4.gateway 192.168.50.1 ipv4.dns 192.168.50.1 \
  ipv6.method manual ipv6.addresses fd00:50::10/64 \
  connection.autoconnect yes
nmcli con up lab-static
```

To change an existing connection, use `nmcli con mod NAME ...` and then `nmcli con up NAME`.

Verify:

```
ip -br addr show eth1
nmcli -f ipv4.addresses,ipv6.addresses,connection.autoconnect con show lab-static
ping -c2 192.168.50.1
```

Task 41: Hostname and name resolution

Task: Set the hostname to `client.lab.example.com`, and make `server.lab.example.com` (alias `server`) resolve to `192.168.50.1` locally.

Solution:

```
hostnamectl set-hostname client.lab.example.com
echo '192.168.50.1 server.lab.example.com server' >> /etc/hosts
```

Verify:

```
hostnamectl status | head -3
getent hosts server
cat /etc/resolv.conf     # DNS set by NetworkManager from Task 40
```

Task 42: Modify the boot loader

Task: Add the kernel argument `audit=1` to every installed kernel, and make the GRUB menu wait 10 seconds.

Solution:

```
grubby --update-kernel=ALL --args="audit=1"
sed -i 's/^GRUB_TIMEOUT=.*/GRUB_TIMEOUT=10/' /etc/default/grub
grub2-mkconfig -o /boot/grub2/grub.cfg
```

On RHEL 9 the right path is `/boot/grub2/grub.cfg` on both BIOS and UEFI; the EFI file is only a stub that points to it. On a UEFI VM whose menu is hidden, also run `grub2-editenv - unset menu_auto_hide`.

Verify:

```
grubby --info=DEFAULT | grep args
grep GRUB_TIMEOUT /etc/default/grub
# after reboot:
grep -o 'audit=1' /proc/cmdline
```

Task 43: Restrict access with firewalld

Task: Permanently allow HTTP and TCP port 8080 in the default zone. Reject SSH from `192.168.60.0/24` only.

Solution:

```
systemctl enable --now firewalld
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
  source address="192.168.60.0/24" service name="ssh" reject'
firewall-cmd --reload
```

Verify:

```
firewall-cmd --list-all
firewall-cmd --permanent --list-rich-rules
```

Task 44: Default file permissions (umask)

Task: New files created by `harry` in his login shells must be `640` and new directories `750`.

Solution:

```
echo 'umask 027' >> /home/harry/.bashrc
```

Verify:

```
su - harry -c 'umask; touch f; mkdir d; ls -ld f d'
# 0027, -rw-r----- f, drwxr-x--- d
```

Task 45: SSH key-based authentication

Task: Let root on the client log in to `harry@192.168.50.1` with an Ed25519 key and no password. On the client, turn off password logins for root over SSH.

Solution:

```
ssh root@192.168.50.1 'useradd harry; echo Lab-Pass-2026 | passwd --stdin harry'
ssh-keygen -t ed25519 -N '' -f ~/.ssh/id_ed25519
ssh-copy-id -i ~/.ssh/id_ed25519.pub harry@192.168.50.1
echo 'PermitRootLogin prohibit-password' > /etc/ssh/sshd_config.d/40-local.conf
sshd -t && systemctl reload sshd
```

sshd uses the first value it finds for a setting, and it reads the `sshd_config.d` files in lexical order. A low number such as `40-` therefore wins over the RHEL-provided `50-redhat.conf`.

Verify:

```
ssh -o BatchMode=yes harry@192.168.50.1 hostname
sshd -T | grep -i permitrootlogin
```

Task 46: Set the SELinux mode persistently

Task: SELinux is currently permissive. Make it enforcing now and after every reboot.

Solution:

```
setenforce 1
sed -i 's/^SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config
```

If `getenforce` says `Disabled`, the kernel might have been booted with `selinux=0`. Remove that with `grubby --update-kernel=ALL --remove-args=selinux`, run `touch /.autorelabel`, and reboot.

Verify:

```
getenforce                      # Enforcing
grep ^SELINUX= /etc/selinux/config
sestatus
```

Task 47: Fix SELinux file contexts for a web root

Task: Serve files from `/srv/web` with httpd. Make the SELinux context persistent so that a full relabel doesn't break it.

Solution:

```
mkdir -p /srv/web && echo "lab web" > /srv/web/index.html
cat > /etc/httpd/conf.d/srvweb.conf <<'EOF'
DocumentRoot "/srv/web"
<Directory "/srv/web">
    Require all granted
</Directory>
EOF
semanage fcontext -a -t httpd_sys_content_t '/srv/web(/.*)?'
restorecon -Rv /srv/web
systemctl restart httpd
```

Verify:

```
ls -Zd /srv/web /srv/web/index.html     # httpd_sys_content_t
semanage fcontext -l -C                 # your local rule
curl -s http://localhost/                # lab web
```

Task 48: SELinux port labels and booleans

Task: Make httpd also listen on TCP port 82 with SELinux enforcing, and open that port in the firewall. Allow httpd to serve users' `~/public_html` directories, and make that setting persistent.

Solution:

```
echo 'Listen 82' > /etc/httpd/conf.d/port82.conf
semanage port -a -t http_port_t -p tcp 82
firewall-cmd --permanent --add-port=82/tcp && firewall-cmd --reload
systemctl restart httpd
getsebool -a | grep httpd_enable_homedirs
setsebool -P httpd_enable_homedirs on
```

If something is denied, check `ausearch -m AVC -ts recent`. If `setroubleshoot-server` is installed, `journalctl -t setroubleshoot` shows a suggested fix.

Verify:

```
semanage port -l | grep '^http_port_t'   # includes 82
ss -tlnp | grep ':82'
curl -s http://localhost:82/
semanage boolean -l -C                   # httpd_enable_homedirs (on, on)
```

Task 49: Find, inspect and run container images (RHEL 9 exam version)

Task: As a regular user, find the UBI 9 httpd image, inspect it without pulling it, pull it, and run it detached as `web` on host port 8080. Serve content from `~/web` on the host, with the correct SELinux label.

Solution:

```
dnf install -y container-tools            # as root
# then SSH in as harry (not su), because rootless podman needs a real session:
ssh harry@localhost
podman search registry.access.redhat.com/ubi9/httpd
skopeo inspect docker://registry.access.redhat.com/ubi9/httpd-24
mkdir -p ~/web && echo "from container" > ~/web/index.html
podman run -d --name web -p 8080:8080 \
  -v ~/web:/var/www/html:Z registry.access.redhat.com/ubi9/httpd-24
```

Verify:

```
podman ps
curl -s http://localhost:8080/        # from container
ls -Zd ~/web                          # container_file_t
```

Task 50: Run a container as a systemd user service (RHEL 9 exam version)

Task: Make harry's `web` container start at boot as a systemd user service, even when harry is not logged in.

Solution:

```
# as harry, in an SSH session:
mkdir -p ~/.config/systemd/user
cd ~/.config/systemd/user
podman generate systemd --name web --new --files
```

```
podman stop web && podman rm web
systemctl --user daemon-reload
systemctl --user enable --now container-web.service
loginctl enable-linger harry
```

`podman generate systemd` prints a deprecation notice on later RHEL 9 releases, but it works. The newer method is a Quadlet file, `~/.config/containers/systemd/web.container`, followed by `systemctl --user daemon-reload`.

Verify:

```
systemctl --user is-enabled container-web.service
loginctl show-user harry -p Linger       # Linger=yes
# reboot, then without logging in as harry:
curl -s http://client:8080/
```

Practice questions

1. Which command makes the system boot to the text-mode target from now on?
2. In the `rd.break` root-password reset, what happens if you skip `touch /.autorelabel`?
3. After `rd.break` you run `passwd` and get "Authentication token manipulation error". Which step did you probably skip?
4. httpd returns 403 for files in `/srv/web`, and `ls -Z` shows `default_t`. Which two commands fix it persistently?
5. httpd won't start after you add `Listen 8888`, and the log says "Permission denied" on bind. SELinux is enforcing. What is the fix?
6. You ran `firewall-cmd --add-service=http` and it worked, but the rule is gone after a reboot. Why?
7. You ran `firewall-cmd --permanent --add-port=82/tcp`, but the port is still closed. What did you forget?
8. Which two approaches make the journal persistent?
9. Which command finds the package that provides the `semanage` binary?
10. What single command grows LV `/dev/vg1/lv1` by 500 MiB and grows its XFS file system too?
11. Can you shrink an XFS file system? What would you do if you needed it smaller?
12. Which mount option in `/etc/fstab` stops an NFS mount from being attempted before the network is up?
13. In the autofs map line `* -rw 10.0.0.5:/homes/&`, what do `*` and `&` mean?
14. Which command sets an existing user's maximum password age to 60 days?
15. Where do you set the default maximum password age for users created in the future?
16. What does the setgid bit do on a directory?
17. Give the numeric mode for `rwxrws---`.
18. With `umask 027`, what permissions do new files and new directories get?
19. Why does `ln /data/a /home/b` fail with "Invalid cross-device link" when `/home` is a separate file system?
20. What command changes the nice value of running PID 4321 to 10?
21. Which command shows the tuned profile recommended for this machine?
22. Write the cron schedule field for "02:30 every Monday to Friday".
23. What does `Persistent=true` do in a systemd timer?
24. In `chronyc sources` output, what does the `^*` marker mean?
25. Which command adds the kernel argument `quiet` to every installed kernel?
26. You changed `ipv4.addresses` with `nmcli con mod`, but `ip addr` still shows the old address. What next?
27. After you edit fstab and reboot, the system drops into emergency mode. How do you recover, and how could you have caught this before rebooting?

28. A rootless container service stops as soon as its user logs out. Which command fixes this?
29. `systemctl --user` fails with "Failed to connect to bus" after you ran `su - harry`. Why, and what should you do instead?
30. Which grep command prints the lines of a file that are neither comments nor blank?
31. Which tar command creates a bzip2 archive of `/etc` at `/root/etc.tar.bz2`?
32. In a bash script, what do `$#`, `$1` and `$?` hold?
33. Which sudoers line lets group `ops` run any command as root without a password, and where should you put it?
34. `getenforce` prints `Permissive`. Which two commands make it enforcing immediately and after reboots?
35. Which command adds a Flatpak remote named `flathub` system-wide from a `.flatpakrepo` URL?

Answers

1. `systemctl set-default multi-user.target`. It changes the `default.target` symlink, so it persists; `isolate` only switches the running system.
2. `/etc/shadow` is left with the wrong SELinux label, so logins fail after reboot while SELinux is enforcing. The relabel fixes the label on the next boot.
3. You didn't run `mount -o remount,rw /sysroot` (or didn't `chroot /sysroot`), so the root file system was still read-only.
4. `semanage fcontext -a -t httpd_sys_content_t '/srv/web(/.*)?'` and then `restorecon -Rv /srv/web`. `chcon` alone doesn't survive a relabel.
5. `semanage port -a -t http_port_t -p tcp 8888`, then restart httpd. Also open the port in firewalld if clients need to reach it.
6. Without `--permanent`, the change applies only to the runtime configuration. Add it again with `--permanent`, then run `--reload`, or run `firewall-cmd --runtime-to-permanent`.
7. `firewall-cmd --reload`. Permanent rules don't take effect until a reload or reboot.
8. Create `/var/log/journal` (with the default `Storage=auto`), or set `Storage=persistent` in journald.conf or a drop-in file. Then restart systemd-journald.
9. `dnf provides '*/semanage'`, which shows the package `policycoreutils-python-utils`.
10. `lvextend -r -L +500M /dev/vg1/lv1`. The `-r` option calls `xfs_growfs` for you.
11. No, XFS can only grow. Back it up, recreate it at the smaller size, and restore the data.
12. `_netdev`. It marks the mount as needing the network, so systemd orders it after the network is online.
13. `*` matches any key (the directory name being accessed), and `&` is replaced with that key in the source path.
14. `chage -M 60 USER`.
15. `PASS_MAX_DAYS` in `/etc/login.defs`. It applies only to users created after the change.
16. New files and subdirectories inherit the directory's group instead of the creator's primary group.
17. `2770`: setgid (2), then rwx (7), rwx (7) and no access (0).
18. Files get `640` (666 minus 027) and directories get `750` (777 minus 027).
19. A hard link is another directory entry for the same inode, and inode numbers are unique only within one file system. Use `ln -s` instead.
20. `renice -n 10 -p 4321`. Only root can lower a nice value.
21. `tuned-adm recommend`. Apply it with `tuned-adm profile NAME` and check it with `tuned-adm active`.

22. `30 2 * * 1-5`.
23. If the machine was off when the timer should have fired, the job runs once at the next boot. The timer's last run time is stored on disk.
24. `^` means the source is a server, and `*` means it is the source chrony is currently synchronised to.
25. `grubby --update-kernel=ALL --args="quiet"`.
26. Run `nmcli con up NAME` (or `nmcli dev reapply IFACE`). `con mod` only changes the saved profile.
27. Enter the root password in emergency mode, run `mount -o remount,rw /`, fix `/etc/fstab`, and reboot. Before rebooting, `systemctl daemon-reload`, then `mount -a` and `findmnt --verify` would have caught the error.
28. `loginctl enable-linger USER`. It keeps that user's systemd instance running without a login session.
29. `su` doesn't create a login session with its own user bus. Log in over SSH as that user (or at the console) instead.
30. `grep -Ev '^[[:space:]]*(#|$)' FILE`.
31. `tar -cjf /root/etc.tar.bz2 /etc`. Use `-z` for gzip and `-J` for xz.
32. `$#` is the number of arguments, `$1` is the first argument, and `$?` is the exit status of the last command.
33. `%ops ALL=(ALL) NOPASSWD: ALL` in a file under `/etc/sudoers.d/`. Check it with `visudo -c`.
34. `setenforce 1` (runtime), and set `SELINUX=enforcing` in `/etc/selinux/config` (persistent).
35. `flatpak remote-add --if-not-exists flathub URL`. System-wide is the default when you run it as root.

Exam-day tips

Delivery rules:

- Remote exam: you need a valid government-issued photo ID, a bootable Live USB that you prepare beforehand with Red Hat's tool, an external webcam, a microphone, a USB 3.0 port and stable internet. Do the compatibility check days ahead, not on exam morning.
- You get no internet, notes or books. You can use the documentation installed on the exam systems: `man`, `man -k`, `--help` and `/usr/share/doc`. Practise finding things there (Task 6), for example the `semanage-fcontext(8)` man page, which has examples.
- Everything you configure is graded after the exam, usually after the systems reboot. Anything that isn't persistent counts as not done.

Time management (the 3-hour length is unverified - check your booking):

- Read every task first. Do the "gatekeeper" tasks first: a root password reset if one is asked for, then networking and hostname, then the repos. Many later tasks depend on these.
- Next, do the quick, independent tasks (users, sudo, cron, tuned, chrony, SELinux booleans). Then do storage, which is the slowest and riskiest.
- Reboot at least twice: once around the middle and once about 20 minutes before the end. Then run a quick check of every task.
- Keep a scratch list of the tasks you've done and verified.

Common traps:

- An fstab typo drops the system into emergency mode. After every fstab edit, run `systemctl daemon-reload`, `mount -a` and `findmnt --verify`.
- A firewalld rule added without `--permanent`, or with `--permanent` but no `--reload`.
- `semanage fcontext` without `restorecon`, or `chcon` instead of `semanage`.
- `setsebool` without `-P`.

- Configuring a service without `systemctl enable`.
- Running `systemctl --user` after `su`; SSH in as the user instead. Forgetting `loginctl enable-linger`.
- Rootless volume mounts without `:Z`, which fail under SELinux.
- An autofs map under `/etc/auto.master.d/` without the `.autofs` extension, which is ignored.
- LV sizes given in extents (`-l`) versus MiB (`-L`). Read carefully and check the PE size.
- Losing the network with a bad `nmcli` change on the interface you're connected through. Use the console for network changes.
- Leaving a relabel for the end: `/.autorelabel` can make the next boot take several minutes.

Readiness checklist

- ■ Lab VMs built with snapshots, a local repo, an NFS export and an NTP source
- ■ Can reset the root password with `rd.break` from memory, first time
- ■ Can set, isolate and boot once into a target from GRUB
- ■ Can configure dnf repos from a URL and install by provides
- ■ Can add a Flatpak remote and install and remove an app
- ■ Can write a script with args, `if`, `for` and exit codes without looking anything up
- ■ Can kill and renice processes, and set a tuned profile
- ■ Journal persists across reboots, and I can filter by unit, priority and time
- ■ Can create GPT partitions, PVs, VGs (custom PE size) and LVs (by -l and -L)
- ■ Can mount by UUID and by label, and add swap on a partition and on an LV
- ■ Can create vfat, ext4 and xfs, and extend an LV with `-r`
- ■ Can mount NFS in fstab and set up an autofs wildcard map
- ■ Can fix shared-directory permissions with setgid and ACLs
- ■ Can schedule with cron, at and a systemd timer
- ■ Can configure a chrony client and add kernel args with grubby
- ■ Can set static IPv4 and IPv6, and the hostname, with nmcli and hostnamectl
- ■ Can use firewalld services, ports and rich rules permanently
- ■ Can manage users, groups, password aging and sudoers.d files
- ■ Can set a user's umask and set up SSH key login
- ■ Can manage SELinux modes, fcontext, port labels and booleans; can read AVCs
- ■ (RHEL 9 exam) Can run a rootless podman container as a user service with linger
- ■ Have done two full timed mock runs, with every task persisting after a reboot
- ■ Have confirmed the exam version (RHEL 9 or 10) and the duration on the booking page
- ■ Remote exam Live USB and hardware check passed at least 3 days before

Sources

- EX200 exam page (objectives, RHEL 10 basis, allowed resources): https://www.redhat.com/en/services/training/ex200-red-hat-certified-system-administrator-rhcsa-exam
- RHCSA certification page: https://www.redhat.com/en/services/certification/rhcsa

- Individual exams (remote or testing station, free retake, results in 5 days): https://www.redhat.com/en/services/certification/individual-exams
- Red Hat Certification program guide (3-year validity, retake policy, remote kit, results): https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_certification_program_guide/index
- RHEL 9 product documentation: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9
- Rocky Linux downloads: https://rockylinux.org/download
- AlmaLinux downloads: https://almalinux.org/get-almalinux/
- Flathub setup (remote URL used in Task 9): https://flathub.org/setup
- Unverified items: price (about US$500, varies by region), duration (3 hours, reported by third-party prep sites), passing score (210/300, community-reported), and whether a RHEL 9 version of EX200 is still bookable. None of these appear on the official pages checked.

Chapter 2: Linux Foundation LFCS

This guide prepares you; you book, sit and pass the exam yourself.

At a glance

As of September 2026 (verified on training.linuxfoundation.org and docs.linuxfoundation.org, see Sources).

Item	Detail
Credential	Linux Foundation Certified System Administrator (LFCS)
Current version	Competencies revised for exams from 11 May 2023
Format	Online, proctored, performance-based, all command line
Tasks	17-20 hands-on tasks on several remote hosts
Duration	2 hours
Passing score	67%
Price (exam only)	USD 445
Bundles	USD 645 with LFS207 course; USD 625 with THRIVE-ONE
Delivery	Remote, PSI Bridge secure browser, webcam proctor
Scheduling window	12 months from purchase
Retake	1 free retake per purchase, inside the same 12 months
Retake wait	No enforced wait; retake unlocks after No Pass is graded
Results	Emailed within 24 hours
Validity	2 years; renew by passing the exam again
Simulator	killer.sh, 2 sessions, 36 hours of access each
Allowed help	man pages, /usr/share/doc, distro packages
Distribution	Official: "independent of distribution-specific tasks"
Exam OS in practice	Ubuntu (per your brief); exact version unverified

Note on the distribution: the official LFCS page says the exam is distribution-independent and you no longer pick a platform. It does not name Ubuntu or a version, so "Ubuntu, version X" is unverified from official pages. This chapter practises on Ubuntu Server 24.04 LTS, which matches your production servers; the skills transfer to any recent Ubuntu.

Why it fits you: LFCS is the Ubuntu/Debian-flavoured counterpart to RHCSA and grades you on live systems, the way you already work. It rewards what you do daily (systemd, Nginx, SSH, Docker, cron) and forces you to close gaps in LVM, NFS, bonding, NAT, libvirt, AppArmor and LDAP. Two years of validity and a free retake keep the cost risk low.

Objectives map

The five official domains and their weights, with each competency reworded. Strength is judged from your profile.

Objective area	What you must be able to do	Your current strength
Ops Deployment (25%)	Set kernel parameters at runtime and persistently	Refresh
Ops Deployment	Find and fix misbehaving processes and services	Strong
Ops Deployment	Schedule jobs with cron, at and systemd timers	Strong
Ops Deployment	Search, install, pin, verify packages; add repos	Refresh
Ops Deployment	Recover from boot, OS and filesystem failures	Refresh
Ops Deployment	Create and manage VMs with libvirt (virsh)	New
Ops Deployment	Configure a container engine and run containers	Strong

Objective area	What you must be able to do	Your current strength
Ops Deployment	Enforce mandatory access control (SELinux, AppArmor)	New
Networking (25%)	Configure IPv4/IPv6 addresses and name resolution	Refresh
Networking	Set the clock and sync time with NTP servers	Refresh
Networking	Monitor and troubleshoot network traffic	Refresh
Networking	Configure the OpenSSH server and client	Strong
Networking	Packet filtering, port redirection and NAT	Refresh
Networking	Add static routes	New
Networking	Build bridge and bond interfaces	New
Networking	Run reverse proxies and load balancers	Strong
Storage (20%)	Create and grow LVM volumes	New
Storage	Work with virtual filesystems (tmpfs, /proc, /sys)	Refresh
Storage	Create, mount, check and repair filesystems	Refresh
Storage	Use NFS and network block devices	New
Storage	Add and manage swap	Refresh
Storage	Configure automounting (autofs, systemd)	New
Storage	Monitor storage performance	Refresh
Essential Commands (20%)	Basic Git: init, commit, branch, merge, remotes	Strong
Essential Commands	Create, configure and troubleshoot services	Strong
Essential Commands	Monitor system performance and services	Strong
Essential Commands	Find a service's limits, ports and dependencies	Refresh
Essential Commands	Troubleshoot full disks and inode exhaustion	Strong
Essential Commands	Create, inspect and deploy SSL/TLS certificates	Refresh
Users and Groups (10%)	Create and manage local users and groups	Strong
Users and Groups	Manage user and system-wide shell environment	Refresh
Users and Groups	Set per-user resource limits	Refresh
Users and Groups	Configure POSIX ACLs	Refresh
Users and Groups	Use LDAP accounts for login (SSSD)	New

Note: the official list says "SELinux" for mandatory access control, while Ubuntu ships AppArmor. Practise AppArmor as the Ubuntu default and keep SELinux basics warm (Task 12, and Chapter 1).

Study plan

Ten weeks at 6-8 hours a week. Weights drive the time split: Operations and Networking get the most.

Week	Focus	Hands-on tasks	Resources
1	Lab build, Git, baseline timed run	Lab setup, 34-35	Ubuntu Server docs, man git
2	Users, sudo, env, limits, ACLs	43-46	man useradd, limits.conf, setfacl
3	systemd services, timers, cron, processes	2-5, 36-40	man systemd.service, systemd.timer
4	apt, repos, sysctl, boot recovery	1, 6-8	man apt, sources.list, sysctl.d
5	netplan, resolved, time, SSH	13-17	netplan.readthedocs.io, man sshd_config
6	ufw, NAT, routes, bond, bridge, Nginx LB	18-23	man ufw, ufw-framework, nginx docs
7	Partitions, filesystems, LVM, swap	24-27, 31, 33	man lvm, lvextend, mkswap
8	NFS, NBD, autofs, I/O perf, LDAP	28-30, 32, 47	Ubuntu Server docs (NFS, SSSD, LDAP)
9	libvirt, Docker, AppArmor, SELinux, TLS	9-12, 41-42	man virsh, apparmor.d, openssl-x509

Week	Focus	Hands-on tasks	Resources
10	killer.sh session 1, fix gaps, session 2, book	Redo weakest 10	killer.sh (included with exam)

Free resources:

- LFS101 "Introduction to Linux" (free, Linux Foundation / edX)
- Ubuntu Server documentation: https://documentation.ubuntu.com/server/
- Netplan documentation: https://netplan.readthedocs.io/
- man pages and /usr/share/doc on your lab VMs (the same help you get in the exam)

Official, paid:

- LFS207 Linux System Administration Essentials (sold as a bundle with the exam)
- killer.sh LFCS simulator: two sessions come free with the exam purchase; save them for weeks 9-10

Weekly rhythm: 2 short weekday sessions (read plus one task each) and one long weekend session (4-6 tasks timed, then redo the misses from memory).

Lab setup

Target: two Ubuntu Server 24.04 LTS VMs on your Windows 11 machine, with spare disks and extra NICs. Cost: zero.

VM	Role	Host-only IP	Extra hardware
lfcs1	Main practice box	192.168.56.10	3 x 2 GB disks, 2 extra NICs
lfcs2	NFS/NBD/LDAP/Git server, client tests	192.168.56.11	none

Option A: VirtualBox (recommended; easy extra disks and NICs). Install VirtualBox 7.x, download the Ubuntu 24.04.x live server ISO from https://ubuntu.com/download/server, then run in PowerShell:

```
$VBox = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
$Iso  = "$HOME\Downloads\ubuntu-24.04.x-live-server-amd64.iso"   # set real name
$HostOnly = "VirtualBox Host-Only Ethernet Adapter"

foreach ($vm in "lfcs1","lfcs2") {
  $dir = "$HOME\VirtualBox VMs\$vm"
  & $VBox createvm --name $vm --ostype Ubuntu_64 --register
  & $VBox modifyvm $vm --memory 3072 --cpus 2 --nested-hw-virt on
  & $VBox modifyvm $vm --nic1 nat --nic2 hostonly --hostonlyadapter2 $HostOnly
  & $VBox storagectl $vm --name SATA --add sata --controller IntelAhci --portcount 6
  $sa = @("--storagectl","SATA","--device","0")
  & $VBox createmedium disk --filename "$dir\$vm-os.vdi" --size 25600
  & $VBox storageattach $vm @sa --port 0 --type hdd --medium "$dir\$vm-os.vdi"
  & $VBox storageattach $vm @sa --port 5 --type dvddrive --medium $Iso
}

# lfcs1 only: two NICs on an internal network (bond/bridge) and 3 spare disks
& $VBox modifyvm lfcs1 --nic3 intnet --intnet3 labnet --nic4 intnet --intnet4 labnet
foreach ($n in 1..3) {
  $d = "$HOME\VirtualBox VMs\lfcs1\data$n.vdi"
  & $VBox createmedium disk --filename $d --size 2048
  & $VBox storageattach lfcs1 @sa --port $n --type hdd --medium $d
}
& $VBox startvm lfcs1
& $VBox startvm lfcs2
```

Install Ubuntu Server on each VM: keep the default "Use an entire disk" with "Set up this disk as an LVM group" ticked, install OpenSSH server, skip snaps. Do not pick the minimized install (it strips man pages; if you did, run `sudo unminimize`).

After first boot, on both VMs:

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y man-db manpages vim git curl tree acl lvm2 xfsprogs \
  sysstat iotop tcpdump nfs-common apparmor-utils bash-completion
```

```
sudo hostnamectl set-hostname lfcs1.lab.local     # lfcs2.lab.local on the other
lsblk                                             # lfcs1: sda plus sdb, sdc, sdd
ip -br link                                       # enp0s3 (NAT), enp0s8, enp0s9, enp0s10
```

Set the host-only address once (Task 13 does it properly; this is enough to SSH in from Windows):

```
sudo ip addr add 192.168.56.10/24 dev enp0s8      # .11 on lfcs2
sudo ip link set enp0s8 up
```

Snapshots are your reset button. Power the VMs off and take a clean snapshot before you start the tasks:

```
& $VBox snapshot lfcs1 take clean-install
& $VBox snapshot lfcs2 take clean-install
# later, with the VM powered off:
& $VBox snapshot lfcs1 restore clean-install
```

Notes:

- If Hyper-V or WSL2 is enabled, VirtualBox still runs but nested KVM may be unavailable. Task 9 then falls back to `--virt-type qemu` (slow but fine for a tiny CirrOS guest).
- No spare disks? Loop devices work for Tasks 24-27: `truncate -s 2G /var/tmp/d1.img && sudo losetup -fP --show /var/tmp/d1.img`.
- Option B (no VirtualBox): `winget install Canonical.Multipass`, then `multipass launch 24.04 --name lfcs1 --cpus 2 --memory 3G --disk 25G`. Easy, but extra NICs and disks are harder, so use loop devices.
- For Task 12 (SELinux) reuse the Rocky/Alma 9 VM from Chapter 1.

Hands-on tasks

All tasks assume Ubuntu Server 24.04 LTS, run as a sudo user, lfcs1 = 192.168.56.10 and lfcs2 = 192.168.56.11. Every change must survive a reboot unless the task says otherwise. Task counts follow the weights: Operations 12, Networking 11, Storage 10, Essential Commands 9, Users and Groups 5.

Task 1: Kernel parameters, runtime and persistent

Task: On lfcs1, set `vm.swappiness` to 10 for the running system only. Make IPv4 forwarding and `net.core.somaxconn = 4096` persistent across reboots.

Solution:

```
sudo sysctl -w vm.swappiness=10                   # runtime only, gone after reboot

sudo tee /etc/sysctl.d/60-lfcs.conf >/dev/null <<'EOF'
net.ipv4.ip_forward = 1
net.core.somaxconn = 4096
EOF
sudo sysctl --system                              # load every sysctl.d file now
```

Verify:

```
sysctl vm.swappiness net.ipv4.ip_forward net.core.somaxconn
grep -r somaxconn /etc/sysctl.d/
```

Task 2: Troubleshoot a runaway process

Task: A CPU hog is running. Find the process using the most CPU, lower its priority to nice 15, then stop it gracefully. Start `tar` backups at nice 10 from now on.

Solution:

```
yes > /dev/null &                                 # simulate the hog
ps -eo pid,user,ni,pcpu,comm --sort=-pcpu | head -5
PID=$(pgrep -x yes)
sudo renice -n 15 -p "$PID"
kill -TERM "$PID"                                 # use -KILL only if TERM fails
nice -n 10 tar czf /tmp/etc.tgz /etc 2>/dev/null
```

Verify:

```
pgrep -x yes || echo "no hog running"
ps -o pid,ni,comm -p "$PID" 2>/dev/null || echo "process gone"
```

Task 3: Fix a service that will not start

Task: Nginx on lfcs1 fails to start. Find the cause from the logs, fix it, and make sure it starts at boot.

Solution:

```
# setup the fault
sudo apt install -y nginx
echo "server { listen 80 }" | sudo tee /etc/nginx/conf.d/broken.conf
sudo systemctl restart nginx                    # fails

# diagnose
systemctl status nginx --no-pager
journalctl -u nginx -b --no-pager | tail -20
sudo nginx -t                                   # points at broken.conf (missing ;)
sudo ss -tlnp 'sport = :80'                     # also rule out a port clash

# fix
sudo rm /etc/nginx/conf.d/broken.conf
sudo nginx -t && sudo systemctl enable --now nginx
```

Verify:

```
systemctl is-active nginx; systemctl is-enabled nginx
curl -sI http://localhost | head -1
```

Task 4: Schedule jobs with cron and at

Task: As user `deploy`, run `/usr/local/bin/backup.sh` at 02:30 every weekday. Add a system cron job that runs `/usr/local/bin/cleanup.sh` as root every 15 minutes. Schedule a one-off job with `at` that logs a message at 23:00 today.

Solution:

```
sudo useradd -m -s /bin/bash deploy
printf '#!/bin/bash\ndate >> /tmp/backup.log\n' | sudo tee /usr/local/bin/backup.sh
sudo cp /usr/local/bin/backup.sh /usr/local/bin/cleanup.sh
sudo chmod 755 /usr/local/bin/backup.sh /usr/local/bin/cleanup.sh

(sudo crontab -u deploy -l 2>/dev/null; echo "30 2 * * 1-5 /usr/local/bin/backup.sh") \
  | sudo crontab -u deploy -

echo "*/15 * * * * root /usr/local/bin/cleanup.sh" | sudo tee /etc/cron.d/cleanup
sudo chmod 644 /etc/cron.d/cleanup

sudo apt install -y at
echo 'logger "at job ran"' | at 23:00
```

Verify:

```
sudo crontab -u deploy -l
cat /etc/cron.d/cleanup
atq
```

Task 5: systemd timer instead of cron

Task: Create `logrotate-app.service` and a timer that runs it daily at 03:15. If the machine was off at that time, the job must run at next boot.

Solution:

```
sudo tee /etc/systemd/system/logrotate-app.service >/dev/null <<'EOF'
[Unit]
Description=Compress old app logs

[Service]
Type=oneshot
ExecStart=/usr/bin/find /var/log -name "*.log.1" -exec gzip -f {} +
EOF

sudo tee /etc/systemd/system/logrotate-app.timer >/dev/null <<'EOF'
[Unit]
Description=Daily app log compression
```

```
[Timer]
OnCalendar=*-*-* 03:15:00
Persistent=true

[Install]
WantedBy=timers.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now logrotate-app.timer
```

Verify:

```
systemctl list-timers logrotate-app.timer
systemd-analyze calendar "*-*-* 03:15:00"
sudo systemctl start logrotate-app.service && systemctl status logrotate-app --no-pager
```

Task 6: Package queries, pinning and verification

Task: Show the installed and candidate versions of `nginx`, hold it so upgrades skip it, find which package owns `/usr/bin/dig`, list files installed by `curl`, and check whether any file of the `nginx-common` package has been modified.

Solution:

```
apt-cache policy nginx
apt list -a nginx 2>/dev/null
sudo apt-mark hold nginx

dpkg -S /usr/bin/dig                            # works for installed packages
sudo apt install -y apt-file && sudo apt-file update
apt-file search bin/dig                         # works for packages not installed

dpkg -L curl
sudo dpkg --verify nginx-common                 # no output = files unchanged
```

Verify:

```
apt-mark showhold
sudo apt upgrade -s | grep -i nginx || echo "nginx held"
```

Task 7: Add a third-party APT repository the modern way

Task: Add the official nginx.org stable repository for Ubuntu 24.04 using a dedicated keyring and a deb822 `.sources` file. Do not install from it; just make the packages visible.

Solution:

```
sudo apt-mark unhold nginx
curl -fsSL https://nginx.org/keys/nginx_signing.key \
  | sudo gpg --dearmor -o /usr/share/keyrings/nginx.gpg

sudo tee /etc/apt/sources.list.d/nginx.sources >/dev/null <<'EOF'
Types: deb
URIs: http://nginx.org/packages/ubuntu
Suites: noble
Components: nginx
Signed-By: /usr/share/keyrings/nginx.gpg
EOF
sudo apt update
```

Verify:

```
apt-cache policy nginx | grep -A1 nginx.org
ls /etc/apt/sources.list.d/
```

Clean up afterwards if you prefer the Ubuntu build: `sudo rm /etc/apt/sources.list.d/nginx.sources && sudo apt update`.

Task 8: Recover from a bad fstab and a damaged filesystem

Task: A typo in `/etc/fstab` could stop lfcs1 booting. Make an optional data mount non-fatal, check and repair an unmounted ext4 filesystem, and know how to get a root shell on Ubuntu when root is locked.

Solution:

```
# 1. Validate fstab before any reboot
sudo findmnt --verify --verbose
# optional mounts get nofail so boot continues if the device is missing:
# UUID=...  /data1  ext4  defaults,nofail  0  2

# 2. Check/repair an unmounted filesystem (use a loop device to practise)
truncate -s 200M /var/tmp/fs.img
LOOP=$(sudo losetup -f --show /var/tmp/fs.img)
sudo mkfs.ext4 -q "$LOOP"
sudo e2fsck -f -y "$LOOP"                       # never on a mounted filesystem
sudo losetup -d "$LOOP"

# 3. Root shell when boot hangs (console access):
# hold Shift (BIOS) or Esc (UEFI) -> GRUB -> press e on the Ubuntu entry
# append to the line starting with "linux":   init=/bin/bash
# Ctrl+X to boot, then:
#   mount -o remount,rw /
#   vim /etc/fstab            # fix the entry
#   sync; exec /sbin/init     # or: reboot -f
```

Verify:

```
sudo findmnt --verify && echo "fstab OK"
sudo mount -a && echo "all mounts OK"
```

Why `init=/bin/bash`: Ubuntu locks the root password, so `rescue.target` and `emergency.target` refuse to open a shell.

Task 9: Manage a virtual machine with libvirt

Task: Install KVM/libvirt on lfcs1, import the CirrOS test image as VM `vm1` with 512 MiB RAM and 1 vCPU on the `default` network, make it start at host boot, then raise its memory to 768 MiB in the persistent config.

Solution:

```
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients virtinst
sudo usermod -aG libvirt,kvm "$USER"             # log out/in, or use sudo below
kvm-ok 2>/dev/null || ls /dev/kvm                # no /dev/kvm -> add --virt-type qemu

cd /var/lib/libvirt/images
sudo curl -LO https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img
sudo cp cirros-0.6.2-x86_64-disk.img vm1.qcow2

sudo virsh net-start default 2>/dev/null; sudo virsh net-autostart default
sudo virt-install --name vm1 --memory 512 --vcpus 1 \
  --disk /var/lib/libvirt/images/vm1.qcow2 --import \
  --osinfo detect=on,require=off --network network=default \
  --graphics none --noautoconsole

sudo virsh autostart vm1
sudo virsh shutdown vm1
sudo virsh setmaxmem vm1 768M --config
sudo virsh setmem vm1 768M --config
sudo virsh start vm1
```

Verify:

```
sudo virsh list --all
sudo virsh dominfo vm1 | grep -E 'State|Max memory|Autostart|CPU\(s\)'
sudo virsh domifaddr vm1
```

Other verbs worth knowing: `virsh console vm1` (exit with Ctrl+]), `virsh destroy` (hard stop), `virsh undefine vm1 --remove-all-storage`, `virsh edit vm1`.

Task 10: Run and build containers with Docker

Task: Install Docker from the Ubuntu archive. Run `nginx:alpine` as container `web`, publishing host port 8080, serving `/srv/web` read-only, and restarting unless stopped. Build an image `api:1.0` from a Dockerfile for a small Node app and run it on port 3000, bound to localhost only.

Solution:

```
sudo apt install -y docker.io
```

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"                  # new login needed

sudo mkdir -p /srv/web && echo "hello lfcs" | sudo tee /srv/web/index.html
sudo docker run -d --name web -p 8080:80 --restart unless-stopped \
  -v /srv/web:/usr/share/nginx/html:ro nginx:alpine

mkdir -p ~/api && cd ~/api
cat > server.js <<'EOF'
require('http').createServer((q, s) => s.end('api ok\n')).listen(3000);
EOF
cat > Dockerfile <<'EOF'
FROM node:20-alpine
WORKDIR /app
COPY server.js .
EXPOSE 3000
CMD ["node", "server.js"]
EOF
sudo docker build -t api:1.0 .
sudo docker run -d --name api -p 127.0.0.1:3000:3000 --restart unless-stopped api:1.0
```

Verify:

```
sudo docker ps --format '{{.Names}} {{.Ports}} {{.Status}}'
sudo docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' web
curl -s localhost:8080; curl -s localhost:3000
```

Trap: ports published with `-p 8080:80` bypass ufw rules because Docker writes its own iptables chains. Bind to `127.0.0.1` or filter in the `DOCKER-USER` chain. Podman is also in the archive (`apt install podman`) if a task asks for it; the CLI is the same.

Task 11: Confine a script with AppArmor

Task: Write an AppArmor profile for `/usr/local/bin/report.sh`. The script may append to `/var/log/report.log` and run `date`, but must be denied reading `/etc/hostname`. Load it in enforce mode, then show how to switch it to complain mode.

Solution:

```
sudo tee /usr/local/bin/report.sh >/dev/null <<'EOF'
#!/bin/bash
date >> /var/log/report.log
cat /etc/hostname
EOF
sudo chmod 755 /usr/local/bin/report.sh

sudo tee /etc/apparmor.d/usr.local.bin.report.sh >/dev/null <<'EOF'
#include <tunables/global>

/usr/local/bin/report.sh {
  #include <abstractions/base>
  #include <abstractions/bash>
  #include <abstractions/consoles>

  /usr/local/bin/report.sh r,
  /usr/bin/bash ix,
  /usr/bin/date ix,
  /usr/bin/cat ix,
  /var/log/report.log w,
}
EOF
sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.report.sh

# switch modes (from apparmor-utils)
sudo aa-complain /usr/local/bin/report.sh
sudo aa-enforce  /usr/local/bin/report.sh
```

Verify:

```
sudo aa-status | grep -A2 report
sudo /usr/local/bin/report.sh                  # cat: /etc/hostname: Permission denied
tail -1 /var/log/report.log                    # date line was written
sudo journalctl -k --since "5 min ago" | grep 'apparmor="DENIED"'
```

If other harmless denials appear, `sudo aa-logprof` walks you through adding rules.

Task 12: SELinux basics (Rocky/Alma 9 VM)

Task: The official objective names SELinux. On an RHEL-family VM, serve web content from `/srv/web` under Apache, let httpd connect to a backend, and add port 8081 to the web port type.

Solution:

```
sudo dnf install -y httpd policycoreutils-python-utils
sudo mkdir -p /srv/web && echo ok | sudo tee /srv/web/index.html
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/web(/.*)?"
sudo restorecon -Rv /srv/web
sudo setsebool -P httpd_can_network_connect on
sudo semanage port -a -t http_port_t -p tcp 8081
```

Verify:

```
getenforce
ls -Zd /srv/web
getsebool httpd_can_network_connect
sudo semanage port -l | grep ^http_port_t
```

Task 13: Static IPv4 and IPv6 with netplan

Task: On lfcs1, give `enp0s8` the static addresses 192.168.56.10/24 and fd00:56::10/64, DNS 192.168.56.11 then 1.1.1.1, search domain `lab.local`. Keep DHCP on `enp0s3`. Apply without locking yourself out.

Solution:

```
sudo tee /etc/netplan/60-hostonly.yaml >/dev/null <<'EOF'
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s8:
      dhcp4: false
      dhcp6: false
      accept-ra: false
      addresses:
        - 192.168.56.10/24
        - "fd00:56::10/64"
      nameservers:
        addresses: [192.168.56.11, 1.1.1.1]
        search: [lab.local]
EOF
sudo chmod 600 /etc/netplan/60-hostonly.yaml
sudo netplan generate                           # syntax check
sudo netplan try                                # auto-rollback after 120 s
sudo netplan apply
```

Verify:

```
ip -br addr show enp0s8
networkctl status enp0s8 --no-pager
resolvectl dns enp0s8; resolvectl domain enp0s8
ping -c1 192.168.56.11; ping -6 -c1 fd00:56::11   # after lfcs2 gets .11 / ::11
```

Repeat on lfcs2 with .11 and ::11. Do not add a default route on enp0s8; the NAT NIC already has one.

Task 14: Hostname and name resolution

Task: Set the hostname to `lfcs1.lab.local`. Make `lfcs2` and `lfcs2.lab.local` resolve to 192.168.56.11 locally. Configure systemd-resolved globally to use 9.9.9.9 as a fallback DNS server.

Solution:

```
sudo hostnamectl set-hostname lfcs1.lab.local
echo "192.168.56.11 lfcs2.lab.local lfcs2" | sudo tee -a /etc/hosts

sudo mkdir -p /etc/systemd/resolved.conf.d
sudo tee /etc/systemd/resolved.conf.d/lab.conf >/dev/null <<'EOF'
[Resolve]
FallbackDNS=9.9.9.9
```

```
EOF
sudo systemctl restart systemd-resolved
```

Verify:

```
hostnamectl --static
getent hosts lfcs2                              # uses nsswitch: files then dns
resolvectl status | head -20
resolvectl query ubuntu.com
ls -l /etc/resolv.conf                          # symlink to the 127.0.0.53 stub
```

Task 15: Time zone and NTP synchronisation

Task: Set the time zone to Asia/Karachi. Make systemd-timesyncd use `ntp.ubuntu.com` and `0.pool.ntp.org`. Then, on lfcs2, replace timesyncd with chrony and let the 192.168.56.0/24 network use it as an NTP server.

Solution:

```
# lfcs1: timesyncd (Ubuntu 24.04 default)
sudo timedatectl set-timezone Asia/Karachi
sudo mkdir -p /etc/systemd/timesyncd.conf.d
sudo tee /etc/systemd/timesyncd.conf.d/lab.conf >/dev/null <<'EOF'
[Time]
NTP=ntp.ubuntu.com 0.pool.ntp.org
EOF
sudo timedatectl set-ntp true
sudo systemctl restart systemd-timesyncd

# lfcs2: chrony as server (installing chrony removes systemd-timesyncd)
sudo apt install -y chrony
echo "allow 192.168.56.0/24" | sudo tee /etc/chrony/conf.d/lab-server.conf
sudo systemctl restart chrony
sudo ufw allow from 192.168.56.0/24 to any port 123 proto udp
```

Verify:

```
timedatectl                                 # "System clock synchronized: yes"
timedatectl timesync-status                 # lfcs1
chronyc sources -v                          # lfcs2
sudo chronyc clients                        # lfcs2, once lfcs1 points at it
```

Task 16: Harden the OpenSSH server (socket-activated on 24.04)

Task: On lfcs1, make sshd listen on port 2222 (and 22 during the change), disable root login and password authentication. Keep the change in a drop-in file.

Solution:

```
sudo ufw allow 2222/tcp
sudo tee /etc/ssh/sshd_config.d/10-lfcs.conf >/dev/null <<'EOF'
Port 22
Port 2222
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
sudo sshd -t                                    # syntax check first
# Ubuntu 24.04 uses ssh.socket: ports are read at daemon-reload by a generator
sudo systemctl daemon-reload
sudo systemctl restart ssh.socket
```

Verify:

```
sudo ss -tlnp | grep -E ':(22|2222)\b'
sudo sshd -T | grep -Ei '^(port|permitrootlogin|passwordauthentication)'
ssh -p 2222 -o PasswordAuthentication=no localhost true && echo ok
```

Trap: sshd uses the first value it reads, and files in `sshd_config.d` load in name order. A cloud image's `50-cloud-init.conf` can set `PasswordAuthentication yes`, so your file must sort earlier (`10-`).

Task 17: SSH client keys and config

Task: From lfcs2, create an ed25519 key, install it for user `deploy` on lfcs1 (port 2222), and create a client alias `app1` so that `ssh app1` connects as deploy with that key.

Solution:

```
# on lfcs1 first: give deploy a temporary way in, or copy the key as your sudo user
# on lfcs2:
ssh-keygen -t ed25519 -f ~/.ssh/id_lab -N "" -C "lab key"
cat ~/.ssh/id_lab.pub | ssh -p 2222 youruser@192.168.56.10 \
  "sudo install -d -m 700 -o deploy -g deploy ~deploy/.ssh && \
   sudo tee -a ~deploy/.ssh/authorized_keys >/dev/null && \
   sudo chown deploy:deploy ~deploy/.ssh/authorized_keys && \
   sudo chmod 600 ~deploy/.ssh/authorized_keys"

cat >> ~/.ssh/config <<'EOF'
Host app1
    HostName 192.168.56.10
    Port 2222
    User deploy
    IdentityFile ~/.ssh/id_lab
    IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
```

Note: `~deploy` expands on lfcs1 because the whole command runs there. `ssh-copy-id -i ~/.ssh/id_lab -p 2222 deploy@host` is simpler when password login is still on.

Verify:

```
ssh app1 'whoami; hostname'
ssh -G app1 | grep -E '^(hostname|port|user|identityfile) '
```

Task 18: Packet filtering with ufw

Task: On lfcs1: deny incoming by default, allow outgoing, allow SSH (22 and 2222) only from 192.168.56.0/24, allow HTTP and HTTPS from anywhere, and rate-limit port 2222. Delete the rule for port 443 again, then log denied packets.

Solution:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp
sudo ufw limit from 192.168.56.0/24 to any port 2222 proto tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable                                  # answer y; SSH rules exist first
sudo ufw status numbered
sudo ufw delete allow 443/tcp                    # or: ufw delete <number>
sudo ufw logging low
```

Verify:

```
sudo ufw status verbose
sudo ufw show added
nc -zv 192.168.56.10 80                          # from lfcs2
sudo journalctl -k | grep 'UFW BLOCK' | tail -3
```

Remove the earlier broad `ufw allow 2222/tcp` rule if it is still listed, or the limit rule is moot.

Task 19: Port redirection and NAT (masquerade)

Task: On lfcs1, redirect incoming TCP 80 on `enp0s8` to local port 3000. Also make lfcs1 a NAT gateway: hosts on 10.50.0.0/24 (internal NICs) leave via `enp0s3` with masquerading. Everything must persist with ufw.

Solution:

```
# forwarding must be on (Task 1) and ufw must allow routed traffic
sudo sed -i 's/^DEFAULT_FORWARD_POLICY=.*/DEFAULT_FORWARD_POLICY="ACCEPT"/' \
  /etc/default/ufw

# add a nat table at the very top of /etc/ufw/before.rules (before *filter)
sudo cp /etc/ufw/before.rules /etc/ufw/before.rules.bak
sudo sed -i '1i\
*nat\
:PREROUTING ACCEPT [0:0]\
:POSTROUTING ACCEPT [0:0]\
```

```
-A PREROUTING -i enp0s8 -p tcp --dport 80 -j REDIRECT --to-ports 3000\
-A POSTROUTING -s 10.50.0.0/24 -o enp0s3 -j MASQUERADE\
COMMIT\
' /etc/ufw/before.rules

sudo ufw allow in on enp0s8 to any port 3000 proto tcp   # filter sees port 3000
sudo ufw reload

# a test backend on 3000
python3 -m http.server 3000 --directory /srv/web &
```

Verify:

```
sudo iptables -t nat -S | grep -E 'REDIRECT|MASQUERADE'
curl -s http://192.168.56.10/                    # run on lfcs2
sysctl net.ipv4.ip_forward
```

Trap: PREROUTING only sees packets arriving from the network. `curl localhost:80` on lfcs1 itself is not redirected; that needs an OUTPUT-chain rule.

Task 20: Static routes

Task: Traffic to 10.20.0.0/16 must go via 192.168.56.1 on `enp0s8`. Add it now for testing, then make it persistent with netplan.

Solution:

```
sudo ip route add 10.20.0.0/16 via 192.168.56.1 dev enp0s8     # runtime only

# persistent: add under enp0s8 in /etc/netplan/60-hostonly.yaml
#      routes:
#        - to: 10.20.0.0/16
#          via: 192.168.56.1
sudo netplan try && sudo netplan apply
```

The resulting `enp0s8` block:

```
    enp0s8:
      dhcp4: false
      dhcp6: false
      accept-ra: false
      addresses:
        - 192.168.56.10/24
        - "fd00:56::10/64"
      routes:
        - to: 10.20.0.0/16
          via: 192.168.56.1
      nameservers:
        addresses: [192.168.56.11, 1.1.1.1]
        search: [lab.local]
```

Verify:

```
ip route show 10.20.0.0/16
ip route get 10.20.1.1                           # shows via 192.168.56.1 dev enp0s8
```

Task 21: Bond two NICs (active-backup)

Task: Bond `enp0s9` and `enp0s10` into `bond0` in active-backup mode with `enp0s9` primary and 100 ms link monitoring. Address 10.50.0.10/24.

Solution:

```
sudo tee /etc/netplan/70-bond.yaml >/dev/null <<'EOF'
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s9: {dhcp4: false}
    enp0s10: {dhcp4: false}
  bonds:
    bond0:
      interfaces: [enp0s9, enp0s10]
      addresses: [10.50.0.10/24]
```

```
      parameters:
        mode: active-backup
        primary: enp0s9
        mii-monitor-interval: 100
EOF
sudo chmod 600 /etc/netplan/70-bond.yaml
sudo netplan apply
```

Verify:

```
cat /proc/net/bonding/bond0 | grep -E 'Mode|Currently Active|Slave Interface'
ip -br addr show bond0
sudo ip link set enp0s9 down; grep 'Currently Active' /proc/net/bonding/bond0
sudo ip link set enp0s9 up
```

Task 22: Bridge interface

Task: Remove the bond, then create bridge `br0` containing `enp0s9`, with address 10.60.0.10/24, STP off. (Bridges are what libvirt guests attach to for LAN access.)

Solution:

```
sudo rm /etc/netplan/70-bond.yaml
sudo ip link delete bond0 2>/dev/null
sudo tee /etc/netplan/70-bridge.yaml >/dev/null <<'EOF'
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s9: {dhcp4: false}
  bridges:
    br0:
      interfaces: [enp0s9]
      addresses: [10.60.0.10/24]
      parameters:
        stp: false
        forward-delay: 0
EOF
sudo chmod 600 /etc/netplan/70-bridge.yaml
sudo netplan apply
```

Verify:

```
bridge link show
ip -d link show br0 | grep -o 'bridge.*stp_state [0-9]'
ip -br addr show br0
```

Task 23: Reverse proxy and load balancer with Nginx

Task: Serve `app.lab.local` on port 80 with Nginx, balancing across two backends on 127.0.0.1:3001 and 127.0.0.1:3002 using least connections, passing the client IP headers. Remove the default site.

Solution:

```
mkdir -p /tmp/b1 /tmp/b2
echo backend-1 > /tmp/b1/index.html; echo backend-2 > /tmp/b2/index.html
python3 -m http.server 3001 --bind 127.0.0.1 --directory /tmp/b1 &
python3 -m http.server 3002 --bind 127.0.0.1 --directory /tmp/b2 &

sudo tee /etc/nginx/sites-available/app >/dev/null <<'EOF'
upstream app_backend {
    least_conn;
    server 127.0.0.1:3001 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:3002 max_fails=3 fail_timeout=10s;
}

server {
    listen 80;
    server_name app.lab.local;

    location / {
        proxy_pass http://app_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
sudo ln -sf /etc/nginx/sites-available/app /etc/nginx/sites-enabled/app
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

Verify:

```
for i in 1 2 3 4; do curl -s -H 'Host: app.lab.local' http://127.0.0.1/; done
kill %1                                          # stop backend-1
curl -s -H 'Host: app.lab.local' http://127.0.0.1/ # still answers (backend-2)
```

If Task 19's redirect is active on enp0s8, test via 127.0.0.1 as above. HAProxy (`apt install haproxy`) is an acceptable alternative if a task names it.

Task 24: Partition, format and mount persistently

Task: On `/dev/sdb` create a GPT label with two equal partitions. Format the first ext4 (label `data1`) and the second XFS (label `data2`). Mount them at `/data1` and `/data2` by UUID at boot.

Solution:

```
sudo parted -s /dev/sdb mklabel gpt \
  mkpart data1 ext4 1MiB 50% mkpart data2 xfs 50% 100%
sudo mkfs.ext4 -L data1 /dev/sdb1
sudo mkfs.xfs  -L data2 /dev/sdb2
sudo mkdir -p /data1 /data2

U1=$(sudo blkid -s UUID -o value /dev/sdb1)
U2=$(sudo blkid -s UUID -o value /dev/sdb2)
echo "UUID=$U1 /data1 ext4 defaults 0 2" | sudo tee -a /etc/fstab
echo "UUID=$U2 /data2 xfs  defaults 0 0" | sudo tee -a /etc/fstab
sudo systemctl daemon-reload
sudo mount -a
```

Verify:

```
lsblk -f /dev/sdb
findmnt /data1; findmnt /data2
sudo findmnt --verify
```

Task 25: Create an LVM volume

Task: Using `/dev/sdc`, create volume group `vg_app` and a 1 GiB logical volume `lv_web` with ext4, mounted persistently at `/srv/web`.

Solution:

```
sudo pvcreate /dev/sdc
sudo vgcreate vg_app /dev/sdc
sudo lvcreate -n lv_web -L 1G vg_app
sudo mkfs.ext4 /dev/vg_app/lv_web
sudo mkdir -p /srv/web
echo "/dev/vg_app/lv_web /srv/web ext4 defaults 0 2" | sudo tee -a /etc/fstab
sudo systemctl daemon-reload && sudo mount -a
```

Verify:

```
sudo pvs; sudo vgs; sudo lvs
df -h /srv/web
```

Task 26: Grow LVM storage online

Task: Add `/dev/sdd` to `vg_app`, grow `lv_web` by 1.5 GiB with its filesystem in one step, and give the Ubuntu root LV all free space left in `ubuntu-vg`.

Solution:

```
sudo pvcreate /dev/sdd
sudo vgextend vg_app /dev/sdd
sudo lvextend -r -L +1.5G /dev/vg_app/lv_web     # -r resizes ext4/xfs too
```

```
sudo vgs ubuntu-vg                              # installer often leaves VFree
sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
```

Verify:

```
sudo lvs -o lv_name,vg_name,lv_size
df -h /srv/web /
```

Shrinking: ext4 can shrink only when unmounted (`lvreduce -r`); XFS cannot shrink at all.

Task 27: Add swap space

Task: Add a 512 MiB swap file `/swapfile2` with priority 10, and a 256 MiB swap LV `lv_swap` in `vg_app` with priority 5. Both persistent.

Solution:

```
sudo fallocate -l 512M /swapfile2
sudo chmod 600 /swapfile2
sudo mkswap /swapfile2
sudo swapon -p 10 /swapfile2
echo "/swapfile2 none swap sw,pri=10 0 0" | sudo tee -a /etc/fstab

sudo lvcreate -n lv_swap -L 256M vg_app
sudo mkswap /dev/vg_app/lv_swap
sudo swapon -p 5 /dev/vg_app/lv_swap
echo "/dev/vg_app/lv_swap none swap sw,pri=5 0 0" | sudo tee -a /etc/fstab
```

Verify:

```
swapon --show
free -h
sudo swapoff -a && sudo swapon -a && swapon --show   # proves fstab entries work
```

Task 28: NFS server and client

Task: On lfcs2, export `/srv/share` read-write to 192.168.56.0/24. On lfcs1, mount it at `/mnt/share` at boot without hanging the boot if lfcs2 is down.

Solution:

```
# lfcs2
sudo apt install -y nfs-kernel-server
sudo mkdir -p /srv/share && sudo chown nobody:nogroup /srv/share
echo "/srv/share 192.168.56.0/24(rw,sync,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra
sudo ufw allow from 192.168.56.0/24 to any port nfs

# lfcs1
sudo apt install -y nfs-common
sudo mkdir -p /mnt/share
echo "lfcs2:/srv/share /mnt/share nfs defaults,_netdev,nofail 0 0" \
  | sudo tee -a /etc/fstab
sudo systemctl daemon-reload && sudo mount /mnt/share
```

Verify:

```
sudo exportfs -v                                 # lfcs2
showmount -e lfcs2                               # lfcs1
findmnt /mnt/share
touch /mnt/share/hello && ls -l /mnt/share
```

Task 29: Network block device (NBD)

Task: On lfcs2, export a 1 GiB image file as NBD export `disk1`. On lfcs1, attach it as `/dev/nbd0`, format it ext4 and mount it at `/mnt/nbd`. Then detach it cleanly.

Solution:

```
# lfcs2
sudo apt install -y nbd-server
sudo mkdir -p /srv/nbd && sudo truncate -s 1G /srv/nbd/disk1.img
sudo chown nbd:nbd /srv/nbd/disk1.img
sudo tee /etc/nbd-server/config >/dev/null <<'EOF'
[generic]
```

```
    user = nbd
    group = nbd
    allowlist = true
[disk1]
    exportname = /srv/nbd/disk1.img
EOF
sudo systemctl restart nbd-server
sudo ufw allow from 192.168.56.0/24 to any port 10809 proto tcp

# lfcs1
sudo apt install -y nbd-client
sudo modprobe nbd
sudo nbd-client -N disk1 lfcs2 /dev/nbd0
sudo mkfs.ext4 -q /dev/nbd0
sudo mkdir -p /mnt/nbd && sudo mount /dev/nbd0 /mnt/nbd
```

Verify:

```
nbd-client -l lfcs2                              # lists exports (allowlist=true)
lsblk /dev/nbd0; findmnt /mnt/nbd
sudo umount /mnt/nbd && sudo nbd-client -d /dev/nbd0
```

iSCSI is the other common block-over-network option (`targetcli` on the server, `open-iscsi` / `iscsiadm` on the client); know that it exists.

Task 30: Automount with autofs and systemd

Task: On lfcs1, use autofs so that accessing `/nfs/share` mounts `lfcs2:/srv/share` on demand and unmounts after 60 s idle. Then show the systemd-native alternative for `/mnt/share`.

Solution:

```
sudo apt install -y autofs
echo "/nfs /etc/auto.nfs --timeout=60" | sudo tee /etc/auto.master.d/nfs.autofs
echo "share -rw,soft lfcs2:/srv/share" | sudo tee /etc/auto.nfs
sudo systemctl restart autofs
# wildcard form for many exports:  *  -rw  lfcs2:/srv/&

# systemd alternative: fstab line for /mnt/share becomes
# (one line in the real file)
# lfcs2:/srv/share /mnt/share nfs
#   noauto,x-systemd.automount,x-systemd.idle-timeout=60,_netdev 0 0
```

Verify:

```
ls /nfs                                          # empty: nothing mounted yet
ls /nfs/share                                    # triggers the mount
mount | grep /nfs/share
systemctl list-units --type=automount            # for the systemd variant
```

Task 31: Virtual filesystems (tmpfs, /proc, /sys)

Task: Mount a 256 MiB tmpfs at `/mnt/scratch` with sticky world-writable permissions, persistent. Then read the I/O scheduler of `/dev/sda` from sysfs and the kernel command line from procfs.

Solution:

```
sudo mkdir -p /mnt/scratch
echo "tmpfs /mnt/scratch tmpfs size=256M,mode=1777 0 0" | sudo tee -a /etc/fstab
sudo systemctl daemon-reload && sudo mount /mnt/scratch

cat /sys/block/sda/queue/scheduler
cat /proc/cmdline
sudo mount -o remount,size=512M /mnt/scratch     # runtime resize
```

Verify:

```
findmnt /mnt/scratch
df -h /mnt/scratch; stat -c '%A' /mnt/scratch    # drwxrwxrwt
```

Task 32: Monitor storage performance

Task: Generate write load on `/data1`, then identify the busiest disk, its utilisation and wait time, and the process responsible.

Solution:

```
sudo apt install -y sysstat iotop
sudo dd if=/dev/zero of=/data1/load bs=1M count=800 oflag=direct &
iostat -xz 2 3                                  # watch %util, w_await, wMB/s
pidstat -d 2 3                                  # per-process kB_wr/s
sudo iotop -o -b -n 3                           # only processes doing I/O
wait; sudo rm -f /data1/load
```

Verify:

```
iostat -dx sdb 1 1 | tail -2
vmstat 1 3                                      # "wa" column = CPU waiting on I/O
```

Task 33: Check and tune filesystems

Task: On `/data1` (ext4), reduce reserved blocks to 1% and change the label to `appdata`. Run a read-only consistency check of the XFS filesystem on `/dev/sdb2`, then relabel it `archive`.

Solution:

```
sudo tune2fs -m 1 /dev/sdb1
sudo e2label /dev/sdb1 appdata

sudo umount /data2
sudo xfs_repair -n /dev/sdb2                    # -n = check only, no changes
sudo xfs_admin -L archive /dev/sdb2
sudo mount /data2                               # fstab uses UUID, so still works
```

Verify:

```
sudo tune2fs -l /dev/sdb1 | grep -E 'volume name|Reserved block count'
lsblk -o NAME,FSTYPE,LABEL /dev/sdb
```

Task 34: Git basics in a repository

Task: In `~/infra`, initialise a repo on branch `main`, commit an `nginx.conf`, create branch `feature/tls`, commit a change there, merge it back into `main`, tag the result `v1.0`, and undo the last commit safely with a new commit.

Solution:

```
git config --global user.name "Rizwan Khalid"
git config --global user.email "rizwan@lab.local"
git config --global init.defaultBranch main

mkdir -p ~/infra && cd ~/infra && git init
echo "worker_processes auto;" > nginx.conf
git add nginx.conf && git commit -m "Add base nginx config"
git switch -c feature/tls
echo "# tls enabled" >> nginx.conf
git commit -am "Enable TLS"
git switch main
git merge --no-ff feature/tls -m "Merge feature/tls"
git tag -a v1.0 -m "First release"
git revert --no-edit -m 1 HEAD                  # HEAD is the merge commit
```

Verify:

```
git log --oneline --graph --all
git tag -n
git status -s
```

Note: HEAD is a merge commit, so `git revert -m 1 HEAD` is the form that works; plain `git revert HEAD` is for normal commits.

Task 35: Git remotes over SSH

Task: On lfcs2, create a bare repository `/srv/git/infra.git`. From lfcs1, add it as `origin`, push `main` and tags, then clone it into `/tmp/infra-clone` and confirm the history matches.

Solution:

```
# lfcs2
sudo mkdir -p /srv/git && sudo chown "$USER": /srv/git
git init --bare /srv/git/infra.git
```

```
# lfcs1
cd ~/infra
git remote add origin youruser@lfcs2:/srv/git/infra.git
git push -u origin main --tags
git clone youruser@lfcs2:/srv/git/infra.git /tmp/infra-clone
```

Verify:

```
git -C ~/infra remote -v
git -C /tmp/infra-clone log --oneline -3
git -C ~/infra status -sb                       # "## main...origin/main"
```

Task 36: Create a systemd service for a Node.js app

Task: Run `/opt/api/server.js` as system user `nodeapp` under systemd service `api`, with `NODE_ENV=production`, restart on failure after 3 s, started after the network is online and enabled at boot.

Solution:

```
sudo apt install -y nodejs
sudo useradd -r -s /usr/sbin/nologin -d /opt/api nodeapp
sudo mkdir -p /opt/api
echo "require('http').createServer((q,s)=>s.end('ok\n')).listen(4000);" \
  | sudo tee /opt/api/server.js
sudo chown -R nodeapp:nodeapp /opt/api

sudo tee /etc/systemd/system/api.service >/dev/null <<'EOF'
[Unit]
Description=Demo Node API
Wants=network-online.target
After=network-online.target

[Service]
User=nodeapp
Group=nodeapp
WorkingDirectory=/opt/api
Environment=NODE_ENV=production
ExecStart=/usr/bin/node /opt/api/server.js
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now api
```

Verify:

```
systemctl status api --no-pager
curl -s localhost:4000
sudo kill -9 "$(systemctl show -p MainPID --value api)"; sleep 4
systemctl show -p NRestarts api                 # increments after the crash
```

Task 37: Constrain a service with a drop-in

Task: Without editing `api.service`, limit it to 256 MiB of memory, 50% of one CPU, 4096 open files and 64 tasks.

Solution:

```
sudo mkdir -p /etc/systemd/system/api.service.d
sudo tee /etc/systemd/system/api.service.d/limits.conf >/dev/null <<'EOF'
[Service]
MemoryMax=256M
CPUQuota=50%
LimitNOFILE=4096
TasksMax=64
EOF
sudo systemctl daemon-reload
sudo systemctl restart api
# interactive equivalent: sudo systemctl edit api
```

Verify:

```
systemctl show api -p MemoryMax -p CPUQuotaPerSecUSec -p LimitNOFILE -p TasksMax
```

```
systemctl cat api                                # shows the drop-in
grep 'open files' /proc/$(systemctl show -p MainPID --value api)/limits
```

Task 38: Determine a service's constraints and dependencies

Task: For the `api` service, report: its main PID and user, the ports it listens on, its number of open files versus its limit, its cgroup memory use, the shared libraries `node` needs, and which units it depends on.

Solution:

```
PID=$(systemctl show -p MainPID --value api)
ps -o pid,user,rss,cmd -p "$PID"
sudo ss -tlnp | grep "pid=$PID"
sudo ls /proc/"$PID"/fd | wc -l
grep 'open files' /proc/"$PID"/limits
systemctl show api -p MemoryCurrent -p MemoryMax
ldd /usr/bin/node | head
systemctl list-dependencies api --no-pager
```

Verify:

```
systemd-cgtop -1 -n 1 | grep api
sudo lsof -p "$PID" | head
```

Task 39: Monitor system performance

Task: Record a quick health snapshot: load average, top CPU and memory consumers, memory pressure, CPU steal/iowait, errors logged this boot, and the slowest units at boot. Enable sysstat history collection.

Solution:

```
uptime
ps -eo pid,user,pcpu,pmem,comm --sort=-pcpu | head -6
ps -eo pid,user,pcpu,pmem,comm --sort=-pmem | head -6
free -h
vmstat 2 5                                       # r, si/so, wa, st
mpstat -P ALL 2 1
journalctl -p err -b --no-pager | tail -20
systemd-analyze blame | head

sudo sed -i 's/^ENABLED=.*/ENABLED="true"/' /etc/default/sysstat
sudo systemctl enable --now sysstat
```

Verify:

```
systemctl list-timers | grep sysstat
sar -u 1 3                                       # history appears after ~10 min
```

Task 40: Troubleshoot a full disk

Task: `/` reports full. Find the largest directories and files, find space held by deleted-but-open files and release it, check inode usage, and cap the journal at 200 MiB.

Solution:

```
# setup: a deleted log still held open
sudo fallocate -l 500M /var/tmp/big.log
sudo sh -c 'tail -f /var/tmp/big.log >/dev/null &'
sudo rm /var/tmp/big.log

df -hT /
df -i /
sudo du -xh / --max-depth=1 2>/dev/null | sort -h | tail
sudo find / -xdev -type f -size +100M -exec ls -lh {} + 2>/dev/null
sudo lsof +L1                                    # deleted files still open
sudo pkill -f 'tail -f /var/tmp/big.log'          # or: truncate /proc/PID/fd/N

journalctl --disk-usage
sudo journalctl --vacuum-size=200M
sudo mkdir -p /etc/systemd/journald.conf.d
printf '[Journal]\nSystemMaxUse=200M\n' \
  | sudo tee /etc/systemd/journald.conf.d/size.conf
sudo systemctl restart systemd-journald
sudo apt clean
```

Verify:

```
df -h /
sudo lsof +L1 | grep big.log || echo "released"
journalctl --disk-usage
```

Task 41: Create and inspect TLS certificates

Task: Create a 2048-bit RSA key and a self-signed certificate for `app.lab.local` (SAN including the IP 192.168.56.10) valid for 365 days. Also create a CSR from the same key. Show subject, SANs, and expiry, prove key and certificate match, and test whether it expires within 30 days.

Solution:

```
sudo openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
  -keyout /etc/ssl/private/app.key -out /etc/ssl/certs/app.crt \
  -subj "/CN=app.lab.local" \
  -addext "subjectAltName=DNS:app.lab.local,IP:192.168.56.10"
sudo chmod 600 /etc/ssl/private/app.key

sudo openssl req -new -key /etc/ssl/private/app.key -out /tmp/app.csr \
  -subj "/CN=app.lab.local"
```

Verify:

```
openssl x509 -in /etc/ssl/certs/app.crt -noout -subject -dates -ext subjectAltName
openssl req -in /tmp/app.csr -noout -subject -verify
diff <(openssl x509 -in /etc/ssl/certs/app.crt -noout -pubkey) \
     <(sudo openssl pkey -in /etc/ssl/private/app.key -pubout) && echo "match"
openssl x509 -in /etc/ssl/certs/app.crt -noout -checkend 2592000 \
  && echo "valid > 30 days"
```

Task 42: Deploy the certificate and trust it

Task: Serve `app.lab.local` over HTTPS on 443 with the Task 41 certificate, redirect HTTP to HTTPS, and add the certificate to the system trust store so `curl` accepts it without `-k`.

Solution:

```
sudo tee /etc/nginx/sites-available/app-tls >/dev/null <<'EOF'
server {
    listen 443 ssl;
    server_name app.lab.local;
    ssl_certificate     /etc/ssl/certs/app.crt;
    ssl_certificate_key /etc/ssl/private/app.key;
    location / { proxy_pass http://app_backend; }
}
server {
    listen 80;
    server_name app.lab.local;
    return 301 https://$host$request_uri;
}
EOF
sudo rm -f /etc/nginx/sites-enabled/app          # replace the port-80 vhost
sudo ln -sf /etc/nginx/sites-available/app-tls /etc/nginx/sites-enabled/
# app_backend upstream is defined in sites-available/app; move it to conf.d:
sudo sed -n '/^upstream/,/^}/p' /etc/nginx/sites-available/app \
  | sudo tee /etc/nginx/conf.d/upstream-app.conf
sudo nginx -t && sudo systemctl reload nginx
sudo ufw allow 443/tcp

sudo cp /etc/ssl/certs/app.crt /usr/local/share/ca-certificates/app-lab.crt
sudo update-ca-certificates
echo "127.0.0.1 app.lab.local" | sudo tee -a /etc/hosts
```

Verify:

```
curl -sI http://app.lab.local | grep -i location
curl -s https://app.lab.local/                   # no -k needed
openssl s_client -connect app.lab.local:443 -servername app.lab.local </dev/null \
  2>/dev/null | openssl x509 -noout -subject -enddate
```

Task 43: Users, groups and sudo

Task: Create group `developers` (GID 3000). Create `dev1` and `dev2` with home directories, bash, and `developers` as a supplementary group. Force `dev1` to change password at next login and expire passwords every 90 days with 7 days warning. Let `developers` restart the `api` service via sudo without a password, and nothing else. Lock `dev2`. Create a shared directory `/srv/dev` where new files inherit the group.

Solution:

```
sudo groupadd -g 3000 developers
sudo useradd -m -s /bin/bash -G developers dev1
sudo useradd -m -s /bin/bash -G developers dev2
echo 'dev1:ChangeMe#2026' | sudo chpasswd
sudo chage -d 0 -M 90 -W 7 dev1
sudo usermod -L dev2                              # or: passwd -l dev2

echo '%developers ALL=(root) NOPASSWD: /usr/bin/systemctl restart api' \
  | sudo tee /etc/sudoers.d/developers
sudo chmod 440 /etc/sudoers.d/developers
sudo visudo -cf /etc/sudoers.d/developers

sudo mkdir -p /srv/dev
sudo chgrp developers /srv/dev
sudo chmod 2770 /srv/dev
```

Verify:

```
id dev1; getent group developers
sudo chage -l dev1
sudo passwd -S dev2                               # "L" = locked
sudo -l -U dev1
ls -ld /srv/dev                                   # drwxrws---
```

Task 44: Shell environment, per-user and system-wide

Task: For all users' login shells set `APP_ENV=lab` and add `/opt/tools/bin` to PATH. New users must get an alias `ll='ls -alF --color=auto'` and `EDITOR=vim`. For `dev1` only, set `HISTSIZE=5000`. Set `COMPANY=acme` for every PAM session, including non-shell ones.

Solution:

```
sudo tee /etc/profile.d/lab.sh >/dev/null <<'EOF'
export APP_ENV=lab
case ":$PATH:" in *":/opt/tools/bin:"*) ;; *) PATH="$PATH:/opt/tools/bin" ;; esac
export PATH
EOF

printf "alias ll='ls -alF --color=auto'\nexport EDITOR=vim\n" \
  | sudo tee -a /etc/skel/.bashrc

echo 'HISTSIZE=5000' | sudo tee -a /home/dev1/.bashrc
echo 'COMPANY=acme' | sudo tee -a /etc/environment   # KEY=value, read by pam_env
```

Verify:

```
sudo useradd -m -s /bin/bash dev3
sudo -iu dev3 bash -ic 'alias ll; echo $EDITOR $APP_ENV $COMPANY; echo $PATH'
sudo -iu dev1 bash -ic 'echo $HISTSIZE'
```

Remember: `/etc/profile.d` is read by login shells; `~/.bashrc` by interactive shells (Ubuntu's `~/.profile` sources it); `/etc/skel` only affects users created afterwards.

Task 45: Per-user resource limits

Task: Members of `developers` get a soft limit of 200 processes and a hard limit of 300. `dev1` gets 4096 soft and 8192 hard open files.

Solution:

```
sudo tee /etc/security/limits.d/90-developers.conf >/dev/null <<'EOF'
@developers  soft  nproc   200
@developers  hard  nproc   300
dev1         soft  nofile  4096
dev1         hard  nofile  8192
EOF
```

Verify:

```
sudo -iu dev1 bash -c 'ulimit -Su; ulimit -Hu; ulimit -Sn; ulimit -Hn'
ssh dev1@localhost 'ulimit -u'                    # if dev1 has a key or password
```

Trap: limits are applied by `pam_limits` at session start (ssh, login, sudo). On Ubuntu, `su` does not load it unless you uncomment the `pam_limits.so` line in `/etc/pam.d/su`. Services ignore limits.conf; use `LimitNPROC=` / `LimitNOFILE=` in the unit (Task 37).

Task 46: POSIX ACLs

Task: On `/srv/project` (owned by root, mode 750), give `dev2` read-write access and the `developers` group read-only access, including to all files created there in future. Then remove `dev2`'s access.

Solution:

```
sudo mkdir -p /srv/project && sudo chmod 750 /srv/project
sudo setfacl -m u:dev2:rwx /srv/project
sudo setfacl -m g:developers:rx /srv/project
sudo setfacl -d -m u:dev2:rwx /srv/project        # default ACL for new files
sudo setfacl -d -m g:developers:rx /srv/project
sudo touch /srv/project/notes.txt

sudo setfacl -x u:dev2 /srv/project               # remove later
sudo setfacl -d -x u:dev2 /srv/project
```

Verify:

```
getfacl /srv/project
getfacl /srv/project/notes.txt                     # inherited entries + mask
ls -ld /srv/project                                # trailing "+" = ACL present
```

Task 47: LDAP accounts via SSSD

Task: On lfcs2, run an OpenLDAP server for `dc=lab,dc=local` containing POSIX user `alice` (UID 5001) in group `devs` (GID 5000). On lfcs1, configure SSSD so `alice` resolves and can log in, with her home created on first login.

Solution:

```
# lfcs2: server
sudo apt install -y slapd ldap-utils            # set an admin password
sudo dpkg-reconfigure slapd                     # DNS domain: lab.local, org: lab
slappasswd -s 'Alice#2026'                      # copy the {SSHA} hash
cat > /tmp/users.ldif <<'EOF'
dn: ou=people,dc=lab,dc=local
objectClass: organizationalUnit
ou: people

dn: ou=groups,dc=lab,dc=local
objectClass: organizationalUnit
ou: groups

dn: cn=devs,ou=groups,dc=lab,dc=local
objectClass: posixGroup
cn: devs
gidNumber: 5000
memberUid: alice

dn: uid=alice,ou=people,dc=lab,dc=local
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: alice
cn: Alice Lab
sn: Lab
uidNumber: 5001
gidNumber: 5000
homeDirectory: /home/alice
loginShell: /bin/bash
userPassword: {SSHA}PASTE_HASH_HERE
EOF
ldapadd -x -D cn=admin,dc=lab,dc=local -W -f /tmp/users.ldif
sudo ufw allow from 192.168.56.0/24 to any port 389 proto tcp
```

```
# lfcs1: client
sudo apt install -y sssd-ldap ldap-utils libnss-sss libpam-sss
sudo tee /etc/sssd/sssd.conf >/dev/null <<'EOF'
[sssd]
services = nss, pam
domains = lab

[domain/lab]
id_provider = ldap
auth_provider = ldap
ldap_uri = ldap://192.168.56.11
ldap_search_base = dc=lab,dc=local
ldap_id_use_start_tls = false
ldap_auth_disable_tls_never_use_in_production = true
cache_credentials = true
EOF
sudo chmod 600 /etc/sssd/sssd.conf              # sssd refuses other modes
sudo systemctl restart sssd
sudo pam-auth-update --enable mkhomedir
```

Verify:

```
ldapsearch -x -H ldap://192.168.56.11 -b dc=lab,dc=local uid=alice uidNumber
getent passwd alice; id alice
grep -E '^(passwd|group):' /etc/nsswitch.conf     # should include sss
su - alice -c 'pwd; id'                           # /home/alice created
```

The no-TLS options are for the lab only. In production use `ldaps://` or StartTLS with `ldap_tls_cacert`.

Practice questions

1. Which single command loads every file under `/etc/sysctl.d/` into the running kernel?
2. You are editing netplan over SSH. Which command applies the change but rolls it back automatically if you do not confirm?
3. On Ubuntu 24.04 you added `Port 2222` to an sshd drop-in and ran `systemctl restart ssh`, but nothing listens on 2222. Why, and what fixes it?
4. Which command shows the DNS servers and search domains systemd-resolved uses for each interface?
5. How do you stop `apt upgrade` from upgrading the `nginx` package, and how do you list such packages?
6. What is the difference between `dpkg -S /usr/bin/dig` and `apt-file search bin/dig`?
7. Multiple choice. Which rule allows SSH only from 10.0.0.0/8? (a) `ufw allow 22/tcp from 10.0.0.0/8` (b) `ufw allow from 10.0.0.0/8 to any port 22 proto tcp` (c) `ufw permit ssh 10.0.0.0/8` (d) `ufw route allow 22 10.0.0.0/8`
8. ufw denies all incoming traffic, yet a container started with `docker run -p 8080:80` is reachable from other hosts. Why? Name one fix.
9. You redirected port 80 to 3000 in the PREROUTING chain. Remote clients work, but `curl localhost:80` on the same host fails. Why?
10. What one command grows `/dev/vg_app/lv_web` by 2 GiB and resizes its filesystem too?
11. True or false: an XFS filesystem can be shrunk with `lvreduce -r`.
12. Which two fstab options should an NFS mount have so that boot is not blocked when the server is down?
13. Two swap areas have priorities 5 and 10. Which one is used first?
14. In the autofs map line `* -rw lfcs2:/home/&`, what do `*` and `&` mean?
15. `df` reports `/var` 100% full, but `du -sh /var` shows far less. What is the likely cause and which command confirms it?
16. `df -h` shows free space but writes fail with "No space left on device". What do you check next?
17. Explain soft versus hard limits in `/etc/security/limits.conf`.

18. A file has the ACL entry `user:bob:rw-`. You run `chmod g=r file`. What are bob's effective permissions now, and why?
19. What is wrong with the line `export COMPANY=acme` in `/etc/environment`?
20. You want every user created from now on to have an alias. Which file do you edit?
21. Which command puts the AppArmor profile for `/usr/local/bin/report.sh` into complain mode, and where do denials appear?
22. On an RHEL-family host, files copied to a custom web root have the wrong SELinux type after a relabel. Which two commands give a permanent fix?
23. What does `Persistent=true` do in a systemd `.timer` unit?
24. What does the cron schedule `*/15 9-17 * * 1-5` mean?
25. Which `virsh` commands make VM `vm1` start at host boot and set its maximum memory to 2 GiB for the next boot?
26. Which Docker restart policy restarts a container after a daemon or host restart, but not after you stopped it by hand?
27. What load-balancing method does an Nginx `upstream` use by default, and which directive changes it to fewest active connections?
28. Which OpenSSL command exits non-zero if a certificate expires within 7 days?
29. You pushed a bad commit to a shared branch. What is the safe way to undo it?
30. Write the netplan fragment that adds a persistent route to 10.20.0.0/16 via 192.168.56.1.
31. Which bonding mode gives failover without any switch configuration?
32. `timedatectl` shows "System clock synchronized: no". Name three things to check.
33. Which command shows which process is listening on TCP port 8080?
34. An Ubuntu server hangs in boot because of a bad fstab entry, and the emergency shell says the root account is locked. How do you get a root shell?
35. `ldapsearch` finds `alice`, but `getent passwd alice` returns nothing on the SSSD client. Name three likely causes.
36. A service must be allowed 65536 open files. Why does adding it to `/etc/security/limits.conf` not work, and what does?

Answers

1. `sudo sysctl --system`. It reads `/etc/sysctl.d/*.conf`, `/run/sysctl.d`, `/usr/lib/sysctl.d` and `/etc/sysctl.conf` in order.
2. `sudo netplan try`. It reverts after 120 seconds unless you press Enter.
3. Ubuntu 24.04 starts sshd through `ssh.socket`; ports come from a generator. Run `systemctl daemon-reload` then `systemctl restart ssh.socket`.
4. `resolvectl status` (or `resolvectl dns` and `resolvectl domain`).
5. `sudo apt-mark hold nginx`; list with `apt-mark showhold`, undo with `apt-mark unhold`.
6. `dpkg -S` searches only installed packages; `apt-file` searches the repository file lists, so it finds packages you have not installed.
7. (b). ufw syntax is `allow from SRC to any port N proto tcp`.
8. Docker adds its own iptables NAT and FORWARD rules, which are evaluated before ufw's input rules. Publish on `127.0.0.1:` or add rules to the `DOCKER-USER` chain.
9. Locally generated packets do not pass PREROUTING. You need a matching rule in the nat OUTPUT chain.

10. `sudo lvextend -r -L +2G /dev/vg_app/lv_web`. `-r` calls `fsadm` to resize ext4 or XFS.
11. False. XFS can only grow; back up, recreate and restore to shrink.
12. `_netdev` (wait for the network) and `nofail` (do not fail boot); `x-systemd.automount` also helps.
13. Priority 10. Higher priority swap is used first; equal priorities are striped.
14. `*` matches any key looked up under the mount point; `&` is replaced by that key, so `/home/bob` mounts `lfcs2:/home/bob`.
15. A process still holds a deleted file open. `sudo lsof +L1` lists them; restart the process or truncate via `/proc/PID/fd/N`.
16. Inodes: `df -i`. Many tiny files (sessions, cache) can exhaust inodes while blocks remain free.
17. The soft limit is the enforced default a user can raise up to the hard limit; the hard limit is the ceiling only root can raise.
18. `r--`. With ACLs, the group permission bits become the ACL mask, which caps named-user and group entries.
19. `/etc/environment` is read by `pam_env`, not by a shell. Use plain `KEY=value` lines with no `export`.
20. `/etc/skel/.bashrc` (skeleton files are copied at `useradd -m` time only).
21. `sudo aa-complain /usr/local/bin/report.sh`. Denials and complain-mode events go to the kernel log (`journalctl -k`), or `/var/log/audit/audit.log` if auditd runs.
22. `semanage fcontext -a -t httpd_sys_content_t "/srv/web(/.*)?"` then `restorecon -Rv /srv/web`.
23. It stores the last run time on disk and runs a missed job once at next boot if the system was off when it was due.
24. Every 15 minutes from 09:00 to 17:45, Monday to Friday.
25. `virsh autostart vm1` and `virsh setmaxmem vm1 2G --config` (plus `virsh setmem ... --config` if needed).
26. `unless-stopped`. `always` would also restart a container you had stopped once the daemon restarts.
27. Round robin by default; `least_conn;` inside the `upstream` block.
28. `openssl x509 -in cert.crt -noout -checkend 604800` (seconds).
29. `git revert <commit>` creates a new commit that undoes it, so shared history is not rewritten.
30. Under the interface: `routes:` then a list item `- to: 10.20.0.0/16` with `via: 192.168.56.1`, then `netplan apply`.
31. `active-backup` (mode 1).
32. That NTP is on (`timedatectl set-ntp true`), that `systemd-timesyncd` or `chronyd` is running (`timedatectl timesync-status`, `chronyc sources`), and that UDP 123 and DNS for the NTP host work.
33. `sudo ss -tlnp 'sport = :8080'` (or `sudo lsof -i :8080`).
34. Edit the GRUB entry, append `init=/bin/bash`, boot, `mount -o remount,rw /`, fix `/etc/fstab`, then `sync` and `reboot -f`.
35. `sssd.conf` not mode 0600 (sssd will not start), sssd not running or wrong `ldap_search_base`, or `/etc/nsswitch.conf` missing `sss`; clear stale cache with `sss_cache -E`.
36. limits.conf is applied by PAM at login sessions; systemd services do not use PAM. Set `LimitNOFILE=65536` in a unit drop-in and restart.

Exam-day tips

Delivery rules (official):

- Remote exam through the PSI Bridge secure browser. Run the system check well before the day.

- Bring a valid, unexpired, government-issued photo ID (passport or national ID card). The first and last name must exactly match the name on your exam checklist.
- Test alone in a private, quiet, well-lit room with a clear desk: no phone, smartwatch, notes or second screen. Only clear liquids.
- You stay on camera and talk only to the proctor.
- Allowed help inside the exam: man pages, docs under `/usr/share/doc`, and distribution packages. No web browsing.
- Results arrive by email within 24 hours. A No Pass unlocks your free retake in the portal.

Environment habits (from the official LFCS instructions):

- Each task names a host. SSH to it from the base system, and exit back to base before the next task. Nested SSH (host to host) is not supported.
- Never reboot the base system. Rebooting a task node is fine when needed.
- Use `sudo -i` for root. Vim, nano and emacs are available.
- Terminal copy/paste is usually Ctrl+Shift+C / Ctrl+Shift+V; re-read the instructions page just before the exam.

Time management:

- 17-20 tasks in 120 minutes is about 6 minutes each. Read all tasks first; do the quick, high-value ones first.
- Flag and skip anything that eats more than 10 minutes; come back once the easy points are banked.
- Leave 10 minutes at the end to re-check persistence on the tasks you flagged.

Common traps:

- Runtime-only fixes. Most tasks are graded after a reboot or re-check: use fstab, netplan, sysctl.d, `systemctl enable`, `ufw` (not raw iptables), `crontab`.
- Wrong host. Check `hostname` before every change.
- netplan YAML indentation and file mode 600; always `netplan generate` or `netplan try` first.
- `ssh.socket` on 24.04 (Task 16) and sshd first-match rules in `sshd_config.d`.
- Enabling ufw before allowing SSH on a remote host.
- Forgetting `daemon-reload` after editing units or fstab.
- `usermod -G` without `-a` wipes existing supplementary groups.
- Mounting by `/dev/sdX` when the task says UUID or label.
- Exact names and paths: graders check them literally.
- Search man pages fast: `man -k keyword`, `/pattern` inside man, and `EXAMPLES` sections (`man 5 crontab`, `man systemd.timer`, `man 5 exports`, `man netplan`).

Readiness checklist

- ■ Lab built: two Ubuntu 24.04 VMs, spare disks, extra NICs, clean snapshots
- ■ I can set sysctl values at runtime and persistently without looking anything up
- ■ I can diagnose a failed service with `systemctl status`, `journalctl -u` and a config test
- ■ I can write a cron entry, an `/etc/cron.d` file and a systemd timer from memory
- ■ I can hold, query, verify packages and add a repo with a keyring and `.sources` file
- ■ I can recover a system with a broken fstab using `init=/bin/bash`
- ■ I can create, start, autostart and resize a libvirt VM with `virt-install` and `virsh`

- [ ] I can run, publish, persist and build Docker containers
- [ ] I can write, load and switch modes of an AppArmor profile; I know the SELinux basics
- [ ] I can configure static IPv4/IPv6, DNS and routes with netplan and `netplan try`
- [ ] I can set hostname, `/etc/hosts`, resolved settings, time zone and NTP (timesyncd, chrony)
- [ ] I can harden sshd on 24.04 and set up client keys and `~/.ssh/config`
- [ ] I can write ufw rules, and port redirection and masquerade in `before.rules`
- [ ] I can build a bond and a bridge with netplan and prove failover
- [ ] I can configure Nginx as a reverse proxy and load balancer, with TLS
- [ ] I can partition, format and mount by UUID; check and repair ext4 and XFS
- [ ] I can create, extend and add PVs to LVM, and grow the Ubuntu root LV
- [ ] I can add swap files and swap LVs with priorities
- [ ] I can set up NFS server and client, NBD, autofs and systemd automount
- [ ] I can find I/O bottlenecks with `iostat`, `pidstat` and `iotop`
- [ ] I can use Git: branch, merge, tag, revert, push to a bare remote
- [ ] I can write a systemd unit with restart policy and resource limits in a drop-in
- [ ] I can find and free disk space, including deleted-open files and inodes
- [ ] I can create, inspect, match and deploy certificates, and trust a CA
- [ ] I can manage users, groups, password aging, sudoers drop-ins and setgid dirs
- [ ] I can set environment variables in the right file for each scope
- [ ] I can set limits for users (limits.d) and for services (unit `Limit*=`)
- [ ] I can set and remove access and default ACLs and explain the mask
- [ ] I can configure an SSSD LDAP client and verify with `getent` and `id`
- [ ] I completed all 47 tasks at least twice, the second time without notes
- [ ] killer.sh session 1 done, gaps fixed; session 2 scored comfortably above 67%
- [ ] ID checked against my exam checklist name; PSI system check passed; room ready

Sources

- LFCS certification page (price, duration, format, domains, validity, retake, simulator): https://training.linuxfoundation.org/certification/linux-foundation-certified-sysadmin-lfcs/
- LFCS exam update announcement (effective 11 May 2023, domain weights, distribution-independent): https://training.linuxfoundation.org/blog/updated-linux-foundation-certified-system-administrator-lfcs-exam/
- Important Instructions: LFCS (17-20 tasks, 67% pass mark, allowed resources, SSH and reboot rules): https://docs.linuxfoundation.org/tc-docs/certification/instructions-lfcs-and-lfce
- Certification exam retake policy: https://training.linuxfoundation.org/about/policies-2-2026/certification-exam-retake-policy/
- Certification FAQ (renewal, results timing): https://training.linuxfoundation.org/about/faqs/certification-faq/
- Candidate handbook, exam rules and policies: https://docs.linuxfoundation.org/tc-docs/certification/lf-handbook2/exam-rules-and-policies
- Candidate handbook, identification and authentication: https://docs.linuxfoundation.org/tc-docs/certification/lf-handbook2/candidate-identification-and-authentication

- Candidate handbook, exam scoring and notification: https://docs.linuxfoundation.org/tc-docs/certification/lf-handbook2/exam-scoring-and-notification
- Candidate handbook, repurchasing the exam: https://docs.linuxfoundation.org/tc-docs/certification/lf-handbook2/repurchasing-the-exam
- killer.sh LFCS simulator: https://killer.sh/lfcs
- Ubuntu Server documentation: https://documentation.ubuntu.com/server/
- Netplan documentation: https://netplan.readthedocs.io/
- Ubuntu Server download: https://ubuntu.com/download/server
- Unverified: the exact Ubuntu release used in the live exam environment (official pages say only that the exam is distribution-independent).

Chapter 3: AWS Certified SysOps Administrator - Associate

This guide prepares you; you register for, sit, and pass the exam yourself.

Name change first: AWS renamed this credential. The SysOps Administrator - Associate exam (SOA-C02) ran for the last time on September 29, 2025. Its replacement, **AWS Certified CloudOps Engineer - Associate (exam code SOA-C03)**, has been live since September 30, 2025. Same exam code family (SOA), same level, updated content. Everything in this chapter targets SOA-C03. If you already held the SysOps cert you would not need to retake anything, but you don't, so SOA-C03 is your exam.

Exam labs: SOA-C02 used to include hands-on exam labs in a live AWS console. AWS removed them on March 28, 2023 ("until further notice"), and SOA-C03 launched as multiple choice and multiple response only. You will not be graded in a live console, but the labs below remain the fastest way to make the scenario questions easy.

As of September 2026.

At a glance

Item	Detail
Official name	AWS Certified CloudOps Engineer - Associate
Exam code	SOA-C03 (replaced SOA-C02 on Sep 30, 2025)
Exam guide version	1.1, published June 1, 2026
Format	Multiple choice and multiple response; no exam labs
Questions	65 (50 scored, 15 unscored, not identified)
Duration	130 minutes
Passing score	720 on a 100-1,000 scaled score
Scoring	Compensatory: pass overall, not per domain
Guessing	No penalty; unanswered = wrong
Price	150 USD (plus local tax where applicable)
Delivery	Pearson VUE test center or online proctored
Languages	English, Japanese, Korean, Simplified Chinese
2026 change	Korean and Chinese versions retire after Nov 19, 2026
Validity	3 years
Retake	Wait 14 days after a fail; full fee each attempt
Prerequisites	None (AWS suggests 1 year of AWS ops)
Next version	No SOA-C03 replacement announced (checked Sep 2026)

Domain weights (SOA-C03):

Domain	Weight
1. Monitoring, logging, analysis, remediation, performance	22%
2. Reliability and business continuity	22%
3. Deployment, provisioning, and automation	22%
4. Security and compliance	16%
5. Networking and content delivery	18%

Why this fits you: you already run Node.js behind Nginx with PM2 on Linux, so the ops mindset (logs, restarts, health checks, patching, backups) is there. This exam turns that into the AWS-native versions: CloudWatch instead of `tail -f`, Systems Manager instead of SSH loops, ALB plus Auto Scaling instead of one hand-built box. It is also

the most hands-on AWS associate credential and a natural bridge from "I deploy apps" to "I operate the platform".

Objectives map

Your strength is judged from your profile: strong Linux/Nginx/PM2/cron ops, lighter on AWS ops services and formal networking.

Objective area	What you must be able to do	Your current strength
1.1 Metrics, alarms, filters	CloudWatch agent, log groups, metric filters, alarms	Refresh
1.1 Notifications, dashboards	SNS alerts, composite alarms, cross-account dashboards	New
1.2 Identify and remediate	EventBridge rules, SSM Automation runbooks, Lambda fixes	New
1.3 Compute performance	Right-size EC2, placement groups, Compute Optimizer	Refresh
1.3 Storage performance	EBS gp3 tuning, S3 multipart, Transfer Acceleration	Refresh
1.3 Database performance	RDS metrics, Performance Insights, RDS Proxy	New
1.3 Shared storage	Choose EFS vs FSx vs S3; EFS lifecycle	New
2.1 Scalability	Auto Scaling policies, caching (CloudFront, ElastiCache)	Refresh
2.1 Database scaling	Read replicas, Aurora, DynamoDB capacity modes	New
2.2 High availability	ELB and Route 53 health checks, Multi-AZ	Refresh
2.3 Backup and restore	AWS Backup, snapshots, PITR, S3 versioning, DR tiers	Refresh
3.1 Provisioning	AMIs, Image Builder, CloudFormation, CDK, StackSets, RAM	New
3.1 Deployment issues	Fix stack errors, subnet sizing, permission failures	New
3.1 Third-party IaC	Terraform and Git in AWS workflows	Refresh
3.2 Automate operations	Systems Manager (Run Command, Patch, State Manager)	New
3.2 Event-driven automation	Lambda, S3 notifications, EventBridge	Refresh
4.1 IAM	Roles, MFA, policy conditions, resource policies	Refresh
4.1 Audit access	CloudTrail, IAM Access Analyzer, policy simulator	New
4.1 Multi-account	Organizations, SCPs, IAM Identity Center	New
4.1 Compliance	Config rules, conformance packs, Trusted Advisor	New
4.2 Data protection	KMS, ACM, Secrets Manager, Parameter Store	Refresh
4.2 Findings	Security Hub, GuardDuty, Inspector, Macie	New
5.1 VPC	Subnets, routes, IGW, NAT, SG vs NACL, endpoints	Refresh
5.1 Network protection	WAF, Shield, Network Firewall, DNS Firewall	New
5.2 DNS and delivery	Route 53 policies, Resolver, CloudFront, Global Accelerator	Refresh
5.3 Troubleshooting	Flow logs, ELB logs, Reachability Analyzer, caching	Refresh

AI-era additions in guide v1.1 (awareness level only): Amazon Bedrock, Kiro, AWS DevOps Agent, AWS Security Agent and AWS Health Dashboard were added to the in-scope list. Know what each does at a sentence level; expect them as options, not deep configuration.

Study plan

Ten weeks at 5-8 hours per week. Do the labs in the order given; each week ends with a cleanup check in Billing.

Week	Focus	Labs	Resources
1	Account, budget, IAM, CLI, exam guide	Tasks 1-2	SOA-C03 exam guide; Skill Builder pretest
2	EC2, user data, IMDSv2, EBS	Tasks 3, 9	EC2 User Guide; Skill Builder EC2 courses
3	CloudWatch agent, logs, alarms	Tasks 4-5	CloudWatch User Guide; One Observability Workshop
4	Systems Manager, EventBridge	Tasks 6-8	SSM User Guide; EventBridge User Guide

Week	Focus	Labs	Resources
5	ELB, Auto Scaling, CloudFront	Tasks 10, 17	ELB and EC2 Auto Scaling guides
6	S3, Lambda events, Backup, RDS	Tasks 11-12, 14, 20	S3 and AWS Backup guides; RDS guide
7	VPC, Route 53, network troubleshooting	Tasks 13, 16	VPC User Guide; Route 53 Developer Guide
8	CloudFormation, CDK, StackSets, RAM	Task 15	CloudFormation User Guide; CDK Workshop
9	KMS, Config, CloudTrail, Organizations	Tasks 18-19	Security pillar of Well-Architected
10	Practice questions, weak areas, book	Redo weakest 3	Official Practice Question Set and Exam

Free and official resources:

- Exam guide (free): https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03.html
- AWS Skill Builder (skillbuilder.aws): the exam page lists a 4-step exam prep plan, Official Pretest, Official Practice Question Set, Official Practice Exam, flashcards, Builder Labs, Cloud Quest, SimuLearn and Jam. Several are free; some need a Skill Builder subscription (which items are free: unverified, check before paying).
- AWS Workshops catalog (free): https://catalog.workshops.aws (search "observability", "systems manager", "cloudformation").
- AWS Well-Architected Framework, Operational Excellence and Reliability pillars (free).
- Service user guides on docs.aws.amazon.com for every service in the objectives map.

Rule of thumb: for every lab, write down the one exam question you think it answers. Week 10 is only questions, reviewing every wrong answer against the docs.

Lab setup

Everything runs in the AWS Free Tier, using the AWS CLI v2 from AWS CloudShell (free, already signed in, no local keys) or from WSL2 Ubuntu on your Windows machine. You create the account and do every sign-in yourself.

Step 1: Account and plan

- Create an AWS account at https://aws.amazon.com/free yourself. Accounts opened after July 15, 2025 choose a **Free plan** or a **Paid plan**. Both get up to 200 USD in credits (100 USD at sign-up, up to 100 USD more for completing "Explore AWS" activities in Console Home).
- The Free plan never bills you, but the account closes 6 months after opening or when credits run out, and some services are restricted. If a lab step says a feature needs the Paid plan, skip it or decide yourself whether to upgrade.
- Free-tier eligible EC2 types for these accounts: t3.micro, t3.small, t4g.micro, t4g.small, c7i-flex.large, m7i-flex.large. The labs use t3.micro.
- Secure root: enable MFA on the root user, never create root access keys, and use an admin identity (IAM Identity Center user, or an IAM user with MFA) for daily work.

Step 2: Region choice

Use **us-east-1** for all labs. Billing metrics, Route 53 health-check metrics and CloudFront certificates live there, which removes a whole class of "why can't I see the metric" problems.

Step 3: CLI (pick one)

Option A, CloudShell: open the console in us-east-1 and click the CloudShell icon. The CLI, `jq`, `zip` and the Session Manager plugin are preinstalled. Your home directory (1 GB) persists between sessions.

Option B, WSL2 Ubuntu on Windows:

```
sudo apt update && sudo apt install -y unzip jq zip curl
curl -s "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install
```

```
aws --version
# Session Manager plugin (official AWS download location)
SMP=https://s3.amazonaws.com/session-manager-downloads/plugin/latest
curl -s -o smp.deb "$SMP/ubuntu_64bit/session-manager-plugin.deb"
sudo dpkg -i smp.deb
# Sign in through IAM Identity Center (you complete the browser sign-in)
aws configure sso --profile lab
export AWS_PROFILE=lab
aws sts get-caller-identity
```

Step 4: Lab environment file

Every lab appends the IDs it creates to `~/lab.env`, so you can close the shell and continue later with `source ~/lab.env`.

```
cat > ~/lab.env <<EOF
export AWS_REGION=us-east-1
export AWS_DEFAULT_REGION=us-east-1
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export EMAIL=you@example.com
EOF
source ~/lab.env && echo "$ACCOUNT_ID"
```

Replace `you@example.com` with your own address before running it.

Step 5: Budget alert before anything else

Do **Task 1** now, before launching any resource. It sets a monthly cost budget that ignores credits, so it warns you while credits are silently being spent.

Step 6: Cost map

Resource in labs	Cost note
EC2 t3.micro, 8-30 GB EBS	Free-tier eligible; stop or terminate after labs
Public IPv4 address	Small hourly charge; credits cover it
ALB (Task 10)	Hourly plus LCU; delete within 2 hours
RDS db.t4g.micro (Task 20)	Free-tier eligible; delete same day
KMS customer key (Task 18)	About 1 USD/month, prorated; schedule deletion
Route 53 health check (Task 13)	Small monthly fee, prorated; delete after
AWS Config (Task 19)	Per item recorded; pennies if scoped
CloudWatch, Lambda, SNS, CloudFront	Always-free allowances cover the labs
NAT gateway	Not used: hourly cost is the classic surprise

Step 7: Core teardown script

Tasks 2-8 and 13-14 share one instance (`lab-web`), its security group, its IAM role and an SNS topic. Each task has its own cleanup; this script removes the shared core when you finish a study session.

```
cat > ~/teardown-core.sh <<'EOF'
#!/bin/bash
source ~/lab.env
aws ec2 terminate-instances --instance-ids "$IID"
aws ec2 wait instance-terminated --instance-ids "$IID"
aws ec2 delete-security-group --group-id "$SG"
aws iam remove-role-from-instance-profile \
  --instance-profile-name lab-ec2-profile --role-name lab-ec2-role
aws iam delete-instance-profile --instance-profile-name lab-ec2-profile
for p in AmazonSSMManagedInstanceCore CloudWatchAgentServerPolicy; do
  aws iam detach-role-policy --role-name lab-ec2-role \
    --policy-arn "arn:aws:iam::aws:policy/$p"
done
aws iam delete-role --role-name lab-ec2-role
aws sns delete-topic --topic-arn "$TOPIC"
for g in /lab/nginx/access /lab/pm2/error /lab/pm2/out; do
  aws logs delete-log-group --log-group-name "$g" 2>/dev/null
done
```

```
aws ssm delete-parameter --name AmazonCloudWatch-lab-web 2>/dev/null
echo "core removed"
EOF
chmod +x ~/teardown-core.sh
```

Once a week: Billing and Cost Management, Bills, and Cost Explorer (group by service). Also check EC2 in every Region you touched for stray instances, volumes, snapshots and Elastic IPs.

Hands-on tasks

Each task lists Task, Solution, Verify and Cleanup. Run `source ~/lab.env` at the start of every session. Commands assume bash in CloudShell or WSL2.

Task 1: Budget alerts that ignore credits

Task: Create a monthly cost budget of 5 USD for the account that emails you at 80% of actual spend and at 100% of forecast spend. The budget must measure gross usage, not usage after credits.

Solution:

```
source ~/lab.env
cat > budget.json <<'EOF'
{"BudgetName":"lab-monthly-5usd",
 "BudgetLimit":{"Amount":"5","Unit":"USD"},
 "TimeUnit":"MONTHLY","BudgetType":"COST",
 "CostTypes":{"IncludeCredit":false,"IncludeRefund":false}}
EOF
cat > notif.json <<EOF
[{"Notification":{"NotificationType":"ACTUAL","ComparisonOperator":"GREATER_THAN",
   "Threshold":80,"ThresholdType":"PERCENTAGE"},
  "Subscribers":[{"SubscriptionType":"EMAIL","Address":"$EMAIL"}]},
 {"Notification":{"NotificationType":"FORECASTED","ComparisonOperator":"GREATER_THAN",
   "Threshold":100,"ThresholdType":"PERCENTAGE"},
  "Subscribers":[{"SubscriptionType":"EMAIL","Address":"$EMAIL"}]}]
EOF
aws budgets create-budget --account-id "$ACCOUNT_ID" \
  --budget file://budget.json --notifications-with-subscribers file://notif.json
```

Why `IncludeCredit:false`: the Budgets API includes credits by default, so a normal budget shows about 0 USD while your credits drain. Optionally also add the console's "Zero spend budget" template (Billing, Budgets, Create budget, Use a template).

Verify:

```
aws budgets describe-budgets --account-id "$ACCOUNT_ID" \
  --query 'Budgets[].[BudgetName,BudgetLimit.Amount,CostTypes.IncludeCredit]'
aws budgets describe-notifications-for-budget --account-id "$ACCOUNT_ID" \
  --budget-name lab-monthly-5usd
```

Cleanup: Keep this budget for the whole study period. When you close the lab account:

```
aws budgets delete-budget --account-id "$ACCOUNT_ID" --budget-name lab-monthly-5usd
```

Task 2: IAM role for EC2, password policy and policy simulation

Task: Create an instance role that allows Systems Manager and the CloudWatch agent and nothing else. Set an account password policy (min 14 chars, all character classes, reuse prevention 5). Prove with the policy simulator that the role can register with SSM but cannot delete S3 buckets. Confirm root MFA is on.

Solution:

```
source ~/lab.env
cat > ec2-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Principal":{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name lab-ec2-role \
  --assume-role-policy-document file://ec2-trust.json
for p in AmazonSSMManagedInstanceCore CloudWatchAgentServerPolicy; do
  aws iam attach-role-policy --role-name lab-ec2-role \
    --policy-arn "arn:aws:iam::aws:policy/$p"
done
```

```
aws iam create-instance-profile --instance-profile-name lab-ec2-profile
aws iam add-role-to-instance-profile --instance-profile-name lab-ec2-profile \
  --role-name lab-ec2-role
aws iam update-account-password-policy --minimum-password-length 14 \
  --require-symbols --require-numbers --require-uppercase-characters \
  --require-lowercase-characters --password-reuse-prevention 5
```

Verify:

```
aws iam simulate-principal-policy \
  --policy-source-arn "arn:aws:iam::$ACCOUNT_ID:role/lab-ec2-role" \
  --action-names ssm:UpdateInstanceInformation s3:DeleteBucket \
  --query 'EvaluationResults[].[EvalActionName,EvalDecision]' --output table
# expect: allowed / implicitDeny
aws iam get-account-password-policy
aws iam get-account-summary --query 'SummaryMap.AccountMFAEnabled'   # expect 1
```

Cleanup: Keep the role for Tasks 3-8 (removed by `~/teardown-core.sh`). Remove the password policy when you finish the course:

```
aws iam delete-account-password-policy
```

Task 3: EC2 web server with Nginx and PM2 via user data

Task: Launch a t3.micro Amazon Linux 2023 instance in the default VPC with IMDSv2 required, the lab instance profile, no SSH port open, and tags `Name=lab-web`, `backup=daily`. User data must install Nginx and Node.js, run a small app under PM2 on 127.0.0.1:3000 (with a `/health` route), make PM2 start at boot, and proxy port 80 to it.

Solution:

```
source ~/lab.env
cat > ~/app.sh <<'EOF'
#!/bin/bash
set -euxo pipefail
dnf install -y nginx nodejs npm
npm install -g pm2
mkdir -p /opt/app
cat > /opt/app/server.js <<'JS'
const http = require('http');
const os = require('os');
http.createServer((req, res) => {
  if (req.url === '/health') { res.end('ok\n'); return; }
  res.end(`hello from ${os.hostname()}\n`);
}).listen(3000, '127.0.0.1');
JS
export HOME=/root
pm2 start /opt/app/server.js --name app
pm2 startup systemd -u root --hp /root
pm2 save
cat > /etc/nginx/conf.d/app.conf <<'NG'
server {
    listen 80;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
NG
nginx -t
systemctl enable --now nginx
EOF

VPC=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true \
  --query 'Vpcs[0].VpcId' --output text)
SG=$(aws ec2 create-security-group --group-name lab-web-sg \
  --description "lab web" --vpc-id "$VPC" --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id "$SG" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
AMI=$(aws ssm get-parameter \
  --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --query Parameter.Value --output text)
IID=$(aws ec2 run-instances --image-id "$AMI" --instance-type t3.micro \
```

```
  --subnet-id "$(aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC \
     Name=availability-zone,Values=us-east-1a --query 'Subnets[0].SubnetId' \
     --output text)" \
  --security-group-ids "$SG" --iam-instance-profile Name=lab-ec2-profile \
  --metadata-options HttpTokens=required,HttpEndpoint=enabled \
  --user-data file://$HOME/app.sh \
  --tag-specifications \
  'ResourceType=instance,Tags=[{Key=Name,Value=lab-web},{Key=backup,Value=daily}]' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-status-ok --instance-ids "$IID"
echo "export VPC=$VPC SG=$SG IID=$IID AMI=$AMI" >> ~/lab.env
```

The Nginx server block is placed in `conf.d`, which Amazon Linux's `nginx.conf` includes before its own default server, so it answers port 80 first.

Verify:

```
source ~/lab.env
PUBIP=$(aws ec2 describe-instances --instance-ids "$IID" \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
echo "export PUBIP=$PUBIP" >> ~/lab.env
curl -s "http://$PUBIP/"          # hello from ip-172-...
curl -s "http://$PUBIP/health"    # ok
aws ec2 describe-instances --instance-ids "$IID" \
  --query 'Reservations[0].Instances[0].MetadataOptions.HttpTokens'  # required
```

Cleanup: Keep `lab-web` for Tasks 4-8, 13, 14. At the end of a session either stop it (`aws ec2 stop-instances --instance-ids "$IID"`; the public IP changes on start, so rerun the PUBIP lines) or run `~/teardown-core.sh`.

Task 4: CloudWatch agent for memory, disk, Nginx and PM2 logs

Task: Using Systems Manager (no SSH), install the CloudWatch agent on `lab-web`. Store its configuration in Parameter Store. Collect `mem_used_percent` and root disk `used_percent`, ship the Nginx access log and PM2 logs to log groups with 7-day retention, then create a metric filter that counts HTTP 5xx responses.

Solution:

```
source ~/lab.env
cat > cwagent.json <<'EOF'
{
 "agent": {"metrics_collection_interval": 60, "run_as_user": "root"},
 "metrics": {
  "append_dimensions": {"InstanceId": "${aws:InstanceId}"},
  "metrics_collected": {
   "mem": {"measurement": ["mem_used_percent"]},
   "disk": {"measurement": ["used_percent"], "resources": ["/"]}
  }
 },
 "logs": {"logs_collected": {"files": {"collect_list": [
  {"file_path": "/var/log/nginx/access.log", "log_group_name": "/lab/nginx/access",
   "log_stream_name": "{instance_id}", "retention_in_days": 7},
  {"file_path": "/root/.pm2/logs/app-error.log", "log_group_name": "/lab/pm2/error",
   "log_stream_name": "{instance_id}", "retention_in_days": 7},
  {"file_path": "/root/.pm2/logs/app-out.log", "log_group_name": "/lab/pm2/out",
   "log_stream_name": "{instance_id}", "retention_in_days": 7}
 ]}}}
}
EOF
aws ssm put-parameter --name AmazonCloudWatch-lab-web --type String \
  --value file://cwagent.json --overwrite
cat > cmd.json <<'EOF'
{"commands":[
 "dnf install -y amazon-cloudwatch-agent",
 "CTL=/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl",
 "$CTL -a fetch-config -m ec2 -s -c ssm:AmazonCloudWatch-lab-web",
 "$CTL -a status"
]}
EOF
CID=$(aws ssm send-command --instance-ids "$IID" \
  --document-name AWS-RunShellScript --parameters file://cmd.json \
  --query Command.CommandId --output text)
aws ssm wait command-executed --command-id "$CID" --instance-id "$IID"
```

```
aws logs put-metric-filter --log-group-name /lab/nginx/access \
  --filter-name nginx-5xx \
  --filter-pattern '[ip, ident, user, ts, request, status=5*, bytes, ref, ua, xff]' \
  --metric-transformations \
  metricName=Nginx5xx,metricNamespace=Lab/Web,metricValue=1,defaultValue=0
```

The parameter name starts with `AmazonCloudWatch-` because the managed `CloudWatchAgentServerPolicy` only allows reading parameters with that prefix. The filter pattern matches the 10 space-delimited fields of the default Nginx "main" log format; quoted and bracketed values count as one field each.

Verify:

```
aws ssm get-command-invocation --command-id "$CID" --instance-id "$IID" \
  --query '[Status,StandardOutputContent]' --output text
aws cloudwatch list-metrics --namespace CWAgent --metric-name mem_used_percent \
  --query 'Metrics[].Dimensions'
aws logs test-metric-filter \
  --filter-pattern '[ip, ident, user, ts, request, status=5*, bytes, ref, ua, xff]' \
  --log-event-messages \
  '1.2.3.4 - - [28/Sep/2026:10:00:00 +0000] "GET / HTTP/1.1" 502 157 "-" "curl" "-"'
for i in $(seq 5); do curl -s -o /dev/null "http://$PUBIP/"; done
sleep 60
aws logs tail /lab/nginx/access --since 10m | tail -3
```

Cleanup: Keep the agent and log groups for Tasks 5 and 8. The core teardown deletes the log groups and parameter. To remove only the filter:

```
aws logs delete-metric-filter --log-group-name /lab/nginx/access --filter-name nginx-5xx
```

Task 5: Alarms, SNS, automatic recovery and a composite alarm

Task: Create an SNS topic that emails you. Create alarms for: CPU above 70% for 10 minutes, memory above 80% (from the agent), and a system status check failure that triggers EC2 recovery. Create a composite alarm that only notifies when CPU and memory are both in ALARM. Test the notification path without stressing the box.

Solution:

```
source ~/lab.env
TOPIC=$(aws sns create-topic --name lab-alerts --query TopicArn --output text)
aws sns subscribe --topic-arn "$TOPIC" --protocol email \
  --notification-endpoint "$EMAIL"      # confirm the email you receive
echo "export TOPIC=$TOPIC" >> ~/lab.env

aws cloudwatch put-metric-alarm --alarm-name lab-web-cpu-high \
  --namespace AWS/EC2 --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value="$IID" --statistic Average \
  --period 300 --evaluation-periods 2 --threshold 70 \
  --comparison-operator GreaterThanThreshold

aws cloudwatch put-metric-alarm --alarm-name lab-web-mem-high \
  --namespace CWAgent --metric-name mem_used_percent \
  --dimensions Name=InstanceId,Value="$IID" --statistic Average \
  --period 60 --evaluation-periods 5 --threshold 80 \
  --comparison-operator GreaterThanThreshold

aws cloudwatch put-metric-alarm --alarm-name lab-web-sys-check \
  --namespace AWS/EC2 --metric-name StatusCheckFailed_System \
  --dimensions Name=InstanceId,Value="$IID" --statistic Maximum \
  --period 60 --evaluation-periods 2 --threshold 0 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions "arn:aws:automate:$AWS_REGION:ec2:recover" "$TOPIC"

aws cloudwatch put-composite-alarm --alarm-name lab-web-degraded \
  --alarm-rule 'ALARM("lab-web-cpu-high") AND ALARM("lab-web-mem-high")' \
  --alarm-actions "$TOPIC"
```

The CPU and memory alarms have no actions on purpose: the composite alarm is the only one that pages you, which is the standard noise-reduction pattern.

Verify:

```
aws cloudwatch set-alarm-state --alarm-name lab-web-sys-check \
  --state-value ALARM --state-reason "notification test"
```

```
# the email arrives; the recover action is not run by a manual state change
aws cloudwatch describe-alarms --alarm-name-prefix lab-web \
  --query 'MetricAlarms[].[AlarmName,StateValue]' --output table
aws cloudwatch describe-alarms --alarm-types CompositeAlarm \
  --query 'CompositeAlarms[].[AlarmName,StateValue,AlarmRule]'
```

The alarm returns to OK at the next evaluation.

Cleanup:

```
aws cloudwatch delete-alarms --alarm-names lab-web-degraded
aws cloudwatch delete-alarms --alarm-names lab-web-cpu-high lab-web-mem-high \
  lab-web-sys-check
```

Keep the SNS topic for Tasks 8 and 13 (core teardown deletes it). Delete the composite alarm first, because a metric alarm referenced by a composite alarm cannot be deleted.

Task 6: Session Manager, Run Command and Parameter Store

Task: Open a shell on `lab-web` with no port 22 and no key pair. Use Run Command, targeting by tag, to check Nginx and PM2 status. Store a database URL as a SecureString parameter and have the app read it at start through PM2.

Solution:

```
source ~/lab.env
aws ssm describe-instance-information \
  --query 'InstanceInformationList[].[InstanceId,PingStatus,PlatformName]'
aws ssm start-session --target "$IID"
# inside the session:
#   sudo -i
#   pm2 ls; systemctl status nginx --no-pager; tail -20 /var/log/cloud-init-output.log
#   exit; exit

cat > status.json <<'EOF'
{"commands":["systemctl is-active nginx","export HOME=/root","pm2 jlist | head -c 300"]}
EOF
CID=$(aws ssm send-command --targets Key=tag:Name,Values=lab-web \
  --document-name AWS-RunShellScript --parameters file://status.json \
  --query Command.CommandId --output text)

aws ssm put-parameter --name /lab/app/DB_URL --type SecureString \
  --value "mysql://app:not-a-real-pass@db.internal:3306/app"
cat > env.json <<'EOF'
{"commands":[
 "export HOME=/root",
 "P=/lab/app/DB_URL",
 "Q='--with-decryption --query Parameter.Value --output text'",
 "export DB_URL=$(aws ssm get-parameter --name $P $Q)",
 "pm2 restart app --update-env",
 "pm2 env 0 | grep -c DB_URL"
]}
EOF
CID2=$(aws ssm send-command --instance-ids "$IID" \
  --document-name AWS-RunShellScript --parameters file://env.json \
  --query Command.CommandId --output text)
```

The instance can decrypt because `AmazonSSMManagedInstanceCore` allows `ssm:GetParameter` and the AWS managed `aws/ssm` key is usable by principals in the account through SSM. With a customer managed key you would also need `kms:Decrypt` on that key.

Verify:

```
aws ssm list-command-invocations --command-id "$CID" --details \
  --query 'CommandInvocations[].CommandPlugins[].Output' --output text
aws ssm get-command-invocation --command-id "$CID2" --instance-id "$IID" \
  --query '[Status,StandardOutputContent]' --output text   # Success, 1
aws ec2 describe-security-groups --group-ids "$SG" \
  --query 'SecurityGroups[0].IpPermissions[].FromPort'     # only 80
```

Cleanup:

```
aws ssm delete-parameter --name /lab/app/DB_URL
```

Task 7: Patch Manager, a maintenance window and an Automation runbook

Task: Scan `lab-web` for missing patches. Create a weekly maintenance window (Sunday 03:00 UTC, 2 hours) that installs patches on instances tagged `Name=lab-web`. Write a custom Automation runbook that restarts the PM2 app and checks `/health`, then run it and the AWS-managed `AWS-RestartEC2Instance` runbook.

Solution:

```
source ~/lab.env
aws ssm send-command --document-name AWS-RunPatchBaseline \
  --targets Key=tag:Name,Values=lab-web --parameters Operation=Scan

MW=$(aws ssm create-maintenance-window --name lab-patching \
  --schedule "cron(0 3 ? * SUN *)" --duration 2 --cutoff 1 \
  --no-allow-unassociated-targets --query WindowId --output text)
MWT=$(aws ssm register-target-with-maintenance-window --window-id "$MW" \
  --resource-type INSTANCE --targets Key=tag:Name,Values=lab-web \
  --query WindowTargetId --output text)
aws ssm register-task-with-maintenance-window --window-id "$MW" \
  --task-arn AWS-RunPatchBaseline --task-type RUN_COMMAND \
  --targets Key=WindowTargetIds,Values="$MWT" \
  --max-concurrency 1 --max-errors 1 \
  --task-invocation-parameters \
  '{"RunCommand":{"Parameters":{"Operation":["Install"]}}}'
echo "export MW=$MW" >> ~/lab.env

cat > restart-app.yaml <<'EOF'
schemaVersion: '0.3'
description: Restart the PM2 app and check health
parameters:
  InstanceId:
    type: String
mainSteps:
  - name: restartApp
    action: aws:runCommand
    inputs:
      DocumentName: AWS-RunShellScript
      InstanceIds:
        - '{{ InstanceId }}'
      Parameters:
        commands:
          - export HOME=/root
          - pm2 restart app
          - sleep 3
          - curl -fsS http://127.0.0.1/health
EOF
aws ssm create-document --name Lab-RestartPm2App --document-type Automation \
  --document-format YAML --content file://restart-app.yaml
EX=$(aws ssm start-automation-execution --document-name Lab-RestartPm2App \
  --parameters InstanceId="$IID" --query AutomationExecutionId --output text)
```

Verify:

```
aws ssm describe-instance-patch-states --instance-ids "$IID" \
  --query 'InstancePatchStates[0].[MissingCount,InstalledCount,Operation]'
aws ssm describe-maintenance-window-tasks --window-id "$MW" \
  --query 'Tasks[].[TaskArn,Type]'
sleep 30
aws ssm get-automation-execution --automation-execution-id "$EX" \
  --query 'AutomationExecution.AutomationExecutionStatus'     # Success
EX2=$(aws ssm start-automation-execution --document-name AWS-RestartEC2Instance \
  --parameters InstanceId="$IID" --query AutomationExecutionId --output text)
aws ssm get-automation-execution --automation-execution-id "$EX2" \
  --query 'AutomationExecution.StepExecutions[].[StepName,StepStatus]'
```

After the reboot, `curl http://$PUBIP/` should work again without intervention; that proves `pm2 startup` did its job.

Cleanup:

```
aws ssm delete-maintenance-window --window-id "$MW"
aws ssm delete-document --name Lab-RestartPm2App
```

Task 8: Self-healing with CloudWatch, EventBridge and Run Command

Task: When the Nginx 5xx count reaches 5 in one minute, alarm to SNS and, through an EventBridge rule, automatically run `pm2 restart app` on the tagged instance. Prove it by stopping the app and generating 502s.

Solution:

```
source ~/lab.env
aws cloudwatch put-metric-alarm --alarm-name lab-web-5xx \
  --namespace Lab/Web --metric-name Nginx5xx --statistic Sum \
  --period 60 --evaluation-periods 1 --threshold 5 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --treat-missing-data notBreaching --alarm-actions "$TOPIC"

cat > events-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Principal":{"Service":"events.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
cat > events-ssm.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":"ssm:SendCommand",
 "Resource":["arn:aws:ssm:$AWS_REGION::document/AWS-RunShellScript",
             "arn:aws:ec2:$AWS_REGION:$ACCOUNT_ID:instance/*"]}]}
EOF
aws iam create-role --role-name lab-events-ssm \
  --assume-role-policy-document file://events-trust.json
aws iam put-role-policy --role-name lab-events-ssm --policy-name send-command \
  --policy-document file://events-ssm.json

cat > pattern.json <<'EOF'
{"source":["aws.cloudwatch"],
 "detail-type":["CloudWatch Alarm State Change"],
 "detail":{"alarmName":["lab-web-5xx"],"state":{"value":["ALARM"]}}}
EOF
aws events put-rule --name lab-restart-on-5xx --event-pattern file://pattern.json
cat > targets.json <<EOF
[{"Id":"restart-app",
  "Arn":"arn:aws:ssm:$AWS_REGION::document/AWS-RunShellScript",
  "RoleArn":"arn:aws:iam::$ACCOUNT_ID:role/lab-events-ssm",
  "Input":"{\"commands\":[\"export HOME=/root\",\"pm2 restart app\"]}",
  "RunCommandParameters":{"RunCommandTargets":[
    {"Key":"tag:Name","Values":["lab-web"]}]}}]
EOF
aws events put-targets --rule lab-restart-on-5xx --targets file://targets.json
```

Verify:

```
aws ssm send-command --instance-ids "$IID" --document-name AWS-RunShellScript \
  --parameters 'commands=["export HOME=/root","pm2 stop app"]'
sleep 5
for i in $(seq 20); do curl -s -o /dev/null -w "%{http_code}\n" "http://$PUBIP/"; done
# wait 2-4 minutes for log delivery, metric and alarm evaluation
aws cloudwatch describe-alarms --alarm-names lab-web-5xx \
  --query 'MetricAlarms[0].StateValue'
aws ssm list-commands --max-results 3 \
  --query 'Commands[].[RequestedDateTime,DocumentName,Status]' --output table
curl -s "http://$PUBIP/"     # hello again, restarted by the rule
```

Cleanup:

```
aws events remove-targets --rule lab-restart-on-5xx --ids restart-app
aws events delete-rule --name lab-restart-on-5xx
aws iam delete-role-policy --role-name lab-events-ssm --policy-name send-command
aws iam delete-role --role-name lab-events-ssm
aws cloudwatch delete-alarms --alarm-names lab-web-5xx
```

Task 9: EBS volume: attach, mount, grow online, snapshot

Task: Add a 1 GiB gp3 data volume to `lab-web`, format it XFS, mount it persistently at `/data`, then grow it to 2 GiB without downtime and snapshot it.

Solution:

```
source ~/lab.env
AZ=$(aws ec2 describe-instances --instance-ids "$IID" \
  --query 'Reservations[0].Instances[0].Placement.AvailabilityZone' --output text)
VOL=$(aws ec2 create-volume --availability-zone "$AZ" --size 1 --volume-type gp3 \
```

```
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=lab-data}]' \
  --query VolumeId --output text)
aws ec2 wait volume-available --volume-ids "$VOL"
aws ec2 attach-volume --volume-id "$VOL" --instance-id "$IID" --device /dev/sdf
echo "export AZ=$AZ VOL=$VOL" >> ~/lab.env
aws ssm start-session --target "$IID"
```

Inside the session (Nitro instances expose EBS as NVMe; match the serial to the volume ID):

```
sudo -i
lsblk -o NAME,SIZE,SERIAL          # e.g. nvme1n1  1G  vol0abc...
mkfs.xfs /dev/nvme1n1
mkdir -p /data
UUID=$(blkid -s UUID -o value /dev/nvme1n1)
echo "UUID=$UUID /data xfs defaults,nofail 0 2" >> /etc/fstab
mount -a && df -h /data
```

Back in CloudShell, grow it, then grow the filesystem in the session:

```
aws ec2 modify-volume --volume-id "$VOL" --size 2
aws ec2 describe-volumes-modifications --volume-ids "$VOL" \
  --query 'VolumesModifications[0].ModificationState'   # optimizing or completed
# in the session:  xfs_growfs /data && df -h /data
SNAP=$(aws ec2 create-snapshot --volume-id "$VOL" --description lab-data \
  --query SnapshotId --output text)
aws ec2 wait snapshot-completed --snapshot-ids "$SNAP"
echo "export SNAP=$SNAP" >> ~/lab.env
```

`nofail` keeps the instance bootable if the volume is missing. After a modification, the same volume cannot be modified again for 6 hours.

Verify:

```
aws ec2 describe-volumes --volume-ids "$VOL" \
  --query 'Volumes[0].[Size,VolumeType,Iops,Throughput,State]'   # 2 gp3 3000 125
aws ec2 describe-snapshots --snapshot-ids "$SNAP" --query 'Snapshots[0].State'
# in the session: df -h /data shows about 2.0G
```

Cleanup: In the session run `umount /data` and delete the `/data` line from `/etc/fstab` (`sed -i '\#/data xfs#d' /etc/fstab`). Then:

```
aws ec2 detach-volume --volume-id "$VOL"
aws ec2 wait volume-available --volume-ids "$VOL"
aws ec2 delete-volume --volume-id "$VOL"
aws ec2 delete-snapshot --snapshot-id "$SNAP"
```

Task 10: Golden AMI, launch template, ALB and Auto Scaling

Task: Bake an AMI from `lab-web` (PM2 and Nginx already configured). Put 2 instances from a launch template behind an Application Load Balancer across two AZs, using ELB health checks on `/health`, a 120-second grace period and a target-tracking policy at 50% CPU. Prove self-healing by killing an instance.

Solution:

```
source ~/lab.env
GOLD=$(aws ec2 create-image --instance-id "$IID" --name "lab-web-$(date +%Y%m%d%H%M)" \
  --query ImageId --output text)
aws ec2 wait image-available --image-ids "$GOLD"
cat > lt.json <<EOF
{"ImageId":"$GOLD","InstanceType":"t3.micro",
 "IamInstanceProfile":{"Name":"lab-ec2-profile"},
 "SecurityGroupIds":["$SG"],
 "MetadataOptions":{"HttpTokens":"required"},
 "TagSpecifications":[{"ResourceType":"instance",
   "Tags":[{"Key":"Name","Value":"lab-asg"}]}]}
EOF
aws ec2 create-launch-template --launch-template-name lab-web-lt \
  --launch-template-data file://lt.json
SUBNETS=$(aws ec2 describe-subnets --filters Name=vpc-id,Values="$VPC" \
  Name=availability-zone,Values=us-east-1a,us-east-1b \
  --query 'Subnets[].SubnetId' --output text)
ALB_ARN=$(aws elbv2 create-load-balancer --name lab-alb --subnets $SUBNETS \
  --security-groups "$SG" --query 'LoadBalancers[0].LoadBalancerArn' --output text)
TG_ARN=$(aws elbv2 create-target-group --name lab-tg --protocol HTTP --port 80 \
```

```
  --vpc-id "$VPC" --health-check-path /health --health-check-interval-seconds 10 \
  --healthy-threshold-count 2 --query 'TargetGroups[0].TargetGroupArn' --output text)
aws elbv2 create-listener --load-balancer-arn "$ALB_ARN" --protocol HTTP --port 80 \
  --default-actions Type=forward,TargetGroupArn="$TG_ARN"
aws autoscaling create-auto-scaling-group --auto-scaling-group-name lab-asg \
  --launch-template 'LaunchTemplateName=lab-web-lt,Version=$Latest' \
  --min-size 1 --max-size 3 --desired-capacity 2 \
  --vpc-zone-identifier "$(echo $SUBNETS | tr ' ' ',')" \
  --target-group-arns "$TG_ARN" --health-check-type ELB \
  --health-check-grace-period 120
cat > tt.json <<'EOF'
{"PredefinedMetricSpecification":{"PredefinedMetricType":"ASGAverageCPUUtilization"},
 "TargetValue":50}
EOF
aws autoscaling put-scaling-policy --auto-scaling-group-name lab-asg \
  --policy-name cpu50 --policy-type TargetTrackingScaling \
  --target-tracking-configuration file://tt.json
echo "export GOLD=$GOLD ALB_ARN=$ALB_ARN TG_ARN=$TG_ARN" >> ~/lab.env
```

For the lab the ALB and instances share `lab-web-sg`. In production give the ALB its own security group and allow port 80 on the instances only from that group.

Verify:

```
DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns "$ALB_ARN" \
  --query 'LoadBalancers[0].DNSName' --output text)
aws elbv2 describe-target-health --target-group-arn "$TG_ARN" \
  --query 'TargetHealthDescriptions[].[Target.Id,TargetHealth.State]'
for i in $(seq 6); do curl -s "http://$DNS/"; done   # two different hostnames
VICTIM=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names lab-asg \
  --query 'AutoScalingGroups[0].Instances[0].InstanceId' --output text)
aws ec2 terminate-instances --instance-ids "$VICTIM"
sleep 120
aws autoscaling describe-scaling-activities --auto-scaling-group-name lab-asg \
  --max-items 3 --query 'Activities[].[StatusCode,Description]'
```

Cleanup: Run it within 2 hours; the ALB bills hourly.

```
source ~/lab.env
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name lab-asg \
  --force-delete
aws elbv2 delete-load-balancer --load-balancer-arn "$ALB_ARN"
aws elbv2 wait load-balancers-deleted --load-balancer-arns "$ALB_ARN"
aws ec2 wait instance-terminated --filters Name=tag:Name,Values=lab-asg
aws elbv2 delete-target-group --target-group-arn "$TG_ARN"
aws ec2 delete-launch-template --launch-template-name lab-web-lt
SNAPS=$(aws ec2 describe-images --image-ids "$GOLD" \
  --query 'Images[0].BlockDeviceMappings[].Ebs.SnapshotId' --output text)
aws ec2 deregister-image --image-id "$GOLD"
for s in $SNAPS; do aws ec2 delete-snapshot --snapshot-id "$s"; done
```

Task 11: S3 versioning, lifecycle and encryption for log archives

Task: Create a private bucket for rotated Nginx and PM2 logs. Enable versioning, confirm Block Public Access and default encryption, and add a lifecycle rule for `logs/`: Standard-IA at 30 days, Glacier Instant Retrieval at 90, delete at 365, expire noncurrent versions after 30 days, abort incomplete multipart uploads after 7 days. Recover an accidentally deleted object.

Solution:

```
source ~/lab.env
B="lab-logs-$ACCOUNT_ID"
aws s3api create-bucket --bucket "$B"        # us-east-1 needs no location config
BPA=BlockPublicAcls=true,IgnorePublicAcls=true
BPA=$BPA,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api put-public-access-block --bucket "$B" \
  --public-access-block-configuration "$BPA"
aws s3api put-bucket-versioning --bucket "$B" \
  --versioning-configuration Status=Enabled
cat > lifecycle.json <<'EOF'
{"Rules":[{"ID":"logs-tiering","Status":"Enabled","Filter":{"Prefix":"logs/"},
  "Transitions":[{"Days":30,"StorageClass":"STANDARD_IA"},
```

```
                {"Days":90,"StorageClass":"GLACIER_IR"}],
  "Expiration":{"Days":365},
  "NoncurrentVersionExpiration":{"NoncurrentDays":30},
  "AbortIncompleteMultipartUpload":{"DaysAfterInitiation":7}}]}
EOF
aws s3api put-bucket-lifecycle-configuration --bucket "$B" \
  --lifecycle-configuration file://lifecycle.json
echo "export B=$B" >> ~/lab.env

echo "GET / 200" > access-2026-09-28.log
aws s3 cp access-2026-09-28.log "s3://$B/logs/web/access-2026-09-28.log"
echo "GET / 502" >> access-2026-09-28.log
aws s3 cp access-2026-09-28.log "s3://$B/logs/web/access-2026-09-28.log"
aws s3 rm "s3://$B/logs/web/access-2026-09-28.log"      # "accident"
```

On a real server, a daily cron or a `postrotate` hook in logrotate would push the rotated file with `aws s3 cp`, using an instance role limited to `s3:PutObject` on `arn:aws:s3:::BUCKET/logs/*`.

Verify and recover:

```
aws s3api get-bucket-encryption --bucket "$B" \
  --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault'
aws s3api get-bucket-lifecycle-configuration --bucket "$B" --query 'Rules[0].ID'
aws s3api list-object-versions --bucket "$B" --prefix logs/web/ \
  --query '{v:Versions[].VersionId,d:DeleteMarkers[].VersionId}'
DM=$(aws s3api list-object-versions --bucket "$B" --prefix logs/web/ \
  --query 'DeleteMarkers[0].VersionId' --output text)
aws s3api delete-object --bucket "$B" --key logs/web/access-2026-09-28.log \
  --version-id "$DM"                                    # removes the marker
aws s3 cp "s3://$B/logs/web/access-2026-09-28.log" -     # object is back
```

Cleanup: A versioned bucket must have every version and delete marker removed first (or use Empty in the console).

```
aws s3api list-object-versions --bucket "$B" --output json \
  --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' > v.json
aws s3api delete-objects --bucket "$B" --delete file://v.json
aws s3api list-object-versions --bucket "$B" --output json \
  --query '{Objects: DeleteMarkers[].{Key:Key,VersionId:VersionId}}' > m.json
aws s3api delete-objects --bucket "$B" --delete file://m.json 2>/dev/null
aws s3api delete-bucket --bucket "$B"
```

Task 12: Event-driven processing: S3 upload triggers Lambda

Task: When an object is created under `uploads/` in a bucket, invoke a Python Lambda function that logs the bucket, key and size as JSON.

Solution:

```
source ~/lab.env
EB="lab-events-$ACCOUNT_ID"
aws s3api create-bucket --bucket "$EB"
cat > lambda-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Principal":{"Service":"lambda.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name lab-lambda-s3 \
  --assume-role-policy-document file://lambda-trust.json
aws iam attach-role-policy --role-name lab-lambda-s3 --policy-arn \
  arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
cat > app.py <<'EOF'
import json
import urllib.parse

def handler(event, context):
    for r in event["Records"]:
        key = urllib.parse.unquote_plus(r["s3"]["object"]["key"])
        print(json.dumps({"bucket": r["s3"]["bucket"]["name"],
                          "key": key, "size": r["s3"]["object"].get("size", 0)}))
    return {"ok": True}
EOF
zip -q fn.zip app.py
sleep 10     # let the new role propagate
```

```
FN_ARN=$(aws lambda create-function --function-name lab-s3-events \
  --runtime python3.13 --handler app.handler --zip-file fileb://fn.zip \
  --role "arn:aws:iam::$ACCOUNT_ID:role/lab-lambda-s3" \
  --query FunctionArn --output text)
aws lambda wait function-active-v2 --function-name lab-s3-events
aws lambda add-permission --function-name lab-s3-events --statement-id s3invoke \
  --action lambda:InvokeFunction --principal s3.amazonaws.com \
  --source-arn "arn:aws:s3:::$EB" --source-account "$ACCOUNT_ID"
cat > notif-s3.json <<EOF
{"LambdaFunctionConfigurations":[{"LambdaFunctionArn":"$FN_ARN",
  "Events":["s3:ObjectCreated:*"],
  "Filter":{"Key":{"FilterRules":[{"Name":"prefix","Value":"uploads/"}]}}}]}
EOF
aws s3api put-bucket-notification-configuration --bucket "$EB" \
  --notification-configuration file://notif-s3.json
echo "export EB=$EB" >> ~/lab.env
```

Verify:

```
echo hi > test.txt
aws s3 cp test.txt "s3://$EB/uploads/test.txt"
aws s3 cp test.txt "s3://$EB/other/test.txt"      # must NOT trigger
sleep 15
aws logs tail /aws/lambda/lab-s3-events --since 5m | grep '"key"'
```

Cleanup:

```
aws s3 rm "s3://$EB" --recursive
aws s3api delete-bucket --bucket "$EB"
aws lambda delete-function --function-name lab-s3-events
aws logs delete-log-group --log-group-name /aws/lambda/lab-s3-events
aws iam detach-role-policy --role-name lab-lambda-s3 --policy-arn \
  arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam delete-role --role-name lab-lambda-s3
```

Task 13: Route 53 health check with a CloudWatch alarm

Task: Create an HTTP health check for `http://PUBLIC_IP/health` every 30 seconds with a failure threshold of 3. Alarm to SNS when it reports unhealthy. Break Nginx and watch it fail. Optionally, if you own a domain, wire failover routing.

Solution:

```
source ~/lab.env
HCC="IPAddress=$PUBIP,Port=80,Type=HTTP,ResourcePath=/health"
HCC="$HCC,RequestInterval=30,FailureThreshold=3"
HC=$(aws route53 create-health-check --caller-reference "lab-$(date +%s)" \
  --health-check-config "$HCC" \
  --query HealthCheck.Id --output text)
aws route53 change-tags-for-resource --resource-type healthcheck \
  --resource-id "$HC" --add-tags Key=Name,Value=lab-web
aws cloudwatch put-metric-alarm --alarm-name lab-web-r53 \
  --namespace AWS/Route53 --metric-name HealthCheckStatus \
  --dimensions Name=HealthCheckId,Value="$HC" --statistic Minimum \
  --period 60 --evaluation-periods 1 --threshold 1 \
  --comparison-operator LessThanThreshold --alarm-actions "$TOPIC"
echo "export HC=$HC" >> ~/lab.env
```

Route 53 health-check metrics are only published in us-east-1, so the alarm and its SNS topic must be there. The health checkers come from public Route 53 IP ranges; the security group must allow them (here port 80 is open to all).

Optional failover (needs a hosted zone you own; about 0.50 USD/month): create a PRIMARY record pointing to the instance with `HealthCheckId` set, and a SECONDARY record pointing to, for example, a static "maintenance" S3 website or another instance.

```
{"Changes":[{"Action":"UPSERT","ResourceRecordSet":{
  "Name":"app.example.com","Type":"A","SetIdentifier":"primary",
  "Failover":"PRIMARY","TTL":60,"HealthCheckId":"HC_ID",
  "ResourceRecords":[{"Value":"PUBLIC_IP"}]}}]}
```

Verify:

```
aws route53 get-health-check-status --health-check-id "$HC" \
  --query 'HealthCheckObservations[].StatusReport.Status' | head -5
aws ssm send-command --instance-ids "$IID" --document-name AWS-RunShellScript \
  --parameters 'commands=["systemctl stop nginx"]'
sleep 180
aws cloudwatch describe-alarms --alarm-names lab-web-r53 \
  --query 'MetricAlarms[0].StateValue'          # ALARM, email received
aws ssm send-command --instance-ids "$IID" --document-name AWS-RunShellScript \
  --parameters 'commands=["systemctl start nginx"]'
```

Cleanup:

```
aws cloudwatch delete-alarms --alarm-names lab-web-r53
aws route53 delete-health-check --health-check-id "$HC"
```

Task 14: AWS Backup plan and a tested restore

Task: Create a backup vault and a daily plan (05:00 UTC, 7-day retention) that selects resources tagged `backup=daily`. Run an on-demand backup of the `lab-web` root volume and restore it to a new volume.

Solution:

```
source ~/lab.env
cat > backup-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Principal":{"Service":"backup.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name lab-backup-role \
  --assume-role-policy-document file://backup-trust.json
for p in AWSBackupServiceRolePolicyForBackup AWSBackupServiceRolePolicyForRestores; do
  aws iam attach-role-policy --role-name lab-backup-role \
    --policy-arn "arn:aws:iam::aws:policy/service-role/$p"
done
BROLE="arn:aws:iam::$ACCOUNT_ID:role/lab-backup-role"
aws backup create-backup-vault --backup-vault-name lab-vault
cat > plan.json <<'EOF'
{"BackupPlanName":"lab-daily",
 "Rules":[{"RuleName":"daily-7d","TargetBackupVaultName":"lab-vault",
   "ScheduleExpression":"cron(0 5 ? * * *)","StartWindowMinutes":60,
   "Lifecycle":{"DeleteAfterDays":7}}]}
EOF
PLAN=$(aws backup create-backup-plan --backup-plan file://plan.json \
  --query BackupPlanId --output text)
cat > selection.json <<EOF
{"SelectionName":"tag-backup-daily","IamRoleArn":"$BROLE",
 "ListOfTags":[{"ConditionType":"STRINGEQUALS","ConditionKey":"backup",
   "ConditionValue":"daily"}]}
EOF
aws backup create-backup-selection --backup-plan-id "$PLAN" \
  --backup-selection file://selection.json
ROOTVOL=$(aws ec2 describe-instances --instance-ids "$IID" --query \
  'Reservations[0].Instances[0].BlockDeviceMappings[0].Ebs.VolumeId' --output text)
JOB=$(aws backup start-backup-job --backup-vault-name lab-vault \
  --resource-arn "arn:aws:ec2:$AWS_REGION:$ACCOUNT_ID:volume/$ROOTVOL" \
  --iam-role-arn "$BROLE" --lifecycle DeleteAfterDays=7 \
  --query BackupJobId --output text)
echo "export BROLE=$BROLE PLAN=$PLAN JOB=$JOB" >> ~/lab.env
```

Wait until the job is COMPLETED (a few minutes), then restore:

```
aws backup describe-backup-job --backup-job-id "$JOB" --query State
RP=$(aws backup describe-backup-job --backup-job-id "$JOB" \
  --query RecoveryPointArn --output text)
aws backup get-recovery-point-restore-metadata --backup-vault-name lab-vault \
  --recovery-point-arn "$RP" --query RestoreMetadata > meta.json
cat meta.json
# if there is no availabilityZone key, add "availabilityZone":"us-east-1a"
RJ=$(aws backup start-restore-job --recovery-point-arn "$RP" \
  --iam-role-arn "$BROLE" --metadata file://meta.json \
  --query RestoreJobId --output text)
echo "export RP=$RP RJ=$RJ" >> ~/lab.env
```

Verify:

```
aws backup list-protected-resources --query 'Results[].ResourceArn'
aws backup describe-restore-job --restore-job-id "$RJ" \
  --query '[Status,CreatedResourceArn]'          # COMPLETED, arn:...:volume/vol-...
aws backup list-backup-selections --backup-plan-id "$PLAN"
```

Cleanup:

```
NEWVOL=$(aws backup describe-restore-job --restore-job-id "$RJ" \
  --query CreatedResourceArn --output text | awk -F/ '{print $NF}')
aws ec2 delete-volume --volume-id "$NEWVOL"
SEL=$(aws backup list-backup-selections --backup-plan-id "$PLAN" \
  --query 'BackupSelectionsList[0].SelectionId' --output text)
aws backup delete-backup-selection --backup-plan-id "$PLAN" --selection-id "$SEL"
aws backup delete-backup-plan --backup-plan-id "$PLAN"
for rp in $(aws backup list-recovery-points-by-backup-vault \
    --backup-vault-name lab-vault \
    --query 'RecoveryPoints[].RecoveryPointArn' --output text); do
  aws backup delete-recovery-point --backup-vault-name lab-vault \
    --recovery-point-arn "$rp"
done
sleep 30
aws backup delete-backup-vault --backup-vault-name lab-vault
for p in AWSBackupServiceRolePolicyForBackup AWSBackupServiceRolePolicyForRestores; do
  aws iam detach-role-policy --role-name lab-backup-role \
    --policy-arn "arn:aws:iam::aws:policy/service-role/$p"
done
aws iam delete-role --role-name lab-backup-role
```

Task 15: CloudFormation stack, change set and drift detection

Task: Deploy a stack with a security group and an EC2 instance whose AMI comes from the public SSM parameter. Preview a change of instance type with a change set, execute it, then create drift by hand and detect it.

Solution:

```
source ~/lab.env
cat > web.yaml <<'EOF'
AWSTemplateFormatVersion: '2010-09-09'
Description: Lab web server
Parameters:
  InstanceType:
    Type: String
    Default: t3.micro
    AllowedValues: [t3.micro, t3.small]
  LatestAmi:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
Resources:
  WebSG:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: cfn lab web
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 80
          ToPort: 80
          CidrIp: 0.0.0.0/0
  Web:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !Ref LatestAmi
      InstanceType: !Ref InstanceType
      SecurityGroupIds: [!GetAtt WebSG.GroupId]
      IamInstanceProfile: lab-ec2-profile
      UserData: !Base64 |
        #!/bin/bash
        dnf install -y nginx
        systemctl enable --now nginx
      Tags:
        - Key: Name
          Value: lab-cfn-web
Outputs:
  PublicIp:
```

```
    Value: !GetAtt Web.PublicIp
  SecurityGroupId:
    Value: !GetAtt WebSG.GroupId
EOF
aws cloudformation validate-template --template-body file://web.yaml
aws cloudformation deploy --stack-name lab-cfn --template-file web.yaml
aws cloudformation create-change-set --stack-name lab-cfn --change-set-name to-small \
  --use-previous-template --parameters ParameterKey=InstanceType,ParameterValue=t3.small \
  ParameterKey=LatestAmi,UsePreviousValue=true
aws cloudformation wait change-set-create-complete --stack-name lab-cfn \
  --change-set-name to-small
aws cloudformation describe-change-set --stack-name lab-cfn --change-set-name to-small \
  --query 'Changes[].ResourceChange.[LogicalResourceId,Action,Replacement]'
aws cloudformation execute-change-set --stack-name lab-cfn --change-set-name to-small
aws cloudformation wait stack-update-complete --stack-name lab-cfn
CSG=$(aws cloudformation describe-stacks --stack-name lab-cfn \
  --query "Stacks[0].Outputs[?OutputKey=='SecurityGroupId'].OutputValue" --output text)
aws ec2 authorize-security-group-ingress --group-id "$CSG" \
  --protocol tcp --port 22 --cidr 0.0.0.0/0          # manual change = drift
DID=$(aws cloudformation detect-stack-drift --stack-name lab-cfn \
  --query StackDriftDetectionId --output text)
```

The change set shows `Modify` with `Replacement: False` for the instance: changing the instance type on an EBS-backed instance is an update with some interruption (stop and start), not a replacement.

Verify:

```
aws cloudformation describe-stack-drift-detection-status \
  --stack-drift-detection-id "$DID" --query '[DetectionStatus,StackDriftStatus]'
aws cloudformation describe-stack-resource-drifts --stack-name lab-cfn \
  --query 'StackResourceDrifts[].[LogicalResourceId,StackResourceDriftStatus]'
# WebSG MODIFIED, Web IN_SYNC
CIP=$(aws cloudformation describe-stacks --stack-name lab-cfn \
  --query "Stacks[0].Outputs[?OutputKey=='PublicIp'].OutputValue" --output text)
curl -s "http://$CIP/" | head -3
```

Cleanup:

```
aws cloudformation delete-stack --stack-name lab-cfn
aws cloudformation wait stack-delete-complete --stack-name lab-cfn
```

Task 16: Build a VPC, add an S3 gateway endpoint, troubleshoot with flow logs

Task: Build `10.20.0.0/16` with two public subnets in different AZs, an internet gateway, a route table and an S3 gateway endpoint (no NAT gateway). Launch the Nginx/PM2 instance there, enable VPC flow logs to CloudWatch Logs, then block your own IP with a network ACL and find the REJECT records.

Solution:

```
source ~/lab.env
V=$(aws ec2 create-vpc --cidr-block 10.20.0.0/16 --tag-specifications \
  'ResourceType=vpc,Tags=[{Key=Name,Value=lab-vpc}]' --query Vpc.VpcId --output text)
aws ec2 modify-vpc-attribute --vpc-id "$V" --enable-dns-hostnames Value=true
SUBA=$(aws ec2 create-subnet --vpc-id "$V" --cidr-block 10.20.1.0/24 \
  --availability-zone us-east-1a --query Subnet.SubnetId --output text)
SUBB=$(aws ec2 create-subnet --vpc-id "$V" --cidr-block 10.20.2.0/24 \
  --availability-zone us-east-1b --query Subnet.SubnetId --output text)
IGW=$(aws ec2 create-internet-gateway \
  --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --internet-gateway-id "$IGW" --vpc-id "$V"
RT=$(aws ec2 create-route-table --vpc-id "$V" --query RouteTable.RouteTableId \
  --output text)
aws ec2 create-route --route-table-id "$RT" --destination-cidr-block 0.0.0.0/0 \
  --gateway-id "$IGW"
aws ec2 associate-route-table --route-table-id "$RT" --subnet-id "$SUBA"
aws ec2 associate-route-table --route-table-id "$RT" --subnet-id "$SUBB"
aws ec2 modify-subnet-attribute --subnet-id "$SUBA" --map-public-ip-on-launch
S3EP=$(aws ec2 create-vpc-endpoint --vpc-id "$V" --vpc-endpoint-type Gateway \
  --service-name "com.amazonaws.$AWS_REGION.s3" --route-table-ids "$RT" \
  --query VpcEndpoint.VpcEndpointId --output text)
VSG=$(aws ec2 create-security-group --group-name lab-vpc-web --description "vpc web" \
  --vpc-id "$V" --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id "$VSG" --protocol tcp --port 80 \
```

```
  --cidr 0.0.0.0/0
VIID=$(aws ec2 run-instances --image-id "$AMI" --instance-type t3.micro \
  --subnet-id "$SUBA" --security-group-ids "$VSG" \
  --iam-instance-profile Name=lab-ec2-profile \
  --metadata-options HttpTokens=required --user-data file://$HOME/app.sh \
  --query 'Instances[0].InstanceId' --output text)
cat > fl-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Principal":{"Service":"vpc-flow-logs.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
cat > fl-policy.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Resource":"*",
 "Action":["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents",
           "logs:DescribeLogGroups","logs:DescribeLogStreams"]}]}
EOF
aws iam create-role --role-name lab-flowlogs \
  --assume-role-policy-document file://fl-trust.json
aws iam put-role-policy --role-name lab-flowlogs --policy-name write-logs \
  --policy-document file://fl-policy.json
sleep 10
FL=$(aws ec2 create-flow-logs --resource-type VPC --resource-ids "$V" \
  --traffic-type ALL --log-destination-type cloud-watch-logs \
  --log-group-name /lab/vpc/flow --max-aggregation-interval 60 \
  --deliver-logs-permission-arn "arn:aws:iam::$ACCOUNT_ID:role/lab-flowlogs" \
  --query 'FlowLogIds[0]' --output text)
cat >> ~/lab.env <<EOF
export V=$V SUBA=$SUBA SUBB=$SUBB IGW=$IGW RT=$RT S3EP=$S3EP VSG=$VSG
export VIID=$VIID FL=$FL
EOF
aws ec2 wait instance-status-ok --instance-ids "$VIID"
VIP=$(aws ec2 describe-instances --instance-ids "$VIID" \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
curl -s "http://$VIP/"                        # works

MYIP=$(curl -s https://checkip.amazonaws.com)
NACL=$(aws ec2 create-network-acl --vpc-id "$V" --query NetworkAcl.NetworkAclId \
  --output text)
aws ec2 create-network-acl-entry --network-acl-id "$NACL" --ingress --rule-number 90 \
  --protocol tcp --port-range From=80,To=80 --cidr-block "$MYIP/32" --rule-action deny
aws ec2 create-network-acl-entry --network-acl-id "$NACL" --ingress --rule-number 100 \
  --protocol -1 --cidr-block 0.0.0.0/0 --rule-action allow
aws ec2 create-network-acl-entry --network-acl-id "$NACL" --egress --rule-number 100 \
  --protocol -1 --cidr-block 0.0.0.0/0 --rule-action allow
ASSOC=$(aws ec2 describe-network-acls \
  --filters Name=association.subnet-id,Values="$SUBA" \
  --query "NetworkAcls[0].Associations[?SubnetId=='$SUBA'].NetworkAclAssociationId" \
  --output text)
aws ec2 replace-network-acl-association --association-id "$ASSOC" --network-acl-id "$NACL"
echo "export NACL=$NACL VIP=$VIP" >> ~/lab.env
```

Verify:

```
curl -s -m 5 "http://$VIP/" || echo "blocked by NACL"
sleep 120
QID=$(aws logs start-query --log-group-name /lab/vpc/flow \
  --start-time $(date -d '-15 min' +%s) --end-time $(date +%s) \
  --query-string 'fields @timestamp, srcAddr, dstPort, action
    | filter action = "REJECT" and dstPort = 80 | limit 10' \
  --query queryId --output text)
sleep 10
aws logs get-query-results --query-id "$QID" --query 'results[][].value' | head -20
aws ec2 describe-route-tables --route-table-ids "$RT" \
  --query 'RouteTables[0].Routes[].[DestinationCidrBlock,DestinationPrefixListId,
    GatewayId]'
```

You should see your IP with `REJECT` on port 80, and a route to the S3 prefix list (`pl-...`) via the `vpce-...` endpoint.

Cleanup:

```
source ~/lab.env
aws ec2 terminate-instances --instance-ids "$VIID"
aws ec2 wait instance-terminated --instance-ids "$VIID"
aws ec2 delete-flow-logs --flow-log-ids "$FL"
aws logs delete-log-group --log-group-name /lab/vpc/flow
```

```
aws ec2 delete-security-group --group-id "$VSG"
aws ec2 delete-vpc-endpoints --vpc-endpoint-ids "$S3EP"
aws ec2 delete-subnet --subnet-id "$SUBA"
aws ec2 delete-subnet --subnet-id "$SUBB"
aws ec2 delete-network-acl --network-acl-id "$NACL"
aws ec2 delete-route-table --route-table-id "$RT"
aws ec2 detach-internet-gateway --internet-gateway-id "$IGW" --vpc-id "$V"
aws ec2 delete-internet-gateway --internet-gateway-id "$IGW"
aws ec2 delete-vpc --vpc-id "$V"
aws iam delete-role-policy --role-name lab-flowlogs --policy-name write-logs
aws iam delete-role --role-name lab-flowlogs
```

Task 17: CloudFront with a private S3 origin, caching and invalidation

Task: Serve `index.html` from a private S3 bucket through CloudFront using Origin Access Control, HTTPS redirect and the managed CachingOptimized policy. Show a cache hit, deploy a new version, and fix the stale content with an invalidation.

Solution:

```
source ~/lab.env
SB="lab-site-$ACCOUNT_ID"
aws s3api create-bucket --bucket "$SB"
echo "<h1>v1</h1>" > index.html
aws s3 cp index.html "s3://$SB/index.html" --content-type text/html
OACC=Name=lab-oac,SigningProtocol=sigv4,SigningBehavior=always
OACC=$OACC,OriginAccessControlOriginType=s3
OAC=$(aws cloudfront create-origin-access-control \
  --origin-access-control-config "$OACC" \
  --query OriginAccessControl.Id --output text)
cat > dist.json <<EOF
{"CallerReference":"lab-$(date +%s)","Comment":"lab","Enabled":true,
 "DefaultRootObject":"index.html",
 "Origins":{"Quantity":1,"Items":[{"Id":"s3site",
   "DomainName":"$SB.s3.$AWS_REGION.amazonaws.com",
   "S3OriginConfig":{"OriginAccessIdentity":""},"OriginAccessControlId":"$OAC"}]},
 "DefaultCacheBehavior":{"TargetOriginId":"s3site",
   "ViewerProtocolPolicy":"redirect-to-https",
   "CachePolicyId":"658327ea-f89d-4fab-a63d-7e88639e58f6"}}
EOF
DIST=$(aws cloudfront create-distribution --distribution-config file://dist.json \
  --query Distribution.Id --output text)
cat > site-policy.json <<EOF
{"Version":"2012-10-17","Statement":[{"Sid":"CloudFrontOnly","Effect":"Allow",
 "Principal":{"Service":"cloudfront.amazonaws.com"},"Action":"s3:GetObject",
 "Resource":"arn:aws:s3:::$SB/*",
 "Condition":{"StringEquals":
   {"AWS:SourceArn":"arn:aws:cloudfront::$ACCOUNT_ID:distribution/$DIST"}}}]}
EOF
aws s3api put-bucket-policy --bucket "$SB" --policy file://site-policy.json
aws cloudfront wait distribution-deployed --id "$DIST"
echo "export SB=$SB OAC=$OAC DIST=$DIST" >> ~/lab.env
```

Verify:

```
CF=$(aws cloudfront get-distribution --id "$DIST" \
  --query Distribution.DomainName --output text)
curl -sI "https://$CF/" | grep -i x-cache      # Miss from cloudfront
curl -sI "https://$CF/" | grep -i x-cache      # Hit from cloudfront
curl -s -o /dev/null -w "%{http_code}\n" \
  "https://$SB.s3.amazonaws.com/index.html"     # 403: bucket not public
echo "<h1>v2</h1>" > index.html
aws s3 cp index.html "s3://$SB/index.html" --content-type text/html
curl -s "https://$CF/"                          # still v1 (cached)
INV=$(aws cloudfront create-invalidation --distribution-id "$DIST" \
  --paths "/index.html" "/" --query Invalidation.Id --output text)
aws cloudfront wait invalidation-completed --distribution-id "$DIST" --id "$INV"
curl -s "https://$CF/"                          # v2
```

Cleanup: A distribution must be disabled and fully deployed before it can be deleted.

```
aws cloudfront get-distribution-config --id "$DIST" > dc.json
ETAG=$(jq -r .ETag dc.json)
jq '.DistributionConfig.Enabled=false | .DistributionConfig' dc.json > dc2.json
```

```
aws cloudfront update-distribution --id "$DIST" --if-match "$ETAG" \
  --distribution-config file://dc2.json
aws cloudfront wait distribution-deployed --id "$DIST"
ETAG=$(aws cloudfront get-distribution --id "$DIST" --query ETag --output text)
aws cloudfront delete-distribution --id "$DIST" --if-match "$ETAG"
OETAG=$(aws cloudfront get-origin-access-control --id "$OAC" --query ETag --output text)
aws cloudfront delete-origin-access-control --id "$OAC" --if-match "$OETAG"
aws s3 rm "s3://$SB" --recursive && aws s3api delete-bucket --bucket "$SB"
```

Task 18: KMS customer managed key for EBS, with rotation

Task: Create a customer managed KMS key with alias `alias/lab-ebs` and annual rotation. Create an EBS volume encrypted with it, encrypt and decrypt a small file, and read the key policy to see who can use the key.

Solution:

```
source ~/lab.env
KEY=$(aws kms create-key --description "lab ebs key" --query KeyMetadata.KeyId \
  --output text)
aws kms create-alias --alias-name alias/lab-ebs --target-key-id "$KEY"
aws kms enable-key-rotation --key-id "$KEY"
EVOL=$(aws ec2 create-volume --availability-zone us-east-1a --size 1 \
  --volume-type gp3 --encrypted --kms-key-id alias/lab-ebs \
  --query VolumeId --output text)
echo -n "hello-kms" > plain.txt
aws kms encrypt --key-id alias/lab-ebs --plaintext fileb://plain.txt \
  --query CiphertextBlob --output text | base64 -d > secret.bin
echo "export KEY=$KEY EVOL=$EVOL" >> ~/lab.env
```

Verify:

```
aws ec2 describe-volumes --volume-ids "$EVOL" --query 'Volumes[0].[Encrypted,KmsKeyId]'
aws kms get-key-rotation-status --key-id "$KEY"             # KeyRotationEnabled: true
aws kms decrypt --ciphertext-blob fileb://secret.bin --query Plaintext \
  --output text | base64 -d; echo                           # hello-kms
aws kms get-key-policy --key-id "$KEY" --policy-name default --output text | head -20
```

The default key policy trusts the account root, which delegates access control to IAM policies. Anyone launching an instance from an AMI encrypted with this key needs `kms:Decrypt`, `kms:CreateGrant` and related permissions on it.

Cleanup:

```
aws ec2 wait volume-available --volume-ids "$EVOL"
aws ec2 delete-volume --volume-id "$EVOL"
aws kms delete-alias --alias-name alias/lab-ebs
aws kms schedule-key-deletion --key-id "$KEY" --pending-window-in-days 7
```

Task 19: CloudTrail, IAM Access Analyzer and an AWS Config rule

Task: Find who opened port 22 in Task 15 using CloudTrail event history. Create an account-level Access Analyzer and trigger a finding with a role trusted by an external account. Record only S3 buckets in AWS Config, add the managed rule that checks bucket versioning, and read the compliance result.

Solution:

```
source ~/lab.env
aws cloudtrail lookup-events --max-results 5 \
  --lookup-attributes \
  AttributeKey=EventName,AttributeValue=AuthorizeSecurityGroupIngress \
  --query 'Events[].[EventTime,Username,EventName]' --output table

aws accessanalyzer create-analyzer --analyzer-name lab-analyzer --type ACCOUNT
cat > ext-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Principal":{"AWS":"arn:aws:iam::111122223333:root"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name lab-external-trust \
  --assume-role-policy-document file://ext-trust.json

CB="lab-config-$ACCOUNT_ID"
aws s3api create-bucket --bucket "$CB"
cat > cfg-bucket.json <<EOF
{"Version":"2012-10-17","Statement":[
```

```
 {"Sid":"AclCheck","Effect":"Allow","Principal":{"Service":"config.amazonaws.com"},
  "Action":["s3:GetBucketAcl","s3:ListBucket"],"Resource":"arn:aws:s3:::$CB",
  "Condition":{"StringEquals":{"AWS:SourceAccount":"$ACCOUNT_ID"}}},
 {"Sid":"Write","Effect":"Allow","Principal":{"Service":"config.amazonaws.com"},
  "Action":"s3:PutObject","Resource":"arn:aws:s3:::$CB/AWSLogs/$ACCOUNT_ID/Config/*",
  "Condition":{"StringEquals":{"s3:x-amz-acl":"bucket-owner-full-control",
   "AWS:SourceAccount":"$ACCOUNT_ID"}}}]}
EOF
aws s3api put-bucket-policy --bucket "$CB" --policy file://cfg-bucket.json
aws iam create-service-linked-role --aws-service-name config.amazonaws.com 2>/dev/null
SLR="arn:aws:iam::$ACCOUNT_ID:role/aws-service-role/config.amazonaws.com"
SLR="$SLR/AWSServiceRoleForConfig"
aws configservice put-configuration-recorder --configuration-recorder \
  "name=default,roleARN=$SLR" --recording-group \
  "allSupported=false,includeGlobalResourceTypes=false,resourceTypes=AWS::S3::Bucket"
aws configservice put-delivery-channel --delivery-channel "name=default,s3BucketName=$CB"
aws configservice start-configuration-recorder --configuration-recorder-name default
cat > rule.json <<'EOF'
{"ConfigRuleName":"s3-versioning-enabled",
 "Source":{"Owner":"AWS","SourceIdentifier":"S3_BUCKET_VERSIONING_ENABLED"},
 "Scope":{"ComplianceResourceTypes":["AWS::S3::Bucket"]}}
EOF
aws configservice put-config-rule --config-rule file://rule.json
echo "export CB=$CB" >> ~/lab.env
```

Verify:

```
aws accessanalyzer list-findings --analyzer-arn "$(aws accessanalyzer get-analyzer \
  --analyzer-name lab-analyzer --query analyzer.arn --output text)" \
  --query 'findings[].[resourceType,resource,status]'
aws configservice describe-configuration-recorder-status \
  --query 'ConfigurationRecordersStatus[0].[recording,lastStatus]'
sleep 120
aws configservice get-compliance-details-by-config-rule \
  --config-rule-name s3-versioning-enabled \
  --query 'EvaluationResults[].[
    EvaluationResultIdentifier.EvaluationResultQualifier.ResourceId,ComplianceType]'
```

The config bucket shows NON_COMPLIANT (no versioning). In the console, Config can attach the `AWS-ConfigureS3BucketVersioning` SSM runbook as an automatic remediation. Trusted Advisor's free core security checks are in the console under Trusted Advisor; read them once.

Cleanup:

```
aws configservice delete-config-rule --config-rule-name s3-versioning-enabled
aws configservice stop-configuration-recorder --configuration-recorder-name default
aws configservice delete-delivery-channel --delivery-channel-name default
aws configservice delete-configuration-recorder --configuration-recorder-name default
aws s3 rm "s3://$CB" --recursive && aws s3api delete-bucket --bucket "$CB"
aws iam delete-role --role-name lab-external-trust
aws accessanalyzer delete-analyzer --analyzer-name lab-analyzer
```

Task 20: RDS backups and point-in-time restore

Task: Create a single-AZ MySQL `db.t4g.micro` instance, not publicly accessible, with 1-day automated backups and the master password managed in Secrets Manager. Take a manual snapshot, then restore the database to the latest restorable time as a new instance.

Solution:

```
source ~/lab.env
aws rds create-db-instance --db-instance-identifier lab-db --engine mysql \
  --db-instance-class db.t4g.micro --allocated-storage 20 --storage-type gp3 \
  --master-username admin --manage-master-user-password \
  --backup-retention-period 1 --no-multi-az --no-publicly-accessible \
  --no-deletion-protection
aws rds wait db-instance-available --db-instance-identifier lab-db   # 10-15 min
aws rds create-db-snapshot --db-instance-identifier lab-db \
  --db-snapshot-identifier lab-db-manual-1
aws rds wait db-snapshot-available --db-snapshot-identifier lab-db-manual-1
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier lab-db --target-db-instance-identifier lab-db-pitr \
  --use-latest-restorable-time --db-instance-class db.t4g.micro --no-multi-az \
```

```
  --no-publicly-accessible
aws rds wait db-instance-available --db-instance-identifier lab-db-pitr
```

A restore always creates a new instance with a new endpoint; the application's connection string (or a Route 53 CNAME in front of it) must be switched.

Verify:

```
aws rds describe-db-instances --db-instance-identifier lab-db \
  --query 'DBInstances[0].[DBInstanceStatus,LatestRestorableTime,
    MasterUserSecret.SecretArn]'
aws rds describe-db-instances --db-instance-identifier lab-db-pitr \
  --query 'DBInstances[0].[DBInstanceStatus,Endpoint.Address]'
aws cloudwatch get-metric-statistics --namespace AWS/RDS --metric-name FreeStorageSpace \
  --dimensions Name=DBInstanceIdentifier,Value=lab-db --statistics Minimum \
  --start-time "$(date -u -d '-30 min' +%FT%TZ)" --end-time "$(date -u +%FT%TZ)" \
  --period 300
```

Cleanup: Same day.

```
for db in lab-db-pitr lab-db; do
  aws rds delete-db-instance --db-instance-identifier "$db" \
    --skip-final-snapshot --delete-automated-backups
done
aws rds delete-db-snapshot --db-snapshot-identifier lab-db-manual-1
aws rds wait db-instance-deleted --db-instance-identifier lab-db
```

Finally, when you are done for the day: `~/teardown-core.sh`, then check Cost Explorer.

Practice questions

All questions are original. Where it says "Choose TWO", exactly two options are correct.

1. A Node.js service on EC2 is slow. CPUUtilization is 20%, but you suspect memory pressure. The AWS/EC2 namespace has no memory metric. What is the most operationally efficient fix? A) Enable detailed monitoring. B) Install the CloudWatch agent and collect `mem_used_percent`. C) Write a cron job that calls `put-metric-data` with `free -m` output. D) Enable VPC flow logs.

2. You must alarm when Nginx returns more than 20 HTTP 5xx responses in 5 minutes. Access logs already flow to CloudWatch Logs. What should you create? A) A CloudWatch Logs Insights saved query. B) A metric filter on the log group, then an alarm on the new metric. C) A subscription filter to Lambda that sends email. D) An EventBridge rule on the log group.

3. An alarm on a custom error-count metric keeps showing INSUFFICIENT_DATA overnight because the metric is only published when errors occur. The team wants it to show OK. What change is best? A) Increase the evaluation periods. B) Set treat missing data to notBreaching (or give the metric filter a default value of 0). C) Switch the statistic to Maximum. D) Use a composite alarm.

4. On-call is paged every time CPU spikes during nightly builds, which is harmless unless latency also rises. How do you page only when both happen? A) Raise the CPU threshold to 95%. B) Create a composite alarm with `ALARM(cpu) AND ALARM(latency)` and move notifications to it. C) Use anomaly detection on CPU. D) Disable the CPU alarm at night with a Lambda function.

5. An EBS-backed EC2 instance must recover automatically from underlying host hardware failure while keeping its instance ID, private IP and Elastic IP. What do you configure? A) An Auto Scaling group of size 1. B) A CloudWatch alarm on `StatusCheckFailed_System` with the EC2 recover action. C) A CloudWatch alarm on `StatusCheckFailed_Instance` with the reboot action. D) A Route 53 health check.

6. You need to run `pm2 restart app` on 40 instances tagged `Role=api`, with no SSH, and keep the output for audit. What should you use? A) EC2 Instance Connect. B) Systems Manager Run Command targeting the tag, with output sent to S3 or CloudWatch Logs. C) A user data script. D) CloudFormation cfn-hup.

7. Security wants port 22 closed on all servers, with an interactive shell still available and sessions logged. Which TWO are required for Systems Manager Session Manager? (Choose TWO.) A) An instance profile with `AmazonSSMManagedInstanceCore`. B) An inbound security group rule on port 443. C) A running SSM Agent that can reach the Systems Manager endpoints. D) A key pair. E) An Elastic IP.

8. Instances in private subnets with no NAT gateway and no internet access never appear in Fleet Manager. The instance profile is correct. What fixes it? A) Add an S3 gateway endpoint only. B) Create interface VPC endpoints for `ssm`, `ssmmessages` and `ec2messages` with private DNS enabled. C) Attach an Elastic IP to each instance. D) Open inbound port 443 from 0.0.0.0/0.

9. Behind an ALB, some Auto Scaling instances return HTTP 500 for every request because the app process died, but Auto Scaling never replaces them. What change is needed? A) Increase desired capacity. B) Set the Auto Scaling group health check type to ELB. C) Enable cross-zone load balancing. D) Enable deletion protection on the ALB.

10. New Auto Scaling instances are terminated as unhealthy before `npm install` and PM2 start finish in user data (about 4 minutes), creating a replace loop. What is the simplest fix? A) Change the health check path to `/`. B) Increase the health check grace period beyond the boot time, or bake dependencies into the AMI. C) Switch to a Network Load Balancer. D) Use a larger instance type.

11. Traffic to an API ramps up and down unpredictably and you want average CPU across the group to stay near 50% with the least configuration. Which scaling policy? A) Simple scaling with two alarms. B) Step scaling. C) Target tracking on `ASGAverageCPUUtilization` = 50. D) Scheduled actions.

12. A reporting app has a guaranteed load spike every weekday at 09:00 and instances take 5 minutes to become healthy. How do you avoid slow responses at 09:00? A) Target tracking only. B) A scheduled action that raises minimum capacity at 08:45 (predictive scaling is another valid option). C) Increase the ALB idle timeout. D) Enable detailed monitoring.

13. ALB access logs show `elb_status_code` 504 with `target_status_code` "-" for long report requests that take 90 seconds. What is the likely cause and fix? A) Targets are unhealthy; fix the health check. B) The request exceeded the ALB idle timeout (60 seconds by default); raise it or make the job asynchronous. C) The security group blocks the ALB. D) The target group uses the wrong port.

14. After a deploy, the ALB returns HTTP 503 for every request. What is the most likely cause? A) The listener certificate expired. B) The target group has no registered or healthy targets. C) The ALB is out of capacity. D) WAF blocked the requests.

15. A Route 53 failover setup never fails over, even though the primary server is down. What is the most likely misconfiguration? A) The TTL is 60 seconds. B) The primary record has no health check associated (or, for an alias, Evaluate Target Health is off). C) The secondary record has a health check. D) The hosted zone is public.

16. A Route 53 health check against an EC2 web server always reports unhealthy, but the site loads fine from your laptop. What should you check first? A) The instance's IAM role. B) Whether the security group and NACL allow the Route 53 health checker IP ranges on the health-check port. C) Whether the EC2 instance has detailed monitoring. D) Whether the hosted zone has DNSSEC.

17. You want to send 10% of users to a new application version by DNS and increase gradually. Which routing policy? A) Latency. B) Failover. C) Weighted. D) Geolocation.

18. A versioned S3 bucket that stores daily log uploads keeps growing even though an expiration rule deletes objects after 90 days. Which TWO lifecycle actions fix it? (Choose TWO.) A) NoncurrentVersionExpiration. B) Transition to S3 One Zone-IA. C) AbortIncompleteMultipartUpload. D) Enable Requester Pays. E) Enable Transfer Acceleration.

19. A developer deleted `config/prod.json` from a versioned bucket. How do you restore it with the least effort? A) Restore from AWS Backup. B) Delete the delete marker for that key. C) Copy the object from CloudTrail. D) Re-enable versioning.

20. Users in Asia upload 2 GB video files to a bucket in us-east-1 and uploads are slow and sometimes fail. Which TWO changes help most? (Choose TWO.) A) Multipart upload. B) S3 Transfer Acceleration. C) S3 Object Lock. D) Glacier Instant Retrieval. E) Cross-Region Replication to us-west-2.

21. A bad migration corrupted an RDS MySQL table at 14:32. Automated backups are enabled with 7-day retention. The business needs data as of 14:30. What do you do? A) Restore the latest automated snapshot in place. B) Point-in-time restore to 14:30 into a new DB instance, then repoint the application. C) Promote a read

replica. D) Reboot with failover.

22. What does RDS Multi-AZ (one standby) primarily provide compared with a read replica? A) Read scaling. B) Automatic failover to a synchronous standby in another AZ. C) Cross-Region disaster recovery. D) Lower storage cost.

23. Compliance requires daily backups of all EC2 instances and EBS volumes tagged `backup=daily`, kept 35 days, with a copy in a second Region. What is the least operational effort? A) A Lambda function on a cron schedule calling `create-snapshot`. B) An AWS Backup plan with a tag-based selection, a 35-day lifecycle and a cross-Region copy action. C) Data Lifecycle Manager plus manual copies. D) S3 replication.

24. A gp2 volume on a database host hits its IOPS ceiling during peaks. You want more IOPS without buying a bigger disk and without downtime. What do you do? A) Detach and restore from snapshot as io2. B) Use Elastic Volumes to modify it to gp3 and set IOPS independently of size. C) Add an instance store. D) Enable EBS encryption.

25. After increasing an EBS root volume from 8 GiB to 20 GiB, `df -h /` still shows 8 GiB on the XFS root. Which commands complete the change? A) `mkfs.xfs` then reboot. B) `growpart /dev/nvme0n1 1` then `xfs_growfs /`. C) `resize2fs /dev/nvme0n1`. D) Nothing; wait for the volume to finish optimizing.

26. A CloudFormation stack contains the production RDS instance. The team wants to delete the stack later without losing the data. What should be in the template? A) A stack policy. B) `DeletionPolicy: Snapshot` (or `Retain`) on the DB resource. C) Termination protection on the EC2 instances. D) An Outputs section.

27. A stack is in `UPDATE_ROLLBACK_FAILED` because a resource was changed outside CloudFormation. What is the supported way forward? A) Delete the stack immediately. B) Fix the underlying issue, or run `continue-update-rollback` and skip the problem resources. C) Edit the template in S3. D) Create a change set.

28. You must deploy the same baseline IAM roles and Config rules to 12 accounts in 3 Regions and keep new accounts in sync automatically. What should you use? A) Nested stacks. B) CloudFormation StackSets with service-managed permissions through AWS Organizations. C) Copy the template manually. D) AWS RAM.

29. After deploying a new `app.js` to S3 behind CloudFront, users still receive the old file for hours. Which approach prevents this in future deploys with the least ongoing cost? A) Invalidate `/*` on every deploy. B) Use versioned (hashed) file names so each deploy changes the URL. C) Set the TTL to 0 for all content. D) Disable CloudFront.

30. A bucket serves a site through CloudFront, but people also read objects directly with S3 URLs. How do you force all access through CloudFront? A) Enable S3 static website hosting. B) Use Origin Access Control and a bucket policy that allows only the distribution's ARN, with Block Public Access on. C) Add a WAF rule to S3. D) Use presigned URLs.

31. Flow logs show inbound HTTP accepted, but response packets to clients are rejected at the subnet. The subnet uses a custom network ACL that allows inbound 80 and outbound 80 only. What is wrong? A) The security group is missing an outbound rule. B) NACLs are stateless; outbound ephemeral ports (1024-65535) must be allowed. C) The route table needs a NAT gateway. D) Flow logs are misconfigured.

32. You need to block one abusive IP address from reaching every instance in a subnet immediately. Which control can do it? A) A security group deny rule. B) A network ACL deny rule with a lower rule number than the allow rules. C) An IAM policy. D) A route to a blackhole.

33. Instances in a private subnet must download packages from the internet but must not be reachable from it. The private route table currently has only the local route. What do you add? A) An internet gateway route in the private route table. B) A NAT gateway in a public subnet and a `0.0.0.0/0` route to it from the private route table. C) An egress-only internet gateway for IPv4. D) A VPC peering connection.

34. Yesterday someone deleted a security group rule and the app went down. How do you find out who, with no prior setup? A) VPC flow logs. B) CloudTrail event history (the last 90 days of management events are available by default). C) AWS Config rules. D) CloudWatch metrics.

35. The organization must prevent anyone, including account admins, from creating resources outside `eu-west-1` and `us-east-1` in member accounts. What do you use? A) IAM permission boundaries in each account. B) A

service control policy that denies actions when `aws:RequestedRegion` is not one of the two (with exceptions for global services). C) AWS Config rule only. D) A billing alarm.

36. You must continuously detect S3 buckets without versioning and fix them automatically. What is the best approach? A) A weekly Trusted Advisor report. B) The AWS Config managed rule for S3 versioning with an automatic remediation using an SSM Automation runbook. C) GuardDuty. D) Macie.

37. An application's database password must rotate automatically every 30 days with no code change beyond fetching the secret. Where should it be stored? A) Parameter Store String parameter. B) AWS Secrets Manager with rotation enabled. C) An environment variable in the PM2 ecosystem file. D) S3 with SSE-KMS.

38. A team shares an AMI whose snapshots are encrypted with a customer managed KMS key. Another account can see the AMI but launches fail. What is missing? A) The AMI must be made public. B) The key policy must grant the other account use of the key (for example `kms:Decrypt` and `kms:CreateGrant`), and that account's IAM must allow it. C) The instance type is wrong. D) EBS encryption by default must be off.

39. You request a public ACM certificate for a CloudFront distribution, but it does not appear in the distribution's certificate list. Why? A) ACM certificates cannot be used with CloudFront. B) The certificate must be requested in us-east-1 (N. Virginia) for CloudFront. C) DNS validation is not supported. D) The certificate needs to be imported.

40. An EventBridge rule matches the right events (its metrics show matched events) but its Lambda target is never invoked. What is the most likely cause? A) The event bus is full. B) The Lambda function lacks a resource-based policy allowing `events.amazonaws.com` to invoke it (or it is failing and invocations appear as FailedInvocations). C) EventBridge cannot target Lambda. D) The rule needs a schedule expression.

Answers

1. B. Memory and disk usage are OS-level metrics; only the CloudWatch agent (or a custom publisher) collects them. Detailed monitoring just increases frequency of the same hypervisor metrics.
2. B. Metric filters turn matching log lines into a metric; alarms work on metrics, not on log queries.
3. B. Sparse metrics need a missing-data policy (notBreaching) or a filter default value of 0, so absence of errors reads as OK.
4. B. Composite alarms combine alarm states with AND/OR/NOT and are the standard way to cut noise.
5. B. The EC2 recover action for system status check failures migrates the instance to healthy hardware and keeps its ID, IPs and EBS volumes.
6. B. Run Command targets by tag, needs no SSH, and records output in S3 or CloudWatch Logs with CloudTrail auditing the call.
7. A and C. Session Manager needs the agent, the instance profile and outbound HTTPS to the endpoints; no inbound ports or key pairs.
8. B. Without internet access, the SSM Agent reaches Systems Manager through interface endpoints for ssm, ssmmessages and ec2messages.
9. B. With EC2 health checks only, Auto Scaling sees a running VM as healthy; the ELB health check type makes it act on failed target health.
10. B. The grace period stops health-check terminations during boot; baking the app into the AMI also shortens boot time.
11. C. Target tracking creates and manages the alarms and scales to hold the metric at the target value.
12. B. Known, recurring spikes are handled by scheduled actions (or predictive scaling) so capacity is warm before the load arrives.
13. B. An ALB 504 with no target status code means the target did not respond before the idle timeout; raise it or move long work to a queue.

14. B. An ALB returns 503 when the target group has no registered targets, or all targets are unhealthy and fail-open does not apply.
15. B. Failover depends on the primary record's health: a health check on non-alias records, or Evaluate Target Health on alias records.
16. B. Route 53 health checkers come from published AWS IP ranges; SGs or NACLs that only allow your IP make the check fail.
17. C. Weighted routing splits DNS responses by relative weights, which is the DNS canary pattern.
18. A and C. Expiration only adds delete markers in a versioned bucket; old versions need NoncurrentVersionExpiration, and failed multipart parts need AbortIncompleteMultipartUpload.
19. B. Deleting in a versioned bucket adds a delete marker; removing the marker makes the previous version current again.
20. A and B. Multipart uploads parallelize and retry parts; Transfer Acceleration routes uploads over the AWS edge network.
21. B. Point-in-time restore always creates a new instance at any second in the retention window; you then switch the endpoint.
22. B. Multi-AZ standby is for availability with automatic failover; read replicas are for read scaling (and can be promoted manually).
23. B. AWS Backup centralizes schedules, retention, tag-based selection and cross-Region copies without custom code.
24. B. Elastic Volumes changes type and performance online; gp3 decouples IOPS and throughput from size.
25. B. The partition must be grown first (`growpart`), then the filesystem (`xfs_growfs` for XFS; `resize2fs` is for ext4).
26. B. DeletionPolicy controls what happens to a resource when the stack or resource is deleted; Snapshot or Retain keeps the data.
27. B. `continue-update-rollback`, optionally with resources to skip, is the documented recovery from UPDATE_ROLLBACK_FAILED.
28. B. StackSets with service-managed permissions deploy across accounts and Regions and can auto-deploy to new accounts in an OU.
29. B. Versioned object names make each deploy a cache miss by design; invalidations work but cost money beyond the free allowance and are slower.
30. B. OAC signs origin requests; the bucket policy allows only that distribution, and Block Public Access removes direct access.
31. B. NACLs are stateless, so return traffic to client ephemeral ports needs an explicit outbound allow.
32. B. Only NACLs support explicit deny; rules are evaluated in number order, so the deny needs a lower number than the allow.
33. B. Private subnets reach the internet outbound through a NAT gateway in a public subnet; egress-only IGWs are for IPv6 only.
34. B. CloudTrail event history keeps 90 days of management events in every account at no charge, including who called RevokeSecurityGroupIngress.
35. B. SCPs set the maximum permissions for member accounts, including admins; `aws:RequestedRegion` restricts Regions.
36. B. Config evaluates continuously and can trigger an SSM Automation runbook to remediate non-compliant resources.
37. B. Secrets Manager provides managed rotation; Parameter Store SecureString has no built-in rotation.

38. B. Encrypted AMI sharing requires sharing the KMS key through its key policy plus IAM permissions in the target account.
39. B. CloudFront only uses ACM certificates from us-east-1, whatever Region the origin is in.
40. B. For Lambda targets, EventBridge needs a resource-based permission on the function; check FailedInvocations to confirm.

Exam-day tips

Delivery rules (from the AWS certification policy pages):

- Book through your AWS Certification account (Pearson VUE). You can reschedule or cancel up to 24 hours before the appointment, and reschedule at most twice. Inside 24 hours or no-show, the fee is lost.
- If English is not your first language, request the ESL accommodation (30 extra minutes) once in your AWS Certification account before you register. It then applies to every future exam. Do this before booking.
- ID: online proctoring needs one valid, unexpired government-issued primary ID with photo and signature, in Roman characters. Test centers need two IDs (two primary, or one primary and one secondary). The first and last name must match your certification account exactly.
- Test center: arrive 15-30 minutes early.
- Online: you can launch the exam up to 30 minutes early; more than 15 minutes late and you forfeit the fee. Run the Pearson OnVUE system test on the same computer and network beforehand, clear the desk and room, and use a single monitor.
- Online: you may not leave the camera view or stand up for any reason, including restroom breaks. Plan for 130 minutes (160 with ESL) plus check-in.
- No reference material: no notes, docs, phone or console. This is a pure recall and judgment exam.

Time management:

- 65 questions in 130 minutes is 2 minutes each. First pass: answer everything, flag anything over 90 seconds. Second pass: flagged items only.
- Never leave a blank. There is no penalty for guessing, and 15 unknown questions are unscored anyway.
- Multiple-response questions state how many to pick; read that number twice.

Common traps:

- "Most operationally efficient" or "least effort" means the managed service (AWS Backup, target tracking, Run Command, Config remediation) beats a custom Lambda or cron job.
- Security group vs NACL: SGs are stateful and allow-only; NACLs are stateless, ordered, and can deny.
- Detailed monitoring does not add memory or disk metrics; only the CloudWatch agent does.
- Region-bound facts: billing metrics, Route 53 health-check metrics and CloudFront ACM certificates are in us-east-1.
- Versioned buckets: expiration creates delete markers; you need noncurrent version rules to actually free space.
- Restores (RDS PITR, snapshots) create new resources with new endpoints or IDs.
- Auto Scaling ignores app failures unless the health check type is ELB.
- Parameter Store vs Secrets Manager: rotation points to Secrets Manager.
- NAT gateway (IPv4, in a public subnet) vs egress-only internet gateway (IPv6).
- Retake: if you fail, wait 14 days. After passing, you cannot retake the same exam version for 2 years.

Readiness checklist

- [ ] I created the lab account myself, root has MFA, and daily work uses a non-root admin identity
- [ ] Task 1 budget exists with credits excluded, and I get the alert emails
- [ ] I can launch an IMDSv2-only instance with user data that runs Nginx and PM2 and survives reboot
- [ ] I can install and configure the CloudWatch agent from Parameter Store without SSH
- [ ] I can write a metric filter for Nginx 5xx and test it with `test-metric-filter`
- [ ] I can build metric, composite and recover alarms and explain missing-data handling
- [ ] I can use Session Manager, Run Command, Patch Manager, maintenance windows and Automation
- [ ] I built an EventBridge rule that remediates an alarm automatically
- [ ] I can grow an EBS volume online and extend the filesystem
- [ ] I can build a launch template, ALB, target group and Auto Scaling group from the CLI
- [ ] I can explain target tracking vs step vs scheduled vs predictive scaling
- [ ] I can write S3 lifecycle rules for versioned buckets and recover a deleted object
- [ ] I wired S3 event notifications to Lambda, including the invoke permission
- [ ] I can create Route 53 health checks, alarm on them, and describe every routing policy
- [ ] I ran an AWS Backup job and a successful restore, and know the DR tiers (backup-restore to active/active)
- [ ] I can deploy, change-set, drift-detect and delete a CloudFormation stack; I know StackSets, RAM and CDK basics
- [ ] I built a VPC with IGW, routes, NACLs and an S3 gateway endpoint and read flow logs with Logs Insights
- [ ] I can put CloudFront in front of private S3 with OAC and explain invalidation vs versioned names
- [ ] I can create a KMS key, use it for EBS, and explain key policies and cross-account sharing
- [ ] I can find an actor in CloudTrail and explain Access Analyzer, Config rules and conformance packs
- [ ] I can explain Organizations, SCPs, IAM Identity Center, GuardDuty, Security Hub, Inspector, Macie
- [ ] I restored RDS to a point in time and know Multi-AZ vs read replicas vs RDS Proxy
- [ ] I can describe in a sentence each of: Bedrock, Kiro, DevOps Agent, Security Agent, Health Dashboard
- [ ] I scored 80%+ on the official practice question set and reviewed every miss
- [ ] ESL accommodation requested (if wanted), ID names match my account, OnVUE system test passed
- [ ] All lab resources deleted and Cost Explorer shows nothing running

Sources

- AWS Certified CloudOps Engineer - Associate exam page (name, SOA-C03, format, 65 questions, 130 minutes, 150 USD, delivery, languages, Korean and Chinese retirement after Nov 19, 2026, SysOps holders): https://aws.amazon.com/certification/certified-cloudops-engineer-associate/
- AWS Certified SysOps Administrator - Associate exam page (last exam date Sep 29, 2025; replacement): https://aws.amazon.com/certification/certified-sysops-admin-associate/
- SOA-C03 exam guide (question types, 50 scored and 15 unscored, 720 of 1,000, compensatory scoring, domain weights): https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03.html
- SOA-C03 Domain 1: https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03-domain1.html

- SOA-C03 Domain 2: https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03-domain2.html
- SOA-C03 Domain 3: https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03-domain3.html
- SOA-C03 Domain 4: https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03-domain4.html
- SOA-C03 Domain 5: https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03-domain5.html
- SOA-C02 vs SOA-C03 comparison (SOA-C02 until Sep 29, 2025; SOA-C03 from Sep 30, 2025): https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/sysops-administrator-associate-03-comparison.html
- SOA-C03 revisions (v1.0 Sep 30, 2025; v1.1 June 1, 2026; added services): https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/soa-03-revisions.html
- SOA-C03 in-scope services: https://docs.aws.amazon.com/aws-certification/latest/sysops-administrator-associate-03/soa-03-in-scope-services.html
- SOA-C02 exam guide v2.3 (exam labs removed as of March 28, 2023): https://d1.awsstatic.com/training-and-certification/docs-sysops-associate/AWS-Certified-SysOps-Administrator-Associate_Exam-Guide.pdf
- AWS Certification policies, before testing (reschedule, ESL accommodation): https://aws.amazon.com/certification/policies/before-testing/
- AWS Certification policies, during testing (ID, check-in, no breaks online): https://aws.amazon.com/certification/policies/during-testing/
- AWS Certification FAQs (14-day retake wait, 2-year retake rule after passing): https://aws.amazon.com/certification/faqs/
- AWS Free Tier (Free and Paid plans, up to 200 USD credits, 6 months): https://aws.amazon.com/free/
- AWS Free Tier FAQs (Free plan expiry and restrictions): https://aws.amazon.com/free/free-tier-faqs/
- EC2 Free Tier usage (eligible instance types for accounts after July 15, 2025): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-free-tier-usage.html
- AWS Budgets CostTypes (IncludeCredit defaults to true): https://docs.aws.amazon.com/aws-cost-management/latest/APIReference/API_budgets_CostTypes.html
- Unverified: which Skill Builder exam-prep items are free vs subscription; exact per-service Free plan restrictions; the dollar amount of each Explore AWS credit activity.

Chapter 4: Cisco NetAcad Linux Unhatched & Linux Essentials

This guide prepares you; you take every course exam and certification exam yourself.

As of September 2026. Facts below were checked against netacad.com, credly.com, lpi.org and pearsonvue.com (see Sources). Anything not confirmed on an official page is marked "unverified".

At a glance

Item	Linux Unhatched	Linux Essentials
Provider	Cisco NetAcad, built with NDG	Cisco NetAcad, built with NDG
Cost	Free	Free
Delivery	Self-paced online (or via an academy)	Self-paced online (or via an academy)
Listed length	8 hours	70 hours
Listed level	Beginner	Intermediate (Credly: Beginner)
Structure	1 chapter + 1 final assessment	18 modules, chapter exams, midterm, finals
Labs	Guided in-browser VM activities	18 labs listed on netacad.com
Lab environment	Browser-based Linux VM, nothing to install	Browser-based Linux VM, nothing to install
Languages	EN, DE, ES, FR, IT, PT	English (as listed)
Badge	"Linux Unhatched" course badge	"Linux Essentials" course badge
Badge platform	Credly, issued by Cisco	Credly, issued by Cisco
How badge is earned	Passing score on the comprehensive exam	Passing score on the comprehensive exam
Badge pass mark	70%, first attempt counts (unverified)	70%, first attempt counts (unverified)
Expiry	None stated	None stated

Optional paid next step: LPI Linux Essentials certificate.

Item	LPI Linux Essentials (010-160)
Exam code / version	010-160, objectives version 1.6
Price	USD 120 (US); USD 100 / USD 80 in reduced tiers; EUR 110
Format	40 questions, 60 minutes, multiple choice
Passing score	500 on a 200-800 scale
Delivery	Pearson VUE test center or OnVUE online proctoring
Prerequisites	None; you need an LPI ID before booking
Validity	Lifetime
Retake	Wait 1 week after 1st attempt, 14 days after later ones
Languages	EN, DE, JA, PT-BR (VUE); lpi.org also lists FR, ES
Discount via NetAcad	NDG lists a coupon; self-paced: unverified

Why this fits you: it is the quick win of the guide. You already live in a Linux shell every day, so both courses are mostly a confirmation exercise that produces two shareable Credly badges in 2-3 weeks at zero cost. The LPI certificate is optional; it is a foundational credential, so only pay for it if you want a vendor-neutral line on your CV before tackling LFCS or RHCSA.

Notes on the badge facts:

- Credly lists the earning criterion for both badges as achieving a passing score on a comprehensive exam. The exact 70% threshold and "only the first attempt counts" come from Cisco community answers, not a page I could load; treat both as unverified and behave as if the first attempt is the one that matters.

- The Linux Essentials Credly badge text describes 24 prescriptive labs; netacad.com lists 18 labs; the NDG page lists 13 module labs plus 4 challenge labs. The count depends on course version; the content is the same.
- The Credly Linux Unhatched badge lists these skills: absolute and relative paths, basic regular expressions, command line syntax, common text editors, directories, files, permissions, processes, root and system users.
- The Credly Linux Essentials badge lists: open vs closed source applications, basic hardware concepts, Linux OS components, command line, file management, creating and restoring backups, system security, user/group permissions, basic scripting. Credly shows it as type Professional Certification, level Beginner/Foundational.
- The Unhatched page points you on to Linux Essentials or Linux 1; Linux 1 and Linux 2 are separate, paid NetAcad courses aimed at LPIC-1 and are out of scope here.
- After finishing a course, NetAcad sends a Credly invitation to the email on your NetAcad account; accept it on Credly to make the badge public and shareable on LinkedIn.

Objectives map

The NetAcad Linux Essentials course is built to align with LPI 010-160, so the LPI topics below are the objectives for both. Unhatched covers a subset (mostly 2.x, 3.x and 5.3).

Objective area	What you must be able to do	Your current strength
1.1 Linux evolution and distros	Name major distros, families and where Linux runs	Strong
1.2 Major open source apps	Match apps to jobs; know deb vs rpm packaging	Strong
1.3 Open source licensing	Tell copyleft (GPL) from permissive (MIT, BSD)	Refresh
1.4 ICT skills	Desktop basics, privacy, passwords, basic encryption	Refresh
2.1 Command line basics	Shell syntax, variables, quoting, history	Strong
2.2 Getting help	man sections, `man -k`, `info`, `--help`	Strong
2.3 Directories and listing	Absolute vs relative paths, hidden files, `ls` flags	Strong
2.4 Create, move, delete	`mkdir -p`, `cp -r`, `mv`, `rm -r`, globbing	Strong
3.1 Archiving	`tar` with gzip/bzip2/xz, `zip`/`unzip`	Refresh
3.2 Search and extract	`grep`, basic regex, pipes, redirection, `sort`, `cut`	Strong
3.3 Scripts	Shebang, arguments, `if`, `for`, exit codes	Refresh
4.1 Choosing an OS	Release cycles, LTS vs rolling, Linux vs others	Strong
4.2 Hardware	CPU, RAM, disks, partitions, `/dev`, drivers	Refresh
4.3 Where data is stored	`/etc`, `/var/log`, `/proc`, processes, logs	Strong
4.4 Networking	IP, gateway, DNS, `ip`, `ss`, `/etc/hosts`	Refresh
5.1 Security and user types	root, system and regular users; `sudo`, `su`	Strong
5.2 Users and groups	`useradd`, `groupadd`, `usermod`, `passwd`, `userdel`	Refresh
5.3 Permissions and ownership	`chmod` symbolic and octal, `chown`, `umask`	Strong
5.4 Special files and dirs	Sticky bit, SUID, SGID, hard and symbolic links	Refresh

LPI weights (sum of sub-objectives): Topic 1 = 7, Topic 2 = 9, Topic 3 = 9, Topic 4 = 8, Topic 5 = 7. Scripting (3.3) is the single heaviest objective at weight 4.

Study plan

Budget: 5-8 hours per week around a full-time job. You can skim much of the reading; spend the time on labs, the scripting module and licensing/hardware/networking theory, which is where the exam asks things you do not use daily.

Week	Hours	Course work	Labs and practice	Resources
1	6-7	Unhatched in full + final; LE modules 1-6	Build WSL2 lab; Tasks 1-5	NetAcad Unhatched and LE

Week	Hours	Course work	Labs and practice	Resources
2	7-8	LE modules 7-12; midterm	Tasks 6-11; Questions 1-18	NetAcad LE, learning.lpi.org
3	7-8	LE modules 13-18; comprehensive final	Tasks 12-19; Questions 19-35	NetAcad LE, Credly
4 (optional)	4-6	Re-read LPI 010 objectives	Redo weak tasks; book 010-160	lpi.org, OnVUE

LE = Linux Essentials. Week 1 is where you earn the first badge (Unhatched); week 3 ends with the second (Linux Essentials). Modules 7-12 cover files, archiving, text and scripting; 13-18 cover storage, networking, security, users, permissions and special files.

Rules of thumb:

- Do every chapter quiz before the midterm and finals; they are the only practice for the NetAcad exam question style.
- Take the comprehensive final only once you score 85%+ on this chapter's practice questions without looking.
- LPI publishes free official study material at https://learning.lpi.org; use it for Topic 1 (licensing, communities) and Topic 4 (hardware, networking) which are the theory-heavy parts.

Lab setup

Option A: NetAcad's own environment (zero setup). Both courses embed a browser-based Linux VM in the lab pages. Use it for the graded course labs. Its state may reset between sessions, so do not rely on it for multi-day work.

Option B: WSL2 on your Windows 11 machine (recommended for the labs below). Run in an elevated PowerShell:

```
wsl --install -d Ubuntu-24.04
wsl --list --verbose
wsl --set-default-version 2
```

Reboot if asked, open "Ubuntu 24.04" from the Start menu, create your Linux user, then:

```
sudo apt update && sudo apt -y full-upgrade
sudo apt -y install man-db manpages zip unzip bzip2 xz-utils tree psmisc \
  iproute2 iputils-ping dnsutils file less nano vim
# Ubuntu minimal images strip man pages; restore them:
sudo unminimize        # answer y; skip if the command is not found
mkdir -p ~/lab && cd ~/lab
```

Snapshot WSL before destructive labs (users, permissions under `/srv`):

```
wsl --export Ubuntu-24.04 C:\wsl\ubuntu-le-clean.tar
# restore later if needed:
# wsl --unregister Ubuntu-24.04
# wsl --import Ubuntu-24.04 C:\wsl\Ubuntu C:\wsl\ubuntu-le-clean.tar
```

Option C: any Linux VM (Hyper-V "Quick Create" Ubuntu, VirtualBox, or a spare cloud VM). All labs below use only standard GNU/Linux tools and work on Ubuntu 22.04/24.04, Debian 12 and Rocky/Alma 9 (replace `apt` with `dnf`).

WSL2 caveats: `last` may show no history (no login records), and `dmesg` needs `sudo`. Everything else behaves like a normal Ubuntu system.

Hands-on tasks

All labs assume Ubuntu 24.04 in WSL2 and start in `~/lab` unless stated.

Task 1: Navigate and get help

Task: Print your working directory, move to `/etc` using an absolute path, then back to your home with a shortcut. List all files in `/etc` including hidden ones in long format, find the man page section for the `passwd` file format, and list all man pages whose description mentions "archive".

Solution:

```
pwd
```

```
cd /etc
cd ~            # or just: cd
ls -la /etc | head
man -f passwd   # same as whatis passwd: shows passwd(1) and passwd(5)
man 5 passwd    # file format page; q to quit
man -k archive  # same as apropos archive
type cd ls      # builtin vs file on disk
ls --help | head -5
```

Verify:

```
man -f passwd | grep '(5)'
cd /etc && pwd && cd - >/dev/null && pwd
```

Task 2: Create, copy, move and delete with globbing

Task: In `~/lab/proj`, create directories `src`, `docs` and `tmp/a/b` in one command. Create `report1.txt` to `report5.txt` in `docs`. Copy all reports ending in an odd digit to `tmp`, rename `docs/report5.txt` to `docs/final.txt`, then delete `tmp` and everything in it.

Solution:

```
mkdir -p ~/lab/proj/{src,docs,tmp/a/b}
cd ~/lab/proj
touch docs/report{1..5}.txt
cp docs/report[135].txt tmp/
ls tmp
mv docs/report5.txt docs/final.txt
rm -r tmp
```

Verify:

```
ls ~/lab/proj/docs     # final.txt report1.txt report2.txt report3.txt report4.txt
test -d ~/lab/proj/tmp && echo "tmp still exists" || echo "tmp removed"
```

Task 3: Search and extract data

Task: Using `/etc/passwd`, count how many accounts use `/bin/bash` as their shell, print only the usernames and home directories of those accounts sorted alphabetically, and show the 5 most common login shells with counts.

Solution:

```
grep -c '/bin/bash$' /etc/passwd
grep '/bin/bash$' /etc/passwd | cut -d: -f1,6 | sort
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn | head -5
```

Verify:

```
awk -F: '$7 == "/bin/bash"' /etc/passwd | wc -l    # must match the grep -c result
```

Task 4: Pipes and redirection

Task: Run `ls` on `/etc` and on a directory that does not exist. Send normal output to `out.txt`, errors to `err.txt`. Then run it again appending both streams to `all.log`. Finally, list `/usr/bin`, show it on screen and save a copy to `bin.txt` at the same time, and discard all output from `find / -name shadow`.

Solution:

```
cd ~/lab
ls /etc /nope > out.txt 2> err.txt
ls /etc /nope >> all.log 2>&1
ls /usr/bin | tee bin.txt | wc -l
find / -name shadow > /dev/null 2>&1
```

Verify:

```
cat err.txt          # contains "No such file or directory"
grep -c nope all.log # 1
wc -l bin.txt
```

Task 5: Basic regular expressions

Task: Create a file with sample log lines and use `grep` to show: lines that start with `ERROR`, lines ending in a 3-digit number, lines that do not contain `DEBUG`, and case-insensitive matches for `timeout`.

Solution:

```
cd ~/lab
cat > app.log <<'EOF'
INFO server started on port 3000
DEBUG cache warmup
ERROR db connection Timeout after 500
WARN slow query 120
error lowercase message
ERROR payment timeout code 504
EOF
grep '^ERROR' app.log
grep -E '[0-9]{3}$' app.log
grep -v 'DEBUG' app.log
grep -i 'timeout' app.log
grep -E '^(ERROR|WARN)' app.log | wc -l
```

Verify:

```
grep -c '^ERROR' app.log        # 2
grep -cE '[0-9]{3}$' app.log    # 3
grep -ci 'timeout' app.log      # 2
```

Task 6: Archive and compress

Task: Create a gzip-compressed tar archive of `~/lab/proj`, list its contents without extracting, extract it into `/tmp/restore`, then create a bzip2 and an xz version and a zip file. Compare sizes.

Solution:

```
cd ~/lab
tar -czf proj.tar.gz -C ~/lab proj
tar -tzf proj.tar.gz
mkdir -p /tmp/restore && tar -xzf proj.tar.gz -C /tmp/restore
tar -cjf proj.tar.bz2 -C ~/lab proj
tar -cJf proj.tar.xz -C ~/lab proj
zip -r proj.zip proj
gzip -k app.log     # -k keeps the original
ls -lh proj.* app.log*
```

Verify:

```
ls /tmp/restore/proj/docs
file proj.tar.gz proj.tar.bz2 proj.tar.xz proj.zip
unzip -l proj.zip | tail -1
gunzip -t app.log.gz && echo "gzip OK"
```

Task 7: First script with arguments and exit codes

Task: Write `greet.sh` that prints "Hello, NAME" for every argument given. With no arguments, it prints a usage message to stderr and exits with status 1.

Solution:

```
cd ~/lab
cat > greet.sh <<'EOF'
#!/bin/bash
if [ $# -lt 1 ]; then
  echo "Usage: $0 NAME..." >&2
  exit 1
fi
for name in "$@"; do
  echo "Hello, $name"
done
exit 0
EOF
chmod +x greet.sh
```

Verify:

```
./greet.sh; echo "exit=$?"          # usage message, exit=1
./greet.sh Ann "Bo Li"; echo "exit=$?"  # two lines, exit=0
```

Task 8: Conditionals, tests and loops

Task: Write `check.sh` that takes a path and reports whether it is a directory, a regular file, or missing. For a directory, also print how many entries it contains. Then loop over `/etc/hosts /etc /nope` calling the script.

Solution:

```
cd ~/lab
cat > check.sh <<'EOF'
#!/bin/bash
p="$1"
if [ -d "$p" ]; then
  count=$(ls -A "$p" | wc -l)
  echo "$p is a directory with $count entries"
elif [ -f "$p" ]; then
  echo "$p is a regular file of $(wc -c < "$p") bytes"
else
  echo "$p does not exist" >&2
  exit 2
fi
EOF
chmod +x check.sh
for x in /etc/hosts /etc /nope; do ./check.sh "$x"; echo "rc=$?"; done
```

Verify:

```
./check.sh /nope 2>/dev/null; echo $?   # 2
./check.sh /etc | grep -q directory && echo PASS
```

Task 9: Backup script with rotation

Task: Write `backup.sh SRC DEST` that creates a timestamped `.tar.gz` of SRC in DEST, reports failure with a non-zero exit code, and keeps only the 3 newest archives.

Solution:

```
cd ~/lab
cat > backup.sh <<'EOF'
#!/bin/bash
SRC="${1:-$HOME/lab/proj}"
DEST="${2:-$HOME/backups}"
[ -d "$SRC" ] || { echo "No such dir: $SRC" >&2; exit 1; }
mkdir -p "$DEST"
STAMP=$(date +%Y%m%d-%H%M%S)
NAME=$(basename "$SRC")
ARCHIVE="$DEST/$NAME-$STAMP.tar.gz"
if tar -czf "$ARCHIVE" -C "$(dirname "$SRC")" "$NAME"; then
  echo "Created $ARCHIVE"
else
  echo "Backup failed" >&2
  exit 2
fi
ls -1t "$DEST/$NAME"-*.tar.gz | tail -n +4 | xargs -r rm --
EOF
chmod +x backup.sh
for i in 1 2 3 4 5; do ./backup.sh; sleep 1; done
```

Verify:

```
ls -1 ~/backups | wc -l             # 3
./backup.sh /nope; echo "rc=$?"     # rc=1
tar -tzf "$(ls -1t ~/backups/*.tar.gz | head -1)" | head -3
```

Task 10: Identify user types and account files

Task: Show your UID, GID and groups. List regular (human) accounts with UID 1000 or higher, list system accounts below 1000, show which accounts have a real login shell, and show who is logged in. Explain what is in `/etc/shadow` without printing hashes.

Solution:

```
id
whoami
awk -F: '$3 >= 1000 && $3 < 65534 {print $1, $3}' /etc/passwd
awk -F: '$3 < 1000 {print $1, $3}' /etc/passwd | head
grep -vE '(nologin|false)$' /etc/passwd | cut -d: -f1,7
```

```
who; w
ls -l /etc/passwd /etc/shadow /etc/group
sudo cut -d: -f1,3,5 /etc/shadow | head -3   # user, last change, max days
```

Verify:

```
id -u root                 # 0
stat -c '%a %U:%G' /etc/shadow   # 640 root:shadow on Ubuntu
```

Task 11: Create and manage users and groups

Task: Create group `devs`. Create users `alice` and `bob` with home directories and bash as shell; add both to `devs` as a supplementary group. Set a password for `alice`, lock `bob`, then delete `bob` including his home directory.

Solution:

```
sudo groupadd devs
sudo useradd -m -s /bin/bash -G devs alice
sudo useradd -m -s /bin/bash bob
sudo usermod -aG devs bob      # -a is vital: without it other groups are replaced
sudo passwd alice              # type a lab-only password
sudo passwd -l bob
sudo passwd -S bob             # status shows L (locked)
sudo userdel -r bob
```

Verify:

```
getent group devs              # devs:x:GID:alice
id alice
getent passwd bob || echo "bob removed"
ls -ld /home/alice
```

Task 12: Permissions with chmod and umask

Task: In `~/lab/perm`, create `script.sh`, `notes.txt` and `secret.txt`. Make `script.sh` 755 using symbolic mode, `notes.txt` 644 using octal, and `secret.txt` readable and writable by the owner only. Then set umask 027, create a new file and directory, and predict their modes.

Solution:

```
mkdir -p ~/lab/perm && cd ~/lab/perm
touch script.sh notes.txt secret.txt
chmod u=rwx,go=rx script.sh
chmod 644 notes.txt
chmod 600 secret.txt
umask 027
touch newfile; mkdir newdir
umask 022      # restore the default for the session
```

Verify:

```
stat -c '%a %n' script.sh notes.txt secret.txt newfile newdir
# 755 script.sh / 644 notes.txt / 600 secret.txt / 640 newfile / 750 newdir
```

Task 13: Ownership and a shared group directory

Task: Create `/srv/team` owned by root with group `devs`. Members of `devs` must be able to create files there, other users must have no access, and new files must inherit the `devs` group. Change the owner of one file to `alice`.

Solution:

```
sudo mkdir -p /srv/team
sudo chown root:devs /srv/team
sudo chmod 2770 /srv/team        # 2 = SGID on a directory
sudo -u alice touch /srv/team/plan.txt
sudo touch /srv/team/root-made.txt
sudo chown alice /srv/team/root-made.txt
```

Verify:

```
ls -ld /srv/team                 # drwxrws--- root devs
sudo ls -l /srv/team             # both files show group devs
sudo -u nobody ls /srv/team; echo "rc=$?"   # Permission denied, rc=2
```

Task 14: Special permissions and links

Task: Show the permission bits that make `/tmp` safe to share and let `passwd` change `/etc/shadow`. Create a sticky world-writable directory of your own. Create a hard link and a symbolic link to a file, show their inodes, then delete the original and observe which link still works.

Solution:

```
ls -ld /tmp                  # drwxrwxrwt: t = sticky bit
ls -l /usr/bin/passwd        # -rwsr-xr-x: s = SUID
sudo mkdir /srv/drop && sudo chmod 1777 /srv/drop
cd ~/lab && echo "data" > orig.txt
ln orig.txt hard.txt
ln -s orig.txt soft.txt
ls -li orig.txt hard.txt soft.txt
rm orig.txt
cat hard.txt; cat soft.txt
```

Verify:

```
stat -c '%a %n' /srv/drop     # 1777
ls -l soft.txt                # broken link, shown in red with ls --color
find ~/lab -xtype l           # lists soft.txt as a dangling symlink
```

Task 15: Processes and job control

Task: Start two background `sleep` processes, one at a lower priority. List your jobs, show PID, parent PID, niceness and command for them, bring one to the foreground and stop it, resume it in the background, then kill both. Show memory and load average.

Solution:

```
sleep 600 &
nice -n 10 sleep 700 &
jobs -l
ps -o pid,ppid,ni,stat,cmd -C sleep
fg %1        # then press Ctrl+Z to stop it
bg %1
pgrep -a sleep
kill %1
pkill -f 'sleep 700'
free -h
uptime
top -b -n 1 | head -15
```

Verify:

```
pgrep sleep || echo "no sleep processes left"
cat /proc/loadavg
```

Task 16: Where data is stored

Task: Find the CPU model and core count, total RAM, disk usage by filesystem, the size of `/var/log`, the kernel version, and the last 10 system log lines. Identify block devices.

Solution:

```
lscpu | grep -E 'Model name|^CPU\(s\)'
grep -c ^processor /proc/cpuinfo
grep MemTotal /proc/meminfo
df -h
sudo du -sh /var/log
uname -r
lsblk
sudo journalctl -n 10 --no-pager   # needs systemd (default on current WSL Ubuntu)
sudo dmesg | tail -5
ls /var/log
```

Verify:

```
nproc                       # equals the processor count above
cat /proc/version
```

Task 17: Your computer on the network

Task: Show your IP addresses, default gateway and DNS configuration. Resolve a hostname, add a local name to `/etc/hosts`, test connectivity, and list listening TCP and UDP ports.

Solution:

```
ip addr show
ip route show default
cat /etc/resolv.conf
dig +short netacad.com        # or: host netacad.com
echo '127.0.0.1 devbox.local' | sudo tee -a /etc/hosts
getent hosts devbox.local
ping -c 3 devbox.local
ss -tuln
python3 -m http.server 8080 >/dev/null 2>&1 &
ss -tln | grep 8080
kill %%
```

Verify:

```
getent hosts devbox.local        # 127.0.0.1 devbox.local
ip -4 route | grep default       # shows "default via <gateway>"
```

Task 18: Packages, licenses and distributions

Task: Identify your distribution and release, find which package owns `/bin/ls`, list installed packages whose name contains `ssh`, install `tree`, and find the license text shipped with Ubuntu for the GPL version 3.

Solution:

```
cat /etc/os-release
dpkg -S /bin/ls                 # coreutils
dpkg -l | grep ssh
apt show tree 2>/dev/null | head -8
sudo apt -y install tree
ls /usr/share/common-licenses/
head -5 /usr/share/common-licenses/GPL-3
less /usr/share/doc/coreutils/copyright
```

Verify:

```
grep '^VERSION_ID' /etc/os-release
tree -L 1 ~/lab | tail -1
dpkg -s tree | grep Status      # install ok installed
```

Task 19: Capstone challenge

Task: Without looking at the earlier solutions: create group `ops` and user `carol` in it; create `/srv/logs-archive` owned by carol:ops with mode 2750; write `archive-logs.sh` that finds `*.log` files in a given directory older than N days (default 0 for the lab), archives them into a dated `.tar.gz` in `/srv/logs-archive`, prints how many files it archived, and exits 1 if none were found. Run it against `~/lab`.

Solution:

```
sudo groupadd ops
sudo useradd -m -s /bin/bash -g ops carol
sudo mkdir -p /srv/logs-archive
sudo chown carol:ops /srv/logs-archive
sudo chmod 2750 /srv/logs-archive
cat > ~/lab/archive-logs.sh <<'EOF'
#!/bin/bash
DIR="${1:?usage: $0 DIR [DAYS]}"
DAYS="${2:-0}"
OUT="/srv/logs-archive/logs-$(date +%F-%H%M%S).tar.gz"
# "not newer than N days ago" = at least N days old; N=0 means every existing file
mapfile -t files < <(find "$DIR" -maxdepth 1 -type f -name '*.log' \
  ! -newermt "$DAYS days ago")
if [ "${#files[@]}" -eq 0 ]; then
  echo "No log files found" >&2
  exit 1
fi
tar -czf "$OUT" "${files[@]}" && echo "Archived ${#files[@]} file(s) to $OUT"
EOF
chmod +x ~/lab/archive-logs.sh
```

```
sudo ~/lab/archive-logs.sh ~/lab 0
```

For real use you would run it from cron with a DAYS value such as 7. `-mtime +7` is the classic alternative, but note it means "more than 7 full days", which is why the lab uses `-newermt`.

Verify:

```
ls -ld /srv/logs-archive                  # drwxr-s--- carol ops
sudo ls -l /srv/logs-archive
sudo tar -tzf "$(sudo ls -1t /srv/logs-archive/*.tar.gz | head -1)"
sudo ~/lab/archive-logs.sh /etc 0; echo "rc=$?"   # rc=1, no *.log in /etc top level
```

Practice questions

1. Which command shows the manual page for the format of the `/etc/passwd` file rather than the `passwd` command? A) `man passwd` B) `man 5 passwd` C) `help passwd` D) `info 1 passwd`
2. Short answer: what single command searches the man page descriptions for the keyword "compress"?
3. You are in `/var/log/nginx`. Which path is relative? A) `/var/log` B) `~/logs` C) `../apt` D) `/home/rizwan`
4. Which command creates `a/b/c` in one step, creating parents as needed? A) `mkdir a/b/c` B) `mkdir -p a/b/c` C) `mkdir -r a/b/c` D) `touch -d a/b/c`
5. What does `ls -a` show that plain `ls` does not?
6. Which command copies the directory `site` and everything inside it to `backup`? A) `cp site backup` B) `mv -r site backup` C) `cp -r site backup` D) `cp -a site`
7. After `cmd > out.txt 2>&1`, where do error messages go?
8. Which command prints only lines of `app.log` that do NOT contain `DEBUG`? A) `grep -i DEBUG app.log` B) `grep -v DEBUG app.log` C) `grep -c DEBUG app.log` D) `grep ^DEBUG app.log`
9. In a basic regular expression, what do `^` and `$` match?
10. Which pipeline counts unique values in the 7th colon-separated field of `/etc/passwd`? A) `cut -f7 /etc/passwd | uniq | wc -l` B) `cut -d: -f7 /etc/passwd | sort -u | wc -l` C) `sort /etc/passwd | cut -d: -f7` D) `wc -l /etc/passwd`
11. Which `tar` command extracts `site.tar.xz` into `/tmp/x`? A) `tar -czf site.tar.xz /tmp/x` B) `tar -xJf site.tar.xz -C /tmp/x` C) `tar -tJf site.tar.xz /tmp/x` D) `tar -xzf site.tar.xz -d /tmp/x`
12. Short answer: which `tar` option lists an archive's contents without extracting?
13. What does `gzip file.log` do to the original `file.log` by default?
14. What is the purpose of the first line `#!/bin/bash` in a script?
15. Inside a script, what do `$#`, `$1` and `$?` hold?
16. A script runs `exit 3`. How can the caller see that value immediately afterwards?
17. What must you do before running `./deploy.sh` if it gives "Permission denied"?
18. Which test checks that `/etc/app` exists and is a directory? A) `[ -f /etc/app ]` B) `[ -e /etc/app ]` C) `[ -d /etc/app ]` D) `[ -x /etc/app ]`
19. What octal mode corresponds to `rwxr-x---`?
20. With a umask of `027`, what permissions does a newly created regular file get?
21. What does `chmod g+s shared/` do on a directory?
22. Why can every user write to `/tmp` but not delete other users' files there?
23. What is the effect of the `s` in `-rwsr-xr-x` on `/usr/bin/passwd`?
24. Which command adds `alice` to the `docker` group while keeping her other supplementary groups? A) `usermod -G docker alice` B) `usermod -aG docker alice` C) `groupadd docker alice` D) `useradd -G docker alice`

25. Which file stores password hashes, and who can read it?
26. On Ubuntu, what UID range is normally used for regular human accounts?
27. You delete a file that has one hard link and one symbolic link pointing to it. Which link still shows the data?
28. Which command shows every process on the system with its parent PID? A) `jobs` B) `ps -ef` C) `top -p 1` D) `pgrep`
29. What does `nice -n 10 cmd` do, and can a normal user use `nice -n -5`?
30. Which command sends the default termination signal to process 4242, and what is that signal?
31. Which directory holds virtual files describing running processes and kernel state? A) `/etc` B) `/var` C) `/proc` D) `/opt`
32. Which command shows the default gateway on a modern Linux system? A) `ifconfig -g` B) `ip route show default` C) `ss -r` D) `hostname -g`
33. Which file contains local hostname-to-IP overrides checked before DNS on most systems?
34. Which license is copyleft: MIT, BSD 2-clause, Apache 2.0, or GPLv3?
35. Which distribution family uses `.rpm` packages: Debian/Ubuntu or RHEL/Fedora/Rocky?

Answers

1. B. Section 5 holds file formats; `man passwd` opens section 1 (the command) first.
2. `man -k compress` (equivalent to `apropos compress`); it searches the whatis database.
3. C. `..` is relative to the current directory; ~ expands to an absolute home path.
4. B. `-p` creates missing parent directories and does not fail if they exist.
5. Hidden entries whose names start with a dot, plus `.` and `..` (`-A` omits those two.)
6. C. `cp` needs `-r` (or `-a`) to copy directories; `mv` has no `-r`.
7. Into `out.txt` as well: stdout goes to the file first, then stderr is pointed at stdout.
8. B. `-v` inverts the match; `-i` ignores case and `-c` counts.
9. `^` anchors the start of a line, `$` anchors the end.
10. B. `uniq` only removes adjacent duplicates, so sort first; `-d:` sets the delimiter.
11. B. `-x` extract, `-J` xz, `-f` file, `-C` change to the target directory first.
12. `-t` (for example `tar -tzf file.tar.gz`).
13. Replaces it with `file.log.gz`; the original is removed unless you use `-k`.
14. The shebang tells the kernel which interpreter to run the script with.
15. Number of arguments, the first argument, and the exit status of the last command.
16. `echo $?` right after the script finishes, before running anything else.
17. Add execute permission, for example `chmod u+x deploy.sh` (or run `bash deploy.sh`).
18. C. `-d` is true only for directories; `-e` is true for any existing path.
19. 750: rwx = 7, r-x = 5, --- = 0.
20. 640 (rw-r-----): files start from 666, and 027 removes group write and all other bits.
21. Sets SGID: new files created inside inherit the directory's group.
22. `/tmp` is mode 1777; the sticky bit limits deletion to the file owner, dir owner or root.
23. SUID: the program runs with the file owner's (root's) privileges, so it can update shadow.

24. B. Without `-a`, `-G` replaces the whole supplementary group list.
25. `/etc/shadow`; only root and (on Ubuntu) members of the `shadow` group can read it.
26. 1000 and above (up to 60000 by default); system accounts use lower UIDs, root is 0.
27. The hard link: it points to the same inode. The symlink points to a name that is gone.
28. B. `-e` selects all processes, `-f` adds full format including PPID.
29. Starts cmd with lower priority (niceness 10); negative values need root.
30. `kill 4242` sends SIGTERM (15); `kill -9` would send SIGKILL.
31. C. `/proc` is a virtual filesystem generated by the kernel.
32. B. `ifconfig` is legacy and has no `-g`; `ss` shows sockets, not routes.
33. `/etc/hosts` (order is set by the `hosts:` line in `/etc/nsswitch.conf`).
34. GPLv3: derived works must be released under the same license. The others are permissive.
35. RHEL/Fedora/Rocky use RPM (`dnf`, `rpm`); Debian/Ubuntu use `.deb` (`apt`, `dpkg`).

Exam-day tips

NetAcad course exams (for the Credly badges):

- The exams are taken online inside the course in your browser; they are not proctored. Official pages I could load do not publish a time limit, question count or number of attempts for the Unhatched or Essentials finals: unverified. Community answers say only the first attempt of the final counts toward the badge and the pass mark is 70%; plan on getting it right the first time.
- Do all chapter exams and the midterm first; the comprehensive final draws on the same pool of topics.
- Some items are "choose two/three" or drag-and-drop; read the instruction line before answering.
- Use a stable connection and a desktop browser; do not open the exam on a phone. Submit before closing the tab.
- After you pass, check the email tied to your NetAcad account for the Credly invitation and accept it; badge issue can take a little time.

LPI 010-160 (optional, paid):

- Create an LPI ID at lpi.org first; you need it to schedule with Pearson VUE.
- OnVUE online: quiet private room, webcam, stable internet, clear desk, and run the Pearson system test beforehand. LPI states the OnVUE interface and proctors are English only.
- Bring government-issued photo ID that matches your registration name exactly (Pearson VUE standard rule; check the confirmation email for specifics).
- No notes, man pages or Linux shell during the exam; everything is from memory.
- 40 questions in 60 minutes is about 90 seconds each. Do one pass answering what you know, flag the rest, come back.
- LPI lists the format as multiple choice. Older LPI exams also used fill-in-the-blank items, so know exact command names and option spellings (unverified for the current version).
- Scaled score: 500 of 800 to pass. You get the result immediately at the end.

Common traps:

- `usermod -G` without `-a` silently removes other groups.
- `tar` compression letters: `z` gzip, `j` bzip2, `J` xz. `-C` goes before the directory argument when creating, and sets the target when extracting.

- `2>&1 > file` (wrong order) still prints errors to the screen; `> file 2>&1` is correct.
- `uniq` without `sort` misses non-adjacent duplicates.
- Default permissions come from 666 (files) and 777 (dirs) minus the umask; files never get execute by default.
- Deleting a symlink's target breaks the symlink; a hard link keeps the data alive.
- Licensing questions: "free" means freedom, not price; GPL is copyleft, MIT/BSD/Apache are permissive.
- Theory topics you rarely touch as a developer: hardware terms (RAM vs storage, `/dev/sda` vs `/dev/nvme0n1`), and privacy/ICT basics in Topic 1.4.

Readiness checklist

- ■ NetAcad account created and both courses enrolled (self-paced)
- ■ WSL2 Ubuntu 24.04 lab built and exported as a clean snapshot
- ■ Linux Unhatched final passed and Credly badge accepted
- ■ Linux Essentials chapter exams and midterm done with 85%+
- ■ Tasks 1-6 done (navigation, files, grep, redirection, regex, archiving)
- ■ Tasks 7-9 done (scripts with arguments, tests, loops, exit codes, backups)
- ■ Tasks 10-14 done (user types, users/groups, chmod/umask, SGID dir, links)
- ■ Tasks 15-18 done (processes, hardware/storage, networking, packages/licenses)
- ■ Task 19 capstone done from memory
- ■ 30+ of 35 practice questions right without looking at answers
- ■ Can explain GPL vs MIT, deb vs rpm, LTS vs rolling in one sentence each
- ■ Linux Essentials comprehensive final passed and Credly badge accepted
- ■ Both badges added to LinkedIn and CV
- ■ Decision made on the paid LPI 010-160 (skip, or book OnVUE after week 4)
- ■ If booking LPI: LPI ID created, OnVUE system test passed, ID ready

Sources

- https://www.netacad.com/courses/linux-unhatched (course page: free, 8 hours, beginner, self-paced, languages, NDG)
- https://www.netacad.com/courses/linux-essentials (course page: free, 70 hours, intermediate, 18 labs, self-paced, LPI alignment, course badge)
- https://www.netacad.com/courses/linux-1 (paid follow-on course, LPIC-1 track)
- https://www.credly.com/org/cisco/badge/linux-unhatched (badge description, skills, earning criteria)
- https://www.credly.com/org/cisco/badge/linux-essentials (badge description, skills, earning criteria, type and level)
- https://www.netdevgroup.com/online/courses/open-source/linux-essentials (NDG course structure, exams, 010-160 alignment, coupon note)
- https://www.netdevgroup.com/online/courses/open-source/linux-unhatched (NDG Unhatched outline)
- https://learningnetwork.cisco.com/s/question/0D56e0000CCkHGtCQN/would-i-be-given-a-credly-badge-upon-completion-of-cisco-network-essentials-selfpaced-via-skillsforall (community source for the 70% / first attempt claim; unverified)

- https://www.lpi.org/our-certifications/linux-essentials-overview/ (exam code, version, format, delivery, validity, languages)
- https://www.lpi.org/our-certifications/exam-010-objectives/ (objectives and weights)
- https://www.lpi.org/exam-pricing/ (USD 120 / 100 / 80, EUR 110)
- https://www.lpi.org/about-lpi/frequently-asked-questions/ (200-800 scale, pass at 500, OnVUE requirements)
- https://www.lpi.org/policies/ (retake waiting periods)
- https://www.pearsonvue.com/us/en/lpi.html (LPI ID requirement, 010-160 details)
- https://learning.lpi.org (free official LPI study materials)
- https://learn.microsoft.com/windows/wsl/install (WSL2 installation)